北京伟景智能科技有限公司

VzNLSDK（激光线检测）使用文档

文件名称：VzNLSDK 使用文档	**文件编号：VIZUM/SY-SW(20220808)-001**
生效日期：发布日期起	**文件页数：160 页**
发文类型：使用文档	**版本号/修订日期：2026 年 05 月 7 日**
发送部门：软件部	**发布日期：2022 年 08 月 08 日**

目录

1 相机工作流程 3
1.1 整体工作流程 3
1.2 参数配置流程 3
1.3 相机采图流程 4
1.4 相机检测流程 4
1.5 相机事件注册流程 5
2 编程指导 5
2.1 伟景 SDK 库简介 5
2.2 Windows 编译平台搭建 6
3 VzNLSDK 详细接口 12
3.1 VzNLSDK 基础接口 12
3.1.1 初始化接口（3.4.8 有修改） 12
3.1.2 添加设备 IP 列表 12
3.1.3 重新搜索设备（3.4.8 有修改） 12
3.1.4 获取所有搜索的设备 13
3.1.5 获取所有打开的设备 13
3.1.6 打开设备 13
3.1.7 关闭设备 15
3.1.8 版本信息 16
3.1.9 获取设备信息 16
3.1.10 获取图像 17
3.1.11 获取错误信息 18
3.1.12 设置图像输出格式 18
3.1.13 连续取图 18
3.1.14 设置 Log 等级 19
3.1.15 设置设备状态提醒 20
3.1.16 重连设备 20
3.1.17 启用所见即所得 ROI 图像（图像矫正） 20
3.1.18 重启相机 21
3.1.19 获取 SDK 内存占用大小 21
3.1.20 查询设备是否被占用 21
3.1.21 关闭 SDK 21
3.1.22 获取相机瞳距信息 22
3.1.23 开启快速采图模式 22
3.1.24 关闭快速采图模式 22
3.1.25 获取左右目分辨率 22
3.1.26 配置检测物体环境参数 22
3.1.27 获取检测物体环境参数 22
3.1.28 获取配置检测物体环境参数范围 22
3.1.29 获取温度信息 23
3.1.30 启用无限制帧率 23
3.1.31 设置是否输出 ROI 图像 23
3.1.32 获取双目固定视差 23

3.1.33 使能双目/RGB Map 本地缓存/加载功能，提升设备连接速度 23
3.1.34 获取是否使能双目/RGB Map 本地缓存/加载功能 23
3.1.35 配置实时采图的模式 23
3.1.36 获取实时采图模式 24
3.1.37 是否支持设置采图模式 24
3.1.38 设置焊缝跟踪功能需要的相机配置参数信息 24
3.1.39 获取焊缝跟踪功能的相机配置参数信息 24
3.1.40 配置当前相机为焊缝跟踪状态 24
3.1.41 恢复当前相机为焊缝跟踪前的初始状态：按备份的参数恢复相机状态 24
3.1.42 获取焊缝跟踪用户 ROI 尺寸 24
3.1.43 获取相机当前时间戳(单位 微妙) 25
3.1.44 是否支持扫描传图信号功能 25
3.1.45 启用扫描传图信号功能 25
3.1.46 获取是否启用扫描传图信号功能 25
3.2 VzNLSDK 基础配置接口 25
3.2.1 检测区域设置 25
3.2.2 设置/获取眼睛曝光值 26
3.2.3 设置/获取相机增益 27
3.2.4 设置 Trigger 模式 27
3.2.5 获取当前 Trigger 模式 27
3.2.6 设置触发极性模式 27
3.2.7 设置触发分频 27
3.2.8 是否忽略外部使能 28
3.2.9 启用外部信号使能 28
3.2.10 生成软外部使能信号 28
3.2.11 获取参数设置范围 28
3.2.12 是否启用同步 RGB 设置 29
3.2.13 是否启用低功耗模式 29
3.2.14 停流后等待多长时间进入低功耗模式 单位：秒 29
3.2.15 设置高斯模式 29
3.2.16 设置触发输出控制模式 29
3.2.17 生成一个触发输出信号 30
3.2.18 是否支持高帧率下降沿模式 30
3.2.19 “全局降噪”功能控制和当前状态获取接口 30
3.2.20 设置/获取扫描噪点过滤阈值接口 30
3.2.21 设置/获取点云平滑门限接口 30
3.2.22 设置/获取激光线检测模式接口 31
3.2.23 是否支持抽帧功能 31
3.2.24 设置/获取抽帧数接口 31
3.3 静态相机配置接口 31
3.3.1 设置工作范围 31
3.3.2 启用/禁用摆动机构 32
3.3.3 是否启用了摆动机构 32
3.3.4 打开/关闭激光器 32

3.3.5 是否启用了激光器.....32
3.3.6 调节激光器亮度.....32
3.3.7 获取激光器亮度.....32
3.3.8 设置角速度.....32
3.3.9 获取摆动机构角速度.....33
3.3.10 设置摆动机构起止角度.....33
3.3.11 获取电机采图起止角度.....33
3.3.12 设置当前位置为开始/结束位置.....33
3.3.13 设置当前位置位结束角度.....33
3.3.14 旋转到某个位置.....33
3.3.15 旋转到开始位置.....34
3.3.16 设置摆动机构扫描模式.....34
3.3.17 获取摆动机构扫描模式.....34
3.3.18 获取扫描机构信息.....34
3.3.19 获取当前角度.....34
3.3.20 设置停顿时间.....34
3.3.21 左移.....34
3.3.22 右移.....35
3.3.23 设置相机摆动方向.....35
3.3.24 获取相机摆动方向.....35
3.3.25 获取版本.....35
3.3.26 设置相机激光器停止位置.....35
3.3.27 获取相机激光器停止位置.....35
3.3.28 设置判断完成了一次扫描的角度阈值.....35
3.3.29 获取判断完成了一次扫描的角度阈值.....36
3.3.30 是否支持静态相机输出激光线时告知是否完成了一次扫描的功能.....36
3.3.31 查找摆动机构模块.....36
3.3.32 重启摆动模块.....36
3.4 VzNLSDK 网络眼睛配置接口.....42
3.4.1 获取设备信息.....42
3.4.2 查询网络配置信息.....43
3.4.3 解绑设备.....45
3.4.4 绑定设备.....45
3.5 VzNLSDK 激光检测接口.....46
3.5.1 开始激光检测.....46
3.5.2 结束激光检测.....47
3.5.3 激光单线检测.....48
3.5.4 获取激光线结果点的个数.....48
3.5.5 获取激光线 2D 结果.....48
3.5.6 获取激光线 3D 结果.....48
3.5.7 设置波峰波谷凹槽最小深度.....51
3.5.8 取激光凹槽数据结果的波峰点的个数.....51
3.5.9 获取激光凹槽数据结果的波谷点的个数.....51
3.5.10 获取激光凹槽 2D 结果.....51

3.5.11 获取激光凹槽 3D 结果 51
3.5.12 开始自动检测 52
3.5.13 激光线自动检测函数 54
3.5.14 设置过滤高度 57
3.5.15 获取过滤高度 58
3.5.16 启用/禁用过滤高度 58
3.5.17 是否启用过滤高度 58
3.5.18 设置杂点过滤阈值 58
3.5.19 设置偏移量 58
3.5.20 设置传送带速度值 58
3.5.21 获取传送带速度值 58
3.5.22 计算平均高度/最低高度/最高高度 59
3.5.23 设置点云处理模式 59
3.5.24 设置固定步长 59
3.5.25 配置转动轴半径 59
3.5.26 设置编码器脉冲精度 59
3.5.27 激光检测填充点模式 59
3.5.28 激光检测扫描 RGB 原图的获取 60
3.5.29 配置编码器使用参数类型【SDK3.6.20.10 新增】 70
3.5.30 配置编码器间距【SDK3.6.20.10 新增】 70
3.5.31 配置/获取激光线过滤范围 71
3.5.32 设置每次回调返回多少帧数据的数量 71
3.5.33 使能/禁止点云数据缓存功能 71
3.5.34 查找输入 2D 点对应的 3D 点坐标(需使能点云数据缓存功能，并完成扫描) 71
3.5.35 执行静态扫描点云拍照(阻塞执行)，输出完整扫描点云，如果为 RGBD 扫描则同步返回 RGB 图像 72
3.5.36 释放 VzNL_PointCloudSnapShort 输出的点云和图像数据 72
3.5.37 执行焊缝跟踪点云扫描 72
3.5.38 释放 VzNL_PointCloudSnapShort 输出的点云和图像数据 72
3.6 工具类接口 73
3.6.1 检测运行环境 73
3.6.2 创建点云后处理工具 Handle 73
3.6.3 设置 3D 点云基准激光线 73
3.6.4 设置深度图/灰度图配置参数(当生成灰度/深度图时需要配置) 73
3.6.5 启用基准线深度值计算模式(当生成灰度/深度图时需要配置) 73
3.6.6 设置输出图像的最小宽度(当生成灰度 / 深度图时需要配置) 74
3.6.7 获取点云配置 74
3.6.8 设置输出图像的最小宽度(当生成灰度 / 深度图时需要配置) 74
3.6.9 开始点云处理(目前仅支持 keResultDataType_Position) 74
3.6.10 点云处理(目前仅支持 keResultDataType_Position) 74
3.6.11 结束点云处理 74
3.6.12 销毁点云处理工具 75
3.6.13 创建图像生成工具 75
3.6.14 开始送入点云数据 75

3.6.15 送入点云数据进行处理 75
3.6.16 结束点云传送 75
3.6.17 通过图像生成工具，生成灰度图像 75
3.6.18 通过图像生成工具，生成 Tif 图像 75
3.6.19 通过 2D 位置找 3D 点 75
3.6.20 销毁图像创建工具 76
3.6.21 使用固定点云数据生成深度图 76
3.6.22 使用固定点云生成灰度图 76
3.6.23 读取图像 76
3.6.24 从 tif 文件中加载 3D 点云 76
3.6.25 将 3D 点云数据存储为 Tif 深度图 77
3.6.26 将输入的点云数据转换为指定分辨率的 2D 伪彩图像输出 77
3.6.27 将输入的点云数据转换为指定像素精度的 2D 伪彩图像输出 77
3.6.28 释放输出的图像数据 77
3.6.29 创建 RGB 点云处理工具 78
3.6.30 开始送入点云数据到 RGB 点云处理工具 78
3.6.31 送入点云数据到 RGB 点云处理工具 78
3.6.32 通过 2D 位置找 3D 点 78
3.6.33 通过 3D 点找 2D 位置 78
3.6.34 结束 RGB 点云处理 78
3.6.35 销毁 RGB 点云工处理工具 79
3.6.36 通过双目校正图 2D 坐标计算 3D 点坐标，根据当前设备内部参数计算 79
3.6.37 通过双目校正图 2D 浮点坐标计算 3D 点坐标，根据当前设备内部参数计算 79
3.6.38 输入 2D RGB 图像数据 + 扫描点云数据(静态 RGBD 扫描)，获取映射后的 2D RGB 图像数据和同分辨率的点云序列 79
3.6.39 输入左目或右目图像数据 + 扫描点云数据(静态非 RGBD 扫描)，获取映射后的 2D 左目或右目图像数据和同分辨率的点云序列 80
3.6.40 释放 VzNL_GetMappedRGBImageAndPointCloud()/VzNL_GetMappedLRImageAndPointCloud() 函数返回的图像和 3D 点数据 80
3.6.41 USB 串口继电器设备搜索 80
3.6.42 获取指定 USB 串口继电器设备的配置信息 81
3.6.43 查看指定 USB 串口继电器设备是否打开 81
3.6.44 打开指定端口的 USB 串口继电器设备，按配置参数打开指定设备 81
3.6.45 关闭指定端口的 USB 串口继电器设备 81
3.6.46 获取所有打开的 USB 串口继电器设备端口列表 81
3.6.47 获取指定 USB 继电器通道当前开关状态 81
3.6.48 打开/关闭指定继电器通道，pState 返回操作后该通道状态 81
3.6.49 获取所有继电器通道开关状态：unsigned int* pState: 低位开始，每位代表一个通道； 82
3.6.50 打开/关闭所有继电器通道，pState 返回操作后所有继电器通道状态(低位开始，每位代表一个通道) 82
3.7 VzNLSDK 图像操作接口 92
3.7.1 开始绘图 92
3.7.2 绘制直线 92

3.7.3 绘制矩形 92
3.7.4 绘制多边形 92
3.7.5 绘制圆形 93
3.7.6 绘制椭圆 93
3.7.7 结束绘图 93
3.7.8 显示图像 93
3.7.9 存储图像 93
3.7.10 压缩图片 94
3.7.11 释放压缩图片 94
3.8 文件相关接口 94
3.8.1 创建文件工具 94
3.8.2 打开文件 94
3.8.3 写入文件 94
3.8.4 关闭文件 95
3.9 RGB 相关配置接口 96
3.9.1 当前相机是否支持 RGB 96
3.9.2 获取 RGB 图像分辨率 96
3.9.3 启用/获取 RGB 功能 96
3.9.4 格式化彩色 ROI 96
3.9.5 配置 RGB ROI 97
3.9.6 启用/获取 自动白平衡 97
3.9.7 设置/获取 自动曝光状态 97
3.9.8 设置 R Value 97
3.9.9 设置 G Value 97
3.9.10 设置 B Value 97
3.9.11 设置彩色 Sensor 帧率 98
3.9.12 设置彩色 Sensor 曝光 98
3.9.13 设置彩色 sensor 增益 98
3.9.14 设置 RGBSensor 触发方式 98
3.9.15 设置彩色 Sensor 期望值 98
3.9.16 生成一个 RGB 信号 98
3.9.17 设置 CCM 99
3.9.18 启用是否输出 RGB 2D 数据 99
3.9.19 获取中间相机单帧 RGB 图像 99
3.9.20 通过 3d 点获取 RGB 图像的 2d 点坐标 99
3.9.21 是否支持 RGB HDR 功能 99
3.9.22 使能彩色 sensor hdr 功能 99
3.9.23 设置彩色 Sensor hdr 高曝光 99
3.9.24 设置彩色 Sensor hdr 低曝光 100
3.9.25 设置熔池检测区域 100
3.9.26 设置 RGBHDR 出图模式 100
3.9.27 开始实时采集 RGB 图像 100
3.9.28 停止实时采集 RGB 图像 100
3.9.29 设置熔池检测区域尺寸 100

3.10 防尘盖相关配置接口 101
3.10.1 是否支持防飞溅翻盖的功能 101
3.10.2 关闭/打开防飞溅翻盖 101
3.10.3 是否关闭了防飞溅翻盖 101
3.10.4 是否开启了防飞溅翻盖控制的功能 101
3.10.5 开启/关闭防飞溅翻盖自动控制功能 101
3.10.6 是否支持 IO 控制防飞溅翻盖的功能 101
3.10.7 开启/关闭 IO 控制防飞溅翻盖的功能 101
3.10.8 是否开启了 IO 控制防飞溅翻盖的功能 102
3.10.9 配置 IO 控制消抖参数 102
3.10.10 获取 IO 控制消抖参数 102
3.10.11 是否支持应用防飞溅翻盖大中小配置的功能 102
3.10.12 根据类型使用防飞溅翻盖配置 102
3.10.13 获取当前使用防飞溅翻盖配置的类型 102
3.11 统一内外部激光器配置接口 102
3.11.1 是否支持关闭激光器 102
3.11.2 关闭激光器 103
3.11.3 激光器是否已关闭 103
3.11.4 是否支持激光器亮度调节功能 103
3.11.5 获取外部激光器功率控制范围 103
3.11.6 配置功率等级 103
3.11.7 获取功率等级 103
3.12 手眼标定转换接口 103
3.12.1 初始化算法库 103
3.12.2 释放算法库资源 104
3.12.3 三轴标定算法函数接口 104
3.12.4 六轴标定算法函数接口：输入点坐标+三轴矢量 104
3.12.5 六轴标定算法函数接口：输入点坐标+姿态角 104
3.12.6 R2RPY 旋转矩阵转 RPY 角 105
3.12.7 R2RPY RPY 角转旋转矩阵 105
3.13 C 版激光器接口 105
3.13.1 是否支持 C 版功能 105
3.13.2 开启/关闭 C 版功能 106
3.13.3 激光器是否开启 C 版功能 106
3.13.4 点亮激光器 106
3.13.5 激光器是否已点亮 106
3.13.6 配置 C 版激光器点亮时长（微妙） 106
3.13.7 获取 C 版激光器点亮时长（微妙） 106
3.13.8 配置 C 版激光器点亮延迟时长（微妙） 106
3.13.9 获取 C 版激光器点亮延迟时长（微妙） 107
3.13.10 是否支持激光器 PWM 调节功能 107
3.13.11 配置 PWM 频率 107
3.13.12 获取 PWM 频率 107
3.13.13 配置 PWM 占空比 107

3.13.14 获取 PWM 占空比 107
3.13.15 是否支持补光灯 107
3.13.16 开启/关闭补光灯 108
3.13.17 是否开启了补光灯 108
3.13.18 设置抗反光增益 108
3.13.19 获取抗反光增益 108
3.13.20 获取 C 版激光器占空比值 108
3.13.21 开启/关闭补光灯自动控制 108
3.13.22 是否开启补光灯自动控制 108
3.13.23 配置自动控制补光灯开启后的等待时间 109
3.13.24 获取自动控制补光灯开启后的等待时间 109
3.13.25 是否支持 ch3 功能 109
3.13.26 开启/关闭 ch3 功能 109
3.13.27 激光器是否 ch3 功能 109
3.13.28 配置 ch3 频率 109
3.13.29 获取 ch3 频率 109
3.13.30 配置 ch3 占空比 110
3.13.31 获取 ch3 占空比 110
3.13.32 配置 C 版激光自动关闭的时间 110
3.13.33 获取 C 版激光自动关闭的时间 110
3.14 相机配置文件相关接口 110
3.14.1 保存用户配置到文件 110
3.14.2 恢复用户配置 110
3.14.3 读取用户文件列表 110
3.14.4 保存用户配置(共 10 个) 111
3.14.5 恢复用户配置 111
3.14.6 恢复用户配置 111
3.14.7 恢复系统默认配置 111
3.15 PetaLinux 相关接口 111
3.15.1 使能 PetaLinux 通道 111
4 接口改动记录 112
4.1 版本 3.6.20.32 112
4.1.1 添加支持 x 轴点云过滤(适用于所有相机) 112
4.1.2 移除激光器在相机上安装(适用于所有相机) 112
4.1.3 移除当前相机检测触发信号接口(适用于所有相机) 113
4.1.4 移除相机是否做过面标定接口(适用于所有相机) 113
4.1.5 移除设置激光门限值接口(适用于所有相机) 113
4.1.6 移除设置数据扫描长度,当扫描达到指定长度后，自动停止扫描接口(适用于所有相机) 113
4.1.7 移除设置网络配置信息接口(适用于所有相机) 114
4.1.8 移除获取 RGB 异步接口(适用于动态 RGBD 相机) 114
4.1.9 移除设置是否输出 ROI 图像接口(适用于所有相机) 115
4.1.10 移除多系统相关函数接口(适用于所有相机) 115
4.1.11 移除激光标定函数接口(适用于所有相机) 116
4.1.12 移除设置自动调节超时时间函数接口(适用于所有相机) 116

4.1.13 移除启用存储参数函数接口(适用于所有相机) 116
4.2 版本 3.6.20.33（没有接口变化） 117
4.3 版本 3.6.20.34 117
4.3.1 添加开启/关闭防飞溅翻盖自动控制功能(适用于静态相机) 117
4.3.2 添加舵机摆动速度范围的接口(适用于静态相机) 117
4.4 版本 3.6.20.35 117
4.4.1 添加开启/关闭快速采图模式功能(适用于静态相机) 117
4.5 版本 3.6.20.36(没有接口变化) 118
4.6 版本 3.6.20.37 118
4.6.1 修改摆动速度的数据类型由浮点型为整形(适用于静态相机) 118
4.6.2 增加瞳距数据的获取的接口(适用于所以相机) 118
4.6.3 统一动/静态激光器开关控制和亮度控制逻辑(适用于所以相机) 119
4.7 版本 3.6.20.38(没有接口变化) 120
4.8 版本 3.6.20.39 120
4.8.1 添加多帧数据一起回调的接口(适用于所以相机) 120
4.9 版本 3.6.20.40(没有接口变化) 120
4.10 版本 3.6.20.41 120
4.10.1 添加获取左右目分辨率的接口(适用于所以相机) 120
4.11 版本 3.6.20.42 120
4.11.1 添加是否支持 IO 控制相机舵机功能的接口(适用于带有翻盖的静态相机) 120
4.11.2 添加使能 IO 控制相机舵机功能的接口(适用于带有翻盖的静态相机) 120
4.11.3 添加是否使能了 IO 控制相机舵机功能的接口(适用于带有翻盖的静态相机) 121
4.11.4 添加配置/获取 IO 控制消抖参数的接口(适用于带有翻盖的静态相机) 121
4.12 版本 3.6.20.43(没有接口变化) 122
4.13 版本 3.6.20.44(没有接口变化) 122
4.14 版本 3.6.20.45 122
4.14.1 添加将输入的点云数据转换 2D 伪彩图像输出的接口(适用于带有翻盖的静态相机) 122
4.15 版本 3.6.20.46(没有接口变化) 123
4.16 版本 3.6.20.47 123
4.16.1 添加防尘盖大中小配置的功能的接口(适用于带有翻盖的静态相机) 123
4.17 版本 3.6.20.48 123
4.17.1 添加开启/关闭以左目坐标为原点来点云计算方式的接口(适用于所有相机) 123
4.17.2 添加获取左目内参的接口(适用于所有相机) 123
4.17.3 添加获取左目到 3D 点云的转换矩阵的接口(适用于所有相机) 123
4.17.4 添加获取 3D 点云到左目的转换矩阵的接口(适用于所有相机) 124
4.18 版本 3.6.20.49 124
4.18.1 添加手眼标定转换接口(适用于所有相机) 124
4.19 版本 3.6.20.50 126
4.19.1 添加激光器闪频接口(适用于所有相机) 126
4.19.2 添加激光器 PWM 功率调节接口(适用于所有相机) 127
4.19.3 添加补光灯接口(适用于所有相机) 127
4.20 版本 3.6.20.51 128
4.20.1 添加配置检测物体环境接口(适用于所有相机) 128
4.21 版本 3.6.20.52 (没有接口变化) 128

4.22 版本 3.6.20.53 128
4.22.1 添加配置抗反光增益接口(适用于所有相机) 128
4.23 版本 3.6.20.54 (没有接口变化) 128
4.24 版本 3.6.20.55 (没有接口变化) 128
4.25 版本 3.6.20.56 128
4.25.1 添加全局降噪功能配置接口(适用于所有相机) 128
4.25.2 添加 C 版激光器点亮时间占空比获取接口(适用于所有相机) 128
4.26 版本 3.6.20.57(没有接口变化) 129
4.27 版本 3.6.20.58(没有接口变化) 129
4.28 版本 3.6.20.59(没有接口变化) 129
4.29 版本 3.6.20.60(没有接口变化) 129
4.30 版本 3.6.20.61(没有接口变化) 129
4.31 版本 3.6.20.62 129
4.31.1 添加开启/关闭补光灯自动控制接口(适用于含有补光灯的相机) 129
4.31.2 添加配置自动控制补光灯开启后的等待时间接口(适用于含有补光灯的相机) 129
4.32 版本 3.6.20.63(没有接口变化) 129
4.33 版本 3.6.20.64 129
4.33.1 添加使能 PetaLinux 通道接口(适用于支持 PetaLinux 的相机) 129
4.34 版本 3.6.20.65(没有接口变化) 130
4.35 版本 3.6.20.66(没有接口变化) 130
4.36 版本 3.6.20.67 130
4.36.1 添加启用数据研所传输的接口(适用于所有相机) 130
4.37 版本 3.6.20.68 130
4.37.1 添加通过 3d 点获取 RGB 图像的 2d 点坐标的接口(适用于静态 RGBD 相机) 130
4.38 版本 3.6.20.69 130
4.38.1 添加 RGB 点云处理工具相关的接口(适用于静态 RGBD 相机) 130
4.39 版本 3.6.20.70(没有接口变化) 131
4.40 版本 3.6.20.71(没有接口变化) 131
4.41 版本 3.6.20.72(没有接口变化) 131
4.42 版本 3.6.20.73(没有接口变化) 131
4.43 版本 3.6.20.74 131
4.43.1 添加获取温度信息的接口(适用于所有相机) 131
4.44 版本 3.6.20.75 131
4.44.1 添加指定设备 IP 列表的接口(适用于所有相机) 131
4.45 版本 3.6.20.76(没有接口变化) 132
4.46 版本 3.6.20.77(没有接口变化) 132
4.47 版本 3.6.20.78 132
4.47.1 添加重启摆动模块的接口(适用于静态相机) 132
4.48 版本 3.6.20.79(没有接口变化) 132
4.49 版本 3.6.20.80(没有接口变化) 132
4.50 版本 3.6.20.81 132
4.50.1 添加设置数据扫描长度,当扫描达到指定长度后，自动停止扫描接口(适用于所有相机) 132
4.51 版本 3.6.20.82 133
4.51.1 添加设置扫描噪点过滤阈值的接口(适用于所有相机) 133

4.52 版本 3.6.20.83(没有接口变化)..... 133
4.53 版本 3.6.20.84(没有接口变化)..... 133
4.53.1 添加启用无限制帧率的接口(适用于所有相机)..... 133
4.54 版本 3.6.20.85(没有接口变化)..... 133
4.55 版本 3.6.20.86..... 133
4.55.1 添加点云数据的缓存相关接口(适用于所有相机)..... 133
4.56 版本 3.6.20.87(没有接口变化)..... 134
4.57 版本 3.6.20.88(没有接口变化)..... 134
4.58 版本 3.6.20.89(没有接口变化)..... 134
4.59 版本 3.6.20.90..... 134
4.59.1 添加获取双目固定视差接口(适用于所有相机)..... 134
4.59.2 添加通过双目校正图 2D 坐标计算 3D 点坐标接口(适用于所有相机)..... 134
4.59.3 移除 C 版功能相关控制接口(适用于所有相机)..... 135
4.60 版本 3.6.20.91(没有接口变化)..... 138
4.61 版本 3.6.20.92..... 138
4.61.1 添加使能双目/RGB Map 本地缓存/加载功能，提升设备连接速度的接口(适用于所有相机)138
4.62 版本 3.6.20.93(没有接口变化)..... 138
4.63 版本 3.6.20.94..... 138
4.63.1 添加 C 版激光相关接口(适用于所有相机)..... 139
4.64 版本 3.6.20.95..... 143
4.64.1 添加获取映射后的 2D 图像数据和同分辨率的点云序列相关接口(适用于所有相机)..... 143
4.65 版本 3.6.20.96..... 144
4.65.1 添加获取点云平滑相关接口(适用于所有相机)..... 144
4.66 版本 3.6.20.97..... 145
4.66.1 添加执行静态扫描点云拍照(阻塞执行)，输出完整扫描点云，如果为 RGBD 扫描则同步返回 RGB 图像相关接口(适用于所有相机)..... 145
4.66.2 添加 RGB HDR 功能相关接口(适用于所有相机)..... 146
4.67 版本 3.6.20.98..... 148
4.67.1 调整点云到伪彩图生成接口，增加转换信息输出功能接口(适用于所有相机)..... 148
4.67.2 增加伪彩图上的点到 3D 点云的回算接口(适用于所有相机)..... 149
4.68 版本 3.6.20.99..... 150
4.68.1 添加配置实时采图模式相关接口(适用于所有相机)..... 150
4.69 版本 3.6.20.100(没有接口变化)..... 150
4.70 版本 3.6.20.101..... 150
4.70.1 添加对焊缝跟踪点云扫描相关接口(适用于静态相机)..... 150
4.71 版本 3.6.20.102..... 151
4.71.1 移除设置数据扫描长度,当扫描达到指定长度后，自动停止扫描相关接口(适用于所以相机)151
4.71.2 移除设置数据采集间隔相关接口(适用于所有相机)..... 151
4.72 版本 3.6.20.103 (没有接口变化)..... 151
4.73 版本 3.6.20.104..... 151
4.73.1 修改设置焊缝跟踪功能需要的相机配置参数信息接口(适用于静态相机)..... 151
4.74 版本 3.6.20.105 (没有接口变化)..... 151
4.75 版本 3.6.20.106..... 151
4.75.1 添加查找摆动机构的接口(适用于静态相机)..... 151

4.76 版本 3.6.20.107 (没有接口变化) 152
4.77 版本 3.6.20.108 (没有接口变化) 152
4.78 版本 3.6.20.109 (没有接口变化) 152
4.79 版本 3.6.20.110 (没有接口变化) 152
4.80 版本 3.6.20.111 152
4.80.1 添加获取相机当前时间戳的接口(适用于所有相机) 152
4.80.2 添加获取相机 RGB 内参以及转换矩阵的接口(适用于静态相机) 152
4.81 版本 3.6.20.112 (没有接口变化) 152
4.82 版本 3.6.20.113 152
4.82.1 添加 USB 串口继电器搜索，打开，通道控制等接口接口(适用于控制气动翻盖的相机) 152
4.83 版本 3.6.20.114 (没有接口变化) 154
4.84 版本 3.6.20.115 (没有接口变化) 154
4.85 版本 3.6.20.116 154
4.85.1 添加启用传图模式的接口(适用于所有相机) 154
4.85.2 移除设置 RGBHDR 背景图叠加帧数的接口(适用于熔池相机) 154
4.85.3 移除设置 RGBHDR 背景图叠加方法的接口(适用于熔池相机) 154
4.86 版本 3.6.20.117 154
4.86.1 添加通过双目校正图 2D 浮点坐标计算 3D 点坐标，根据当前设备内部参数计算的接口(适用于所有相机) 154
4.86.2 添加焊缝跟踪两套 ROI 尺寸设置的支持的接口(适用于所有相机) 155
4.86.3 移除左目坐标原点功能的相关接口(适用于所有相机) 156
4.87 版本 3.6.20.118 156
4.87.1 添加扫描传图信号功能接口(适用于所有相机) 156
4.87.2 添加激光抽帧功能接口(适用于所有相机) 157
4.87.3 添加设置熔池检测区域尺寸的接口(适用于熔池相机) 157
4.88 版本 3.6.20.119 157
4.88.1 移除用户 ROI 尺寸配置功能接口(适用于所有相机) 157
5 示例代码 158

VzNLSDK 使用文档

修改记录

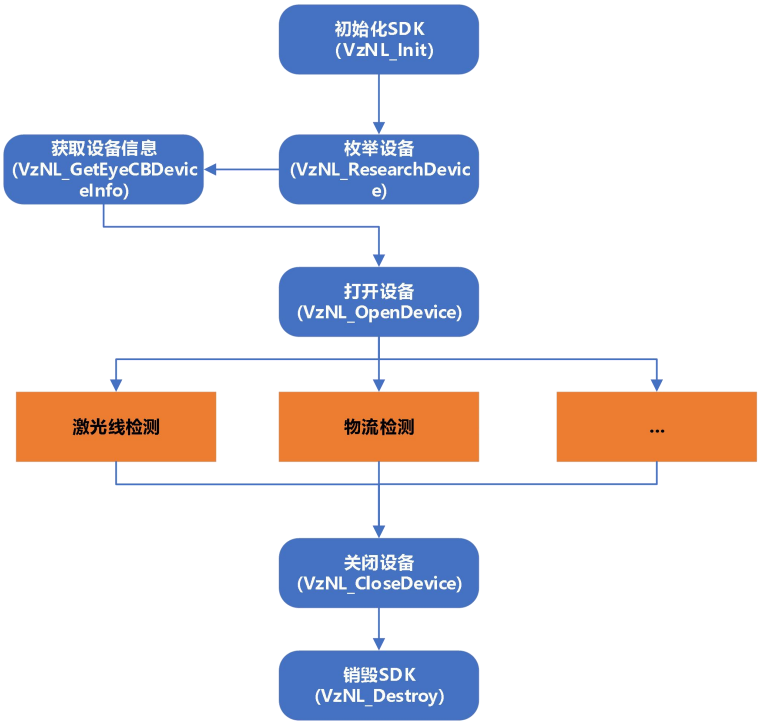
版本	时间	修改人	修改记录
V1.0.0	2019.02.13	Mjw YJ	库增加基础库，库名称修改 增加 VzKernel.dll 基础库 将 VzNL_Detect.dll 改为 VzNLDetect.dll 将 VzNL_Graphics.dll 改为 VzNLGraphics.dll 增加激光眼内容
V2.0.0	2019.07.31	Mjw	增加了 StartCapture 采图接口
V3.0.0	2019.10.17	Mjw	增加了激光线翻转的参数
V3.1.0	2019.11.16	Mjw	对应 SDK3.3.8 版本
V3.2.0	2019.12.28	Mjw	对应 3.4.0 以上版本，增加了智能眼接口
V3.2.1	2020.01.03	Mjw	增加了时间戳说明
V3.2.2	2020.02.17	Mjw	增加了 Trigger 模式 API，增加了 Idle 时可执行函数
V3.2.3	2020.02.25	Mjw	修改了 VzNL_Init 接口功能, VzNL_ResearchDevice 增加了控制搜索类型的 flag 修改了设置眼睛 IP 的接口，改为了使用 VZNLHANDLE 进行控制，以前是使用 index 号控制
V3.3.0	2020.04.20	Mjw	增加了摆动机构的控制接口支持 V3.5.0 以上
V3.3.1	2020.04.27	Mjw	支持 SDK 版本 V3.5.2 版本以上接口，修改了网络配置接口
V3.3.2	2021.04.19	Mjw	支持了 Xilinx 高帧率。 128 * 2048 = 2000FPS 256 * 2048 = 1000FPS ROI 检测最大支持 256 * 2048
V3.3.3	2021.08.13	Mjw	配合 VZNLSDK V3.6.14 更新 增加了旋转矩阵 增加了 RGBD 输出 修改了 RGB Sensor 的触发 增加了 RGB 自动曝光 增加了 RGB 自动白平衡
V3.3.4	2021.08.30	Mjw	新增扩展自动检测接口
V3.3.5	2021.09.16	Mjw	新增了 RGBD 动态相机同步
V3.3.6	2021.10.14	Mjw	增加了横向/竖向高斯的独立设置接口 增加了获取当前设备逆矩阵的接口 3D 转 2D 数据时，带入了逆矩阵的运算过程 增加了功耗配置接口
V3.3.7	2021.12.15	Mjw	增加了点填充模式
V3.3.8	2022.01.13	Mjw	增加了点云栅格化功能
V3.3.9	2022.04.26	Mjw	基于 SDK3.6.17.1 更新
V3.4.0	2022.08.08	Mjw	基于 SDK3.6.17.15 更新 增加了工作流程
V3.4.1	2023.06.02	Mjw	基于 SDK3.6.19.9 更新

			增加了控制 TriggerOut 的控制模式 增加了生成触发输出的信号 增加了启用输出 RGB 2D 点的功能
V3.4.2	2023.06.21	Mjw	基于 SDK3.6.19.13 版本更新 增加了获取扫描图像的 RGB 原图接口
V3.4.3	2023.06.25	Mjw	基于 SDK3.6.19.14 版本更新 增加了获取带颜色数据的帧率范围 Demo
V3.4.4	2023.10.25	Mjw	基于 SDK3.6.20.10 版本更新，增加了编码器参数配置和参数文件存储
V4.25	2024.08.13	YMing	VZNL_DetectConfig.h：增加“全局降噪”参数接口； VZNL_ExtStrobeLaser.h：增加 C 版激光器占空比获取接口；

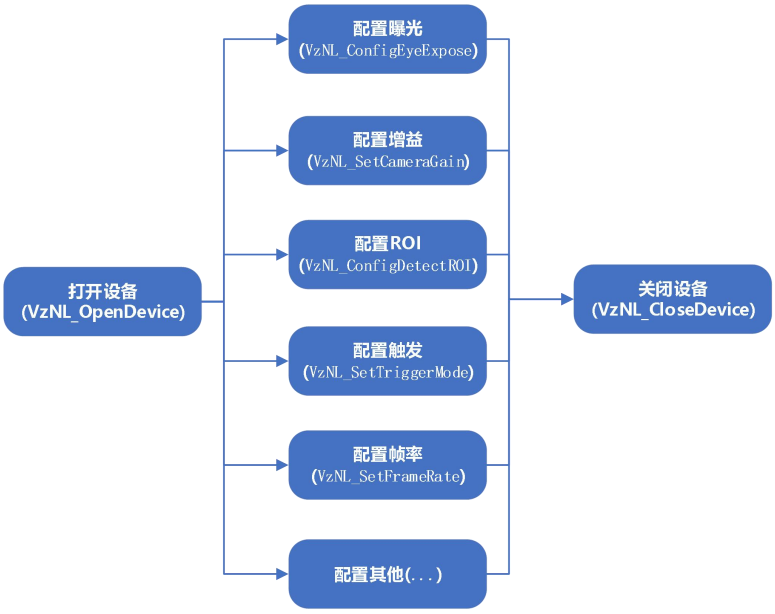
VzNLSDK 使用文档

1 相机工作流程

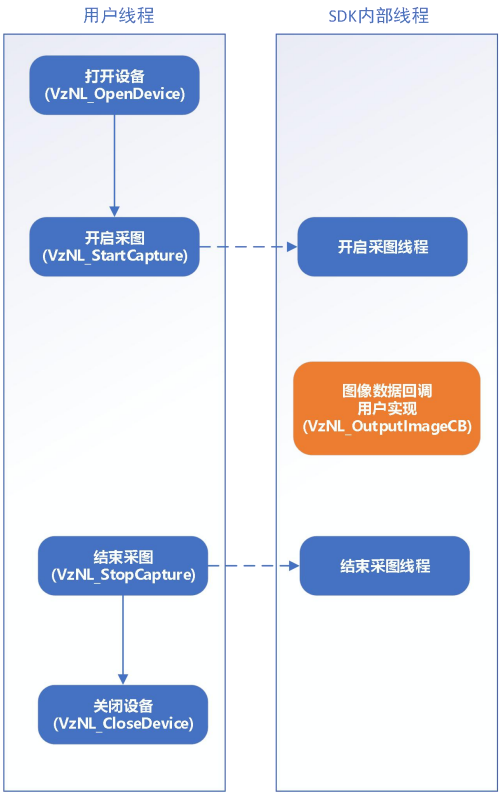
1.1 整体工作流程



1.2 参数配置流程

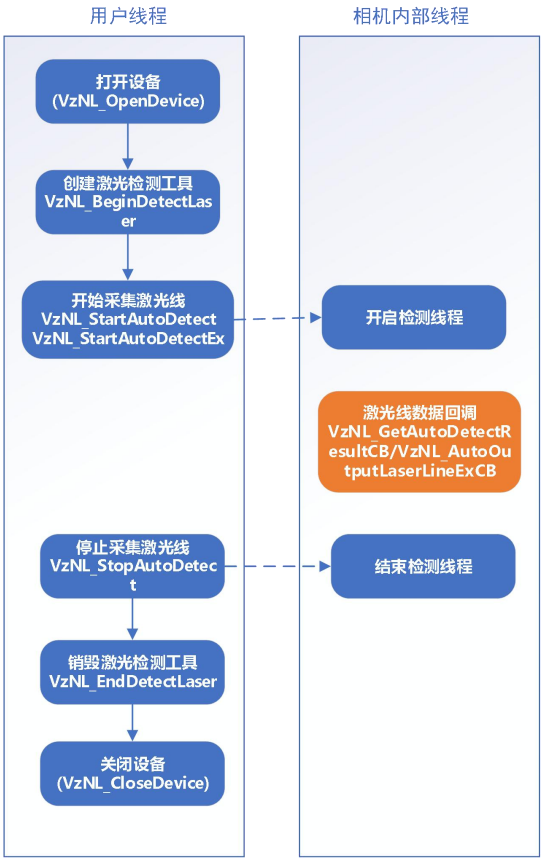


1.3 相机采图流程



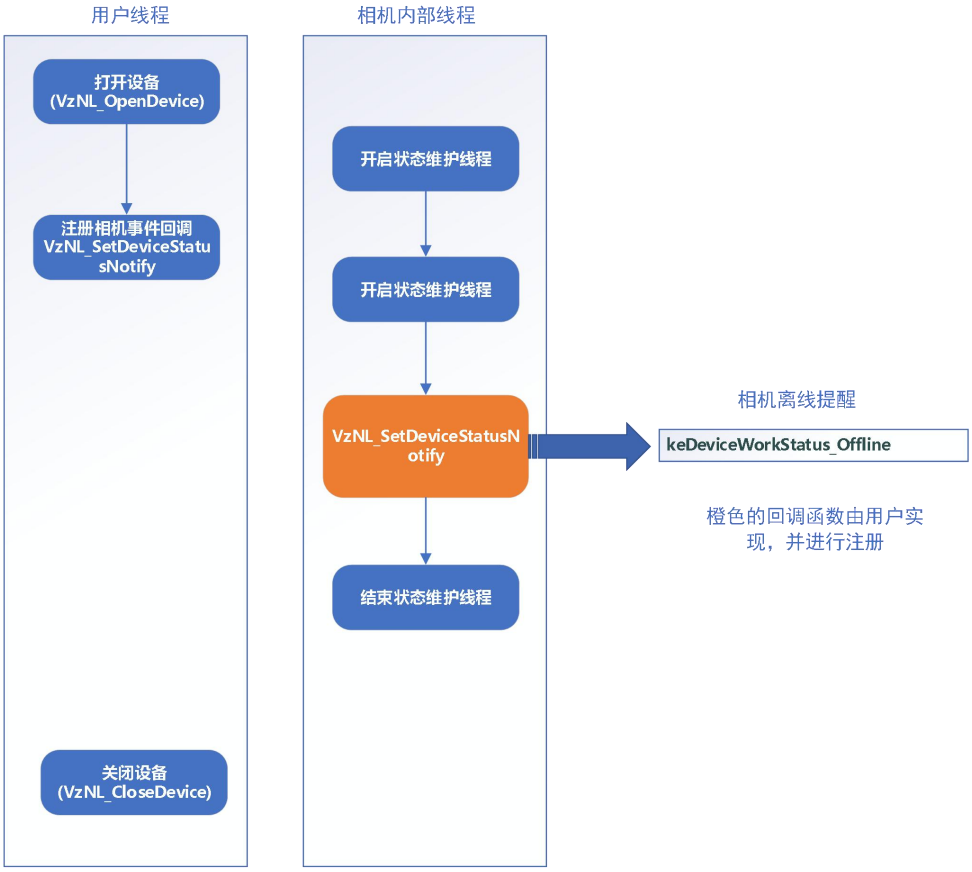
用户在实现回调函数时，请不要在回调函数中做太耗时的工作，否则可能会造成数据丢失

1.4 相机检测流程



用户在实现回调函数时，请不要在回调函数中做太耗时的工作，否则可能会造成数据丢失

1.5 相机事件注册流程



2 编程指导

2.1 伟景 SDK 库简介

伟景相机库使用 C/C++进行封装，在 Windows 上使用 Visual Studio 2015 进行编译，目前有 x86 和 x64 两个平台的版本，在 Linux 上使用 GCC 5.4 在 Ubuntu16.04 平台上进行编译，有 x86 和 x64 两个平台版本。



头文件在 SDK\VzNLSDK\Inc 目录下;

Windows 库文件在 SDK\VzNLSDK\Windows 目录下;

Linux 库文件在 SDK\VzNLSDK\Linux 目录下

头文件

头文件	头文件描述
VZNL_Export.h	SDK 输入输出宏定义
VZNL_Types.h	SDK 数据定义
VZNL_ErrorCode.h	SDK 错误码参考文件
VZNL_Common.h	SDK 公用文件
VZNL_DetectConfig.h	相机通用配置接口文件
VZNL_EyeConfig.h	相机网络配置接口文件
VZNL_FileUtils.h	SDK 文件处理接口

VZNL_Utils.h	SDK 后处理辅助接口
VZNL_RGBConfig.h	RGB 相机配置接口
VZNL_SwingMotor.h	摆动模块接口文件
VZNL_Graphics.h	绘图支持文件
VZNL_DetectLaser.h	激光检测接口文件
VZNL_DustCover.h	防飞溅翻盖控制接口文件

库文件

库名	库文件描述
libViEyeApi.dll	星光眼 支持库
VzKernel.dll	SDK 基础库
VzLog.dll	SDK Log 库
VzNLDetect.dll	SDK 检测库
VzNLGraphics.dll	SDK 绘图库

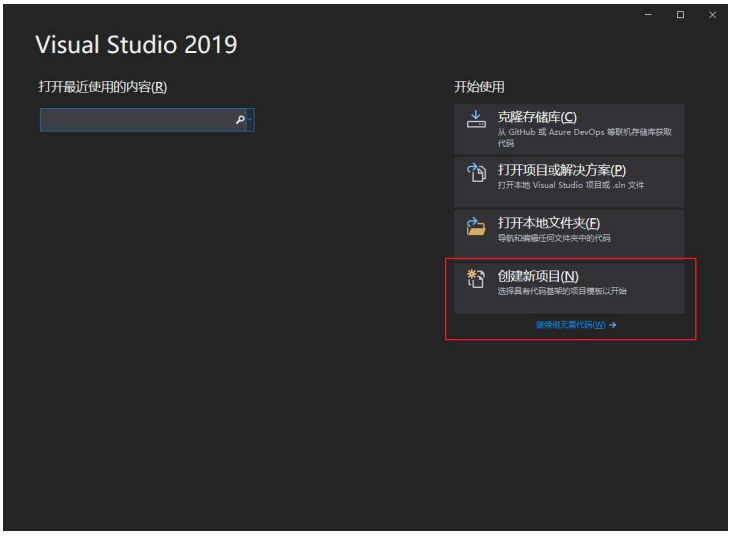
2.2 Windows 编译平台搭建

推荐开发环境: Visual Studio 2015 及以上集成开发环境。

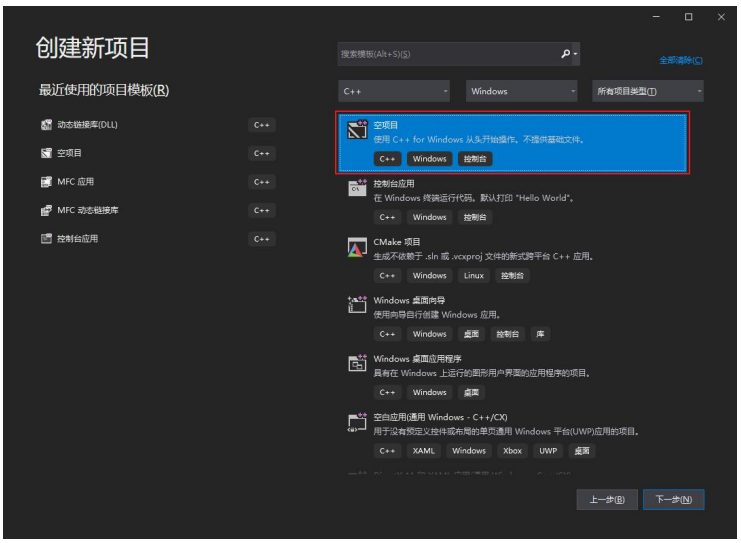
推荐开发平台: Windows10 64bit 及以上操作系统。

本文中以 Visual Studio 2019 为例进行描述

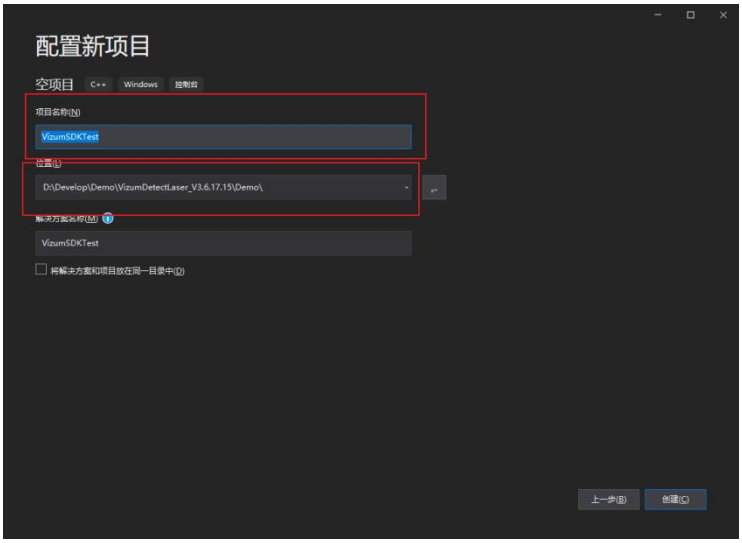
1）创建新项目



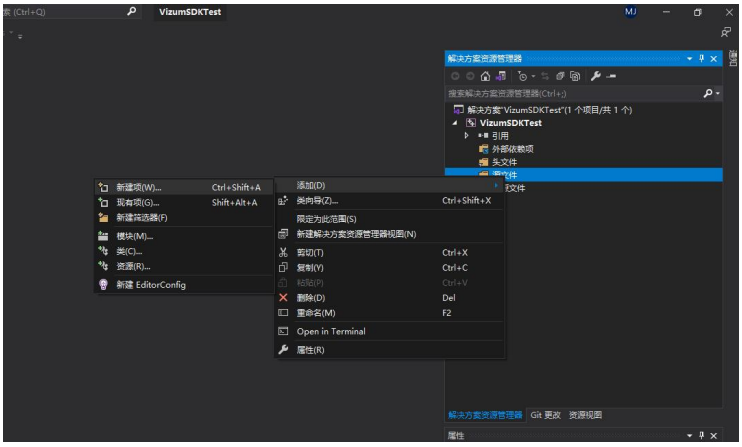
2）选择创建的项目

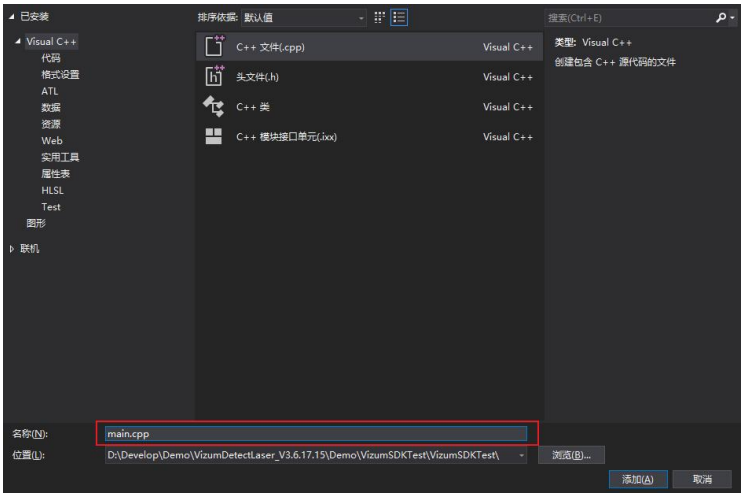


3）给要创建的项目起一个适合的名称

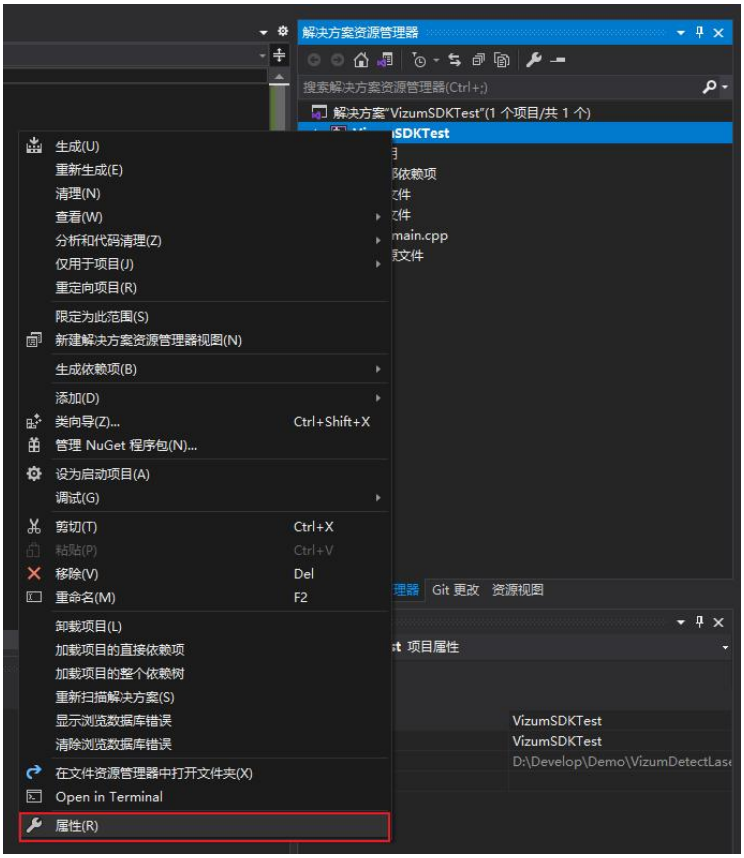


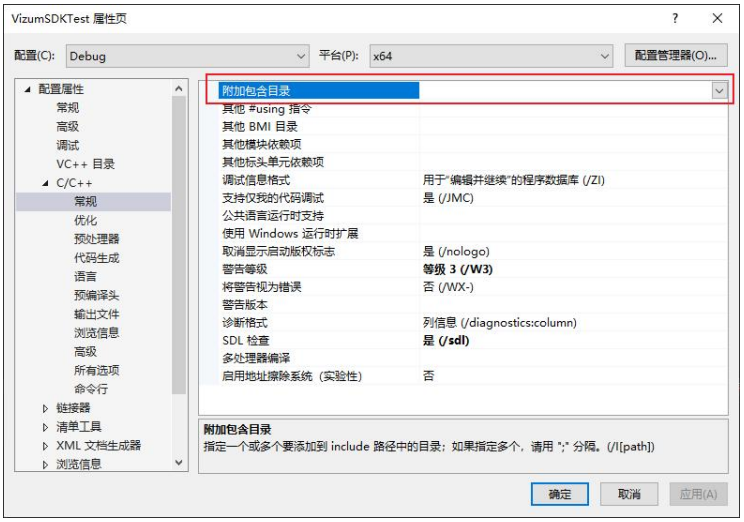
4）创建一个文件程序主文件

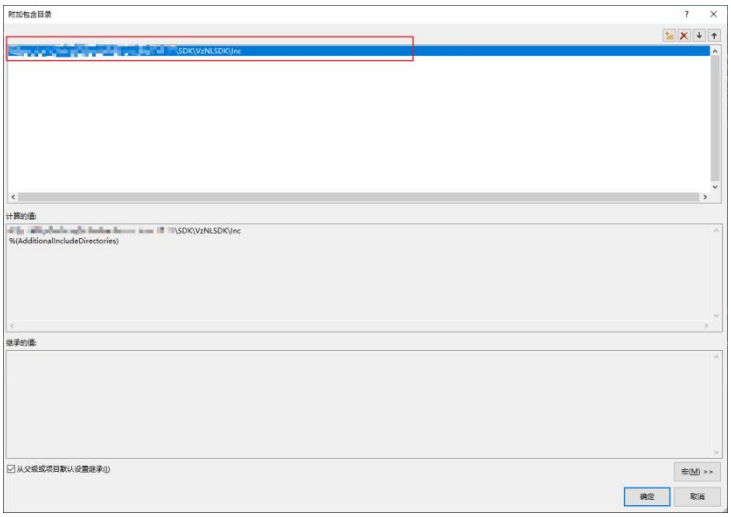


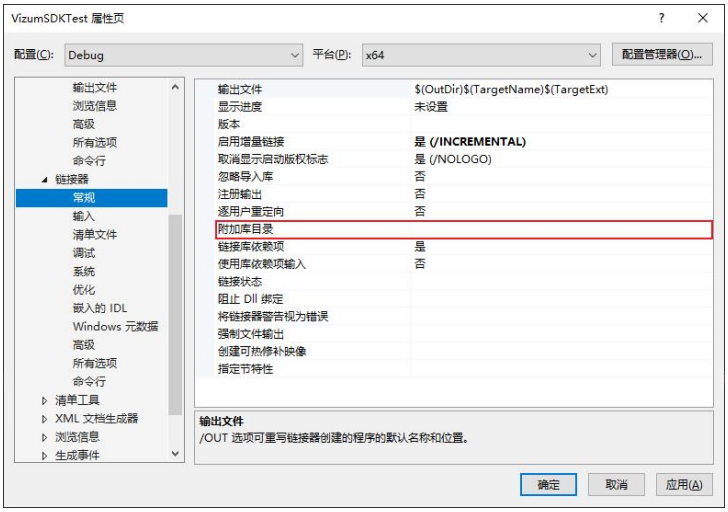

已安装
Visual C++
代码
格式设置
ATL
数据
资源
Web
实用工具
属性表
HLSL
Test
图形
联机
排序依据: 默认值
C++ 文件(.cpp)
头文件(.h)
C++ 类
C++ 模块接口单元(.ixx)
Visual C++
搜索(Ctrl+E)
类型: Visual C++
创建包含 C++ 源代码的文件
名称(N):
main.cpp
位置(L):
D:\Develop\Demo\VizumDetectLaser_V3.6.17.15\Demo\VizumSDKTest\VizumSDKTest\
浏览(B)...
添加(A)
取消

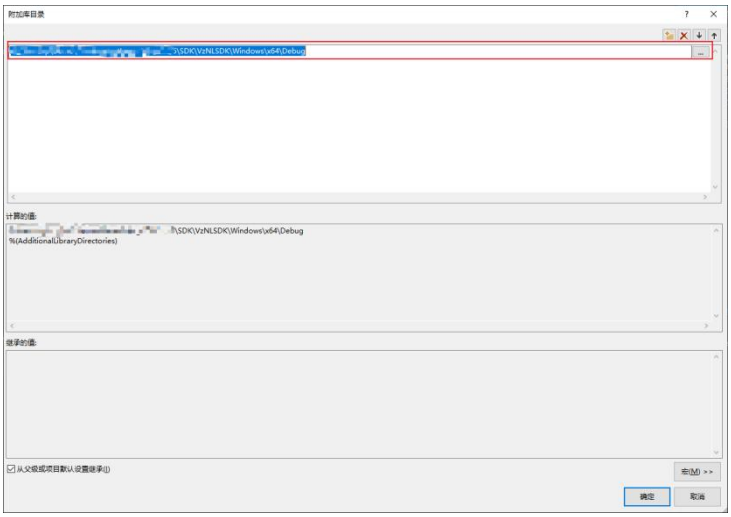
5）修改库文件包含目录


解决方案资源管理器
搜索解决方案资源管理器(Ctrl+;)
解决方案"VizumSDKTest"(1 个项目/共 1 个)
生成(U)
重新生成(E)
清理(N)
查看(W)
分析和代码清理(Z)
仅用于项目(J)
重定向项目(R)
限定为此范围(S)
新建解决方案资源管理器视图(N)
生成依赖项(B)
添加(D)
类向导(Z)... Ctrl+Shift+X
管理 NuGet 程序包(N)...
设为启动项目(A)
调试(G)
剪切(T) Ctrl+X
粘贴(P) Ctrl+V
移除(V) Del
重命名(M) F2
卸载项目(L)
加载项目的直接依赖项
加载项目的整个依赖树
重新扫描解决方案(S)
显示浏览数据库错误
清除浏览数据库错误
在文件资源管理器中打开文件夹(X)
Open in Terminal
属性(R)
Git 更改
资源视图
项目属性
VizumSDKTest
VizumSDKTest


VizumSDKTest 属性页
配置(C): Debug
平台(P): x64
配置管理器(O)...
配置属性
常规
高级
调试
VC++ 目录
C/C++
常规
优化
预处理器
代码生成
语言
预编译头
输出文件
浏览信息
高级
所有选项
命令行
链接器
清单工具
XML 文档生成器
浏览信息
附加包含目录
其他 #using 指令
其他 BMI 目录
其他模块依赖项
其他标头单元依赖项
调试信息格式 用于"编辑并继续"的程序数据库 (/ZI)
支持仅我的代码调试 是 (/JMC)
公共语言运行时支持
使用 Windows 运行时扩展
取消显示启动版权标志 是 (/nologo)
警告等级 等级 3 (/W3)
将警告视为错误 否 (/WX-)
警告版本
诊断格式 列信息 (/diagnostics:column)
SDL 检查 是 (/sdl)
多处理器编译
启用地址擦除系统（实验性） 否
附加包含目录
指定一个或多个要添加到 include 路径中的目录；如果指定多个，请用 ";" 分隔。(/I[path])
确定
取消
应用(A)


附加包含目录
\SDK\VzNLSDK\Inc
计算的值:
%(AdditionalIncludeDirectories)
继承的值:
从父级或项目默认设置继承(I)
宏(M) >>
确定
取消


VizumSDKTest 属性页
配置(C): Debug
平台(P): x64
配置管理器(O)...
输出文件
浏览信息
高级
所有选项
命令行
链接器
常规
输入
清单文件
调试
系统
优化
嵌入的 IDL
Windows 元数据
高级
所有选项
命令行
清单工具
XML 文档生成器
浏览信息
生成事件
输出文件 $(OutDir)$(TargetName)$(TargetExt)
显示进度 未设置
版本
启用增量链接 是 (/INCREMENTAL)
取消显示启动版权标志 是 (/NOLOGO)
忽略导入库 否
注册输出 否
逐用户重定向 否
附加库目录
链接库依赖项 是
使用库依赖项输入 否
链接状态
阻止 DII 绑定
将链接器警告视为错误
强制文件输出
创建可热修补映像
指定节特性
输出文件
/OUT 选项可重写链接器创建的程序的默认名称和位置。
确定
取消
应用(A)



6）编写测试程序

在预先创建的文件 main.cpp 中编写代码

```
#include "stdio.h"
#include <vector>
#include "VzNL_Common.h"

int main(int argc, char* argv[])
{
    SVzNLConfigParam sConfigParam;
    sConfigParam.nDeviceTimeOut = -1;
    int nRet = VzNL_Init(&sConfigParam);
    if (0 != nRet)
    {
        printf("Init SDK Failed!");
        return 0;
    }

    // 搜索智光眼
    VzNL_ResearchDevice(keSearchDeviceFlag_EthLaserRobotEye);

    // 获取设备信息
    int nDevCnt = 0; std::vector<SVzNLEyeCBInfo> vetDevInfo;
    VzNL_GetEyeCBDeviceInfo(nullptr, &nDevCnt);
    vetDevInfo.resize(nDevCnt);
    if (nDevCnt > 0)
    {
        VzNL_GetEyeCBDeviceInfo(vetDevInfo.data(), &nDevCnt);
    }

    // 显示设备信息
    for (int nDevIdx = 0; nDevIdx < nDevCnt; nDevIdx++)
    {
        printf("DevIP: %s\n", vetDevInfo[nDevIdx].byServerIP);
    }

    // 销毁SDK
    VzNL_Destroy();
    return 0;
}
```

7) 编译程序

8) 拷贝 SDK 库(.dll)到编译生成文件夹中

9) 运行

检测库接口入参、出参说明

3 VzNLSDK 详细接口

3.1 VzNLSDK 基础接口

头文件：#include "VZNL_Common.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.1.1 初始化接口（3.4.8 有修改）

int VzNL_Init(const SVzNLConfigParam* pConfigParam);

入参： pConfigParam 初始化检测参数，包含眼睛分辨率，超时时间

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

注意： 3.4.8 以前此函数会自带一次搜索

3.4.8 开始 Init 不带搜索，必须调用 VzNL_ResearchDevice 才可以枚举到设备

```
#include <stdio.h>
#include "VZNL_Common.h"

// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

// 这里编写SDK相关代码

// 销毁SDK
VzNL_Destroy();
```

3.1.2 添加设备 IP 列表

int VzNL_SetDeviceIpList(SVzNLDeviceIp* pDeviceList,unsigned int nCount);

入参： pDeviceList 设备 IP 列表

nCount 设备个数

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.1.3 重新搜索设备（3.4.8 有修改）

int VzNL_ResearchDevice(EVzSearchDeviceFlag nFindDevFlag);

入参： nFindDevFlag

* keSearchDeviceFlag_EyeCB 表示搜索 EyeCB 设备
* keSearchDeviceFlag_USBLargeEye 表示搜索 USB 眼睛设备
* keSearchDeviceFlag_EthLargeEye 表示搜索星光眼
* keSearchDeviceFlag_EthLaserRobotEye 表示搜索以太网智光眼
* keSearchDeviceFlagAll 表示搜索所有支持的设备

注意：此接口的参数也可以拼合使用例如要同时搜索 USB 和网络极光眼的相机的时候可以使用这种写法 keSearchDeviceFlag_USBLargeEye | keSearchDeviceFlag_EthLargeEye

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

注意： 3.4.8 版本以前是无需填写参数的，直接搜索所有的设备，到 3.4.8 版本可以让用户灵活的搜索所有设备

```
// 搜索智光眼相机
VzNL_ResearchDevice(keSearchDeviceFlag_EthLaserRobotEye);
```

3.1.4 获取所有搜索的设备

int VzNL_GetEyeCBDeviceInfo(SVzNLEyeCBInfo* pInfo, int* pnCount);

入参/出参： pInfo 搜索到的所有设备存储数组的首地址 pnCount 为搜索的设备个数。

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

```
SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

for (nDevIdx = 0; nDevIdx < nDevCount; nDevIdx++)
{
    printf("Device %d: %s %s\r\n", nDevIdx, pCBInfo[nDevIdx].szDeviceName,
pCBInfo[nDevIdx].byServerIP);
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}
```

3.1.5 获取所有打开的设备

int VzNL_GetAllOpenDevice(SVzNLEyeCBInfo* pInfo, int* pnCount);

入参/出参： pInfo 打开设备存储数组的首地址 pnCount 为打开的设备个数。

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

```

```

3.1.6 打开设备

VZNLHANDLE VzNL_OpenDevice(const SVzNLEyeCBInfo* pInfo, const SVzNLOpenDeviceParam* pOpenDeviceParam, int* pErrorCode);

入参： pInfo 所要打开设备结构体指针

pOpenDeviceParam 打开设备入参，设备检测类型

pErrorCode 错误码

返回： 设备句柄。NULL 为打开失败

```
#include <stdio.h>
#include "VZNL_Common.h"

// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

if (0 < nDevCount)
{
    SVzNLOpenDeviceParam sOpenDevParam;
    VZNLHANDLE hDevice = VzNL_OpenDevice(&pCBInfo [0], &sOpenDevParam, &nErrorCode);
    If ((hDevice != NULL) && (nErrorCode == 0))
    {
        // 对设备进行操作
    }
    VzNL_CloseDevice(hDevice);
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}

// 销毁SDK
VzNL_Destroy();
```

3.1.7 关闭设备

int VzNL_CloseDevice(VZNLHANDLE hDevice);

入参： hDevice 设备句柄

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息

```
#include <stdio.h>
#include "VZNL_Common.h"

// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

if (0 < nDevCount)
{
    SVzNLOpenDeviceParam sOpenDevParam;
    VZNLHANDLE hDevice = VzNL_OpenDevice(&pCBInfo [0], &sOpenDevParam, &nErrorCode);
    If ((hDevice != NULL) && (nErrorCode == 0))
    {
        // 对设备进行操作
    }
    VzNL_CloseDevice(hDevice);
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}

// 销毁SDK
```

```
VzNL_Destroy();
```

3.1.8 版本信息

int VzNL_GetVersion(VZNLHANDLE hDevice, SVzNLVersionInfo* pVersionInfo);

入参： hDevice 设备句柄

pVersionInfo 版本信息结构体的地址

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

```
SVzNLVersionInfo sVersion = { 0 };
int nErrorCode = VzNL_GetVersion(hDevice, &sVersion);
if (0 != nErrorCode)
{
  // 打印硬件版本号
  if (sVersion.sCap.sVersionCap.bValidVizumEyeVersion)
  {
    printf("VizumEye Version:%s\r\n", sVersion.szVizumEyeVersion);
  }
}
```

3.1.9 获取设备信息

int VzNL_GetDeviceInfo(VZNLHANDLE hDevice, SVzNLEyeCBInfo* pInfo);

入参： hDevice 设备句柄

pInfo 设备信息结构体的首地址

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

```
SVzNLEyeCBInfo sEyeCBInfo;
int nRet = VzNL_GetDeviceInfo(hDevice, (SVzNLEyeCBInfo*)&sEyeCBInfo);
if (sEyeCBInfo.eDeviceType == keDeviceType_EyeCB)
{
    SVzNLEyeCBDeviceInfoEx sEyeDevInfo;
    sEyeDevInfo.sEyeCBInfo.nSize = sizeof(SVzNLEyeCBDeviceInfoEx);
    int nRet = VzNL_GetDeviceInfo(hDevice, (SVzNLEyeCBInfo*)&sEyeDevInfo);
    if (0 == nRet)
    {
        printf("Resolution %d %d\r\n", sEyeDevInfo.sVideoRes.nFrameWidth,
sEyeDevInfo.sVideoRes.nFrameHeight);
    }
}
else if (sEyeCBInfo.eDeviceType == keDeviceType_LaserEye ||
        sEyeCBInfo.eDeviceType == keDeviceType_LaserRobotEye)
{
    SVzNLEyeDeviceInfoEx sEyeDevInfo;
sEyeDevInfo.sEyeCBInfo.nSize = sizeof(SVzNLEyeDeviceInfoEx);
    int nRet = VzNL_GetDeviceInfo(hDevice, (SVzNLEyeCBInfo*)&sEyeDevInfo);
    if (0 == nRet)
    {
        printf("Disparity %d\r\n", sEyeDevInfo.nFixDisparity);
```

```
        printf("Version %s\r\n", sEyeDevInfo.szVersion);
        printf("Resolution %d %d\r\n", sEyeDevInfo.sVideoRes.nFrameWidth,
sEyeDevInfo.sVideoRes.nFrameHeight);
    }
}
```

3.1.10 获取图像

(SDK1.4.0 中修改，将压缩格式参数去掉了，获取出来的 Buffer 为无压缩图像)

int VzNL_GetEyeImage(VZNLHANDLE hDevice, SVzNLImageData** ppLeftEye, SVzNLImageData** ppRightEye, unsigned int nTimeOut);

入参： hDevice 设备句柄

nTimeOut 超时时间

出参： ppLeftEye 左眼睛图像的结构地址

ppRightEye 右眼睛图像的结构地址

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

```
SVzNLImageData* pLeftEye = NULL;
SVzNLImageData* pRightEye = NULL;

int nErrorCode = VzNL_GetEyeImage(hDevice, nullptr, &pRightEye, 5000);
if (0 == nErrorCode)
{
    VzNL_SaveImage("D:\\l.png", pLeftEye);
    VzNL_SaveImage("D:\\r.png", pRightEye);
}

VzNL_ReleaseImage(&pLeftEye);
VzNL_ReleaseImage(&pRightEye);
```

释放图像

int VzNL_ReleaseImage(SVzNLImageData** ppImageData);

入参： ppImageData 图像数据

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

```
SVzNLImageData* pLeftEye = NULL;
SVzNLImageData* pRightEye = NULL;

int nErrorCode = VzNL_GetEyeImage(hDevice, nullptr, &pRightEye, 5000);
if (0 == nErrorCode)
{
    VzNL_SaveImage("D:\\l.png", pLeftEye);
    VzNL_SaveImage("D:\\r.png", pRightEye);
}

VzNL_ReleaseImage(&pLeftEye);
VzNL_ReleaseImage(&pRightEye);
```

3.1.11 获取错误信息

int VzNL_GetErrorInfo(int nErrorCode, char szError[256]);

入参： nErrorCode 错误码

出参： szError 错误信息

返回： 0 为正确。

```
char szErrorInfo[256] = { 0 };
if (0 == nErrorCode)
{
    printf("Execute Success\r\n");
    return;
}
VzNL_GetErrorInfo(nErrorCode, szErrorInfo);
printf("Failed: %s(%d)\r\n", szErrorInfo, nErrorCode);
```

3.1.12 设置图像输出格式

void VzNL_SetOutputImageFormat(EVzNLImageType eOutputImageType);

入参： eOutputImageType 当值为 keVzNLImageType_None 时，表示输出 原图格式，为其他时则返回其对应的格式。

```
// 设置图像输出格式为RGB
VzNL_SetOutputImageFormat(keVzNLImageType_BGR888);
```

3.1.13 连续取图

int VzNL_StartCapture(VZNLHANDLE hDevice, VzNL_OutputImageCB pImageCB, void* pCBParam);

入参： hDevice 设备 Handle

入参： pImageCB 图像回调接口

入参： pCBParam 回调接口参数

回调函数：

typedef void(*VzNL_OutputImageCB)(SVzNLImageData* pLeftImage, SVzNLImageData* pRightImage, SVzNLImageData* pCenterImage, const SVzOutputFrameProps* pFrameProps, void* pParam);

出参： pLeftImage 左图

出参： pRightImage 右图

出参： pFrameProps 输出帧的参数

出参： pParam 回调参数

返回 如果接口调用成功成功返回 0

停止取图

int VzNL_StopCapture(VZNLHANDLE hDevice);

入参：hDevice 设备 Handle

返回：调用接口成功返回 0

是否处于取图中

入参： hDevice 设备 Handle

入参： nErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回： VzTrue,表示正在取图,返回 VzFalse 表示不在取图

VzBool VzNL_IsCapturing(VZNLHANDLE hDevice, int* pnErrorCode);

```
void _OnOutputImageCB(SVzNLImageData* pLeftImage, SVzNLImageData* pRightImage,
SVzNLImageData* pColorImage, const SVzOutputFrameProps* sProps, void* pParam)
{
    printf("Left Image %p Right Image %p CenterImage %p \n", pLeftImage, pRightImage,
pColorImage);
}

// 开始采集示例
{
    int nErrorCode = VzNL_StartCapture(hDevice, _OnOutputImageCB, NULL);
    if (0 != nErrorCode)
    {
        return;
    }

    VzBool bIsCapture = VzNL_IsCapturing(hDevice, &nErrorCode);
    VzBool bIsAutoDetect = VzNL_IsAutoDetecting(hDevice, &nErrorCode);

    if (bIsCapture || bIsAutoDetect)
    {
        printf("当前状态 %s %s\n", bIsCapture ? "正在采集图像" : "", bIsAutoDetect ? "
正在检测激光线" : "");
    }
    else
    {
      printf("当前状态 空闲\n");
    }

    nErrorCode = VzNL_StopCapture(hDevice);
    if (0 != nErrorCode)
    {
      return;
    }
}
```

3.1.14 设置 Log 等级

void VzNL_SetLogLevel(EVzNLLogLevel eLevel, EVzNLLogType eLogType);

入参：eLevel Log 等级

入参：eLogType Log 类型

```
// 输出所有的Log到文件中VizumLog\xxx.txt
```

```
VzNL_SetLogLevel(keNLLogLevel_Information, keNLLogType_File);
```

3.1.15 设置设备状态提醒

int VzNL_SetDeviceStatusNotify(VZNLHANDLE hDevice, VzNL_OnNotifyStatusCB pNotifyCB);

入参： hDevice 设备 Handle

入参： pNotifyCB 返回函数

返回： 成功返回 0，否则为错误码

回调函数：

void(*VzNL_OnNotifyStatusCB)(EVzDeviceWorkStatus eStatus, void* pInfoParam);

设备状态改变 CallBack 函数

eStatus 状态

pInfoParam 状态参数

```
void _OnOutputImageCB(SVzNLImageData* pLeftImage, SVzNLImageData* pRightImage,
SVzNLImageData* pColorImage, const SVzOutputFrameProps* sProps, void* pParam)
{
    printf("Left Image %p Right Image %p CenterImage %p \n", pLeftImage, pRightImage,
pColorImage);
}

int nErrorCode = VzNL_StartCapture(hDevice, _OnOutputImageCB, NULL);
if (0 != nErrorCode)
{
    return;
}

nErrorCode = VzNL_StopCapture(hDevice);
if (0 != nErrorCode)
{
    return;
}
```

3.1.16 重连设备

int VzNL_ReConnectDevice(VZNLHANDLE hDevice);

入参： hDevice 设备 Handle

返回： 成功返回 0，否则为错误码

```
// 当设备离线后，可调用VzNL_ReConnectDevice对当前设备主动重连
VzNL_ReConnectDevice(hDevice)
```

3.1.17 启用所见即所得 ROI 图像（图像矫正）

int VzNL_EnableCalibROI(VZNLHANDLE hDevice, VzBool bEnable);

入参: hDevice 设备 Handle

入参: bEnable VzTrue 启用/VzFalse 禁用

返回 关闭成功返回 0，否则为错误码

是否启用了所见即所得功能

VzBool VzNL_IsEnableCalibROI(VZNLHANDLE hDevice, unsigned int* pnErrorCode);

入参： hDevice 设备 Handle

出参： pnErrorCode 错误码

返回： VzTrue 为启用了所见即所得 VzFalse 为关闭了所见即所得

```
EnableCalibROI(hDevice, VzTrue);

// 获取所见即所得后的图像
// 采图流程

EnableCalibROI(hDevice, VzFalse);
```

3.1.18 重启相机

int VzNL_RebootDevice(VZNLHANDLE hDevice);

入参： hDevice 设备 Handle

```
// 重启设备
VzNL_RebootDevice(hDevice);
```

3.1.19 获取 SDK 内存占用大小

unsigned int VzNL_GetUsingMemorySize(int* pnErrorCode);

入参： pnErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回： 内存占用的大小。

3.1.20 查询设备是否被占用

VzBool VzNL_IsDeviceInUse(const SVzNLEyeCBInfo* pInfo, int* pnErrorCode);

入参： pnErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回： VzTrue 被占用，否则未被占用。

3.1.21 关闭 SDK

void VzNL_Destroy();

```
#include <stdio.h>
#include "VZNL_Common.h"

// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}
```

```
// 这里编写SDK相关代码

// 销毁SDK
VzNL_Destroy();
```

3.1.22 获取相机瞳距信息

unsigned int VzNL_GetPupilDistance(VZNLHANDLE hDevice, int* pnErrorCode);

入参: hDevice 设备 Handle

入参: pnErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回 瞳距

3.1.23 开启快速采图模式

int VzNL_EnableQuickMode(VZNLHANDLE hDevice, VzBool bEnableQuickMode);

入参: hDevice 设备 Handle

入参: bEnableQuickMode 是否启用

返回 关闭成功返回 0，否则为错误码

3.1.24 关闭快速采图模式

VzBool VzNL_IsEnableQuickMode(VZNLHANDLE hDevice, int* pnErrorCode);

入参: hDevice 设备 Handle

入参: pnErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回 VzTrue 为已启用，否则为没有启用

3.1.25 获取左右目分辨率

int VzNL_GetResolution(VZNLHANDLE hDevice, SVzVideoResolution* psVideoRes);

入参: hDevice 设备 Handle

入参: psVideoRes 输出分辨率信息

返回 成功返回 0，否则为错误码

3.1.26 配置检测物体环境参数

int VzNL_ConfigDetectEnvLevel(VZNLHANDLE hDevice, EVzEnvType eEnvType,unsigned int nLevel);

入参: hDevice 设备句柄

入参: eEnvType 环境类型

入参: nLevel 参数等级

返回: 成功返回 0，否则为错误码

3.1.27 获取检测物体环境参数

SVzEnvLevel VzNL_GetConfigDetectEnvLevel(VZNLHANDLE hDevice, int* pnErrorCode);

入参: hDevice 设备句柄

入参: pnErrorCode 非空时返回错误码

返回: 环境参数

3.1.28 获取配置检测物体环境参数范围

SVzNLRange VzNL_GetDetectEnvLevelRange(VZNLHANDLE hDevice, EVzEnvType eEnvType, int* pnErrorCode);

入参: hDevice 设备句柄

入参: eEnvType 环境类型

入参: pnErrorCode 非空时返回错误码

返回: 环境参数范围

3.1.29 获取温度信息

int VzNL_QueryChipInfo(VZNLHANDLE hDevice, EVzDevChipInfo eChipInfo, float* pfMin, float* pfMax, float* pfCur);

入参： hDevice 设备 Handle

入参： eChipInfo 参数类型

入参： pfMin 最低数值

入参： pfMax 最高数值

入参： pfCur 当前数值

返回： 关闭成功返回 0，否则为错误码。

3.1.30 启用无限制帧率

int VzNL_EnableFreeFrameRate(VZNLHANDLE hDevice, VzBool bEnable);

入参： hDevice 设备 Handle

入参： bEnable 是否启用

返回： 关闭成功返回 0，否则为错误码。

3.1.31 设置是否输出 ROI 图像

void VzNL_SetOutputROIImage(VzBool bOutputROI);

入参： bOutputROI 是否输出 ROI 图像

返回： 关闭成功返回 0，否则为错误码。

3.1.32 获取双目固定视差

int VzNL_GetFixDisparity(VZNLHANDLE hDevice, int* pFixDisparity);

入参： hDevice 设备 Handle

入参： pFixDisparity 固定视差

返回： 成功返回 0，否则为错误码。

3.1.33 使能双目/RGB Map 本地缓存/加载功能，提升设备连接速度

int VzNL_EnableLocalMapDataLoad(VZNLHANDLE hDevice, VzBool bEnable);

入参： hDevice 设备 Handle

入参： bEnable 是否启用

返回： 成功返回 0，否则为错误码。

3.1.34 获取是否使能双目/RGB Map 本地缓存/加载功能

VzBool VzNL_IsEnableLocalMapDataLoad(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备 Handle

入参： pnErrorCode 设置为 null 时不对其进行赋值，否则则返回错误码，0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

返回： VzTrue 已使能，否则未使能。

3.1.35 配置实时采图的模式

int VzNL_SetCaptureMode(VZNLHANDLE hDevice, EVzCaptureMode eCaptureMode);

入参： hDevice 设备 Handle

入参： eCaptureType 采图模式。

返回： 成功返回 0，否则为错误码。

3.1.36 获取实时采图模式

EVzCaptureMode VzNL_GetCaptureMode(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备 Handle

入参： pnErrorCode 设置为 null 时不对其进行赋值，否则则返回错误码，0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

返回： 采图模式。

3.1.37 是否支持设置采图模式

VzBool VzNL_IsSupportCaptureMode(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备 Handle

入参： pnErrorCode 设置为 null 时不对其进行赋值，否则则返回错误码，0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

返回： VzTrue 支持，否则不支持。

3.1.38 设置焊缝跟踪功能需要的相机配置参数信息

int VzNL_SetTrackingScanPara(VZNLHANDLE hDevice, SVzTrackingScanPara* pPara);

入参： hDevice 设备 Handle

入参： pPara 输入焊缝跟踪功能配置参数：(pPara== nullptr)：函数内部自动获取当前相机配置参数作为焊缝跟踪配置参数进行保存

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.1.39 获取焊缝跟踪功能的相机配置参数信息

int VzNL_GetTrackingScanPara(VZNLHANDLE hDevice, SVzTrackingScanPara* pPara);

入参： hDevice 设备 Handle

入参： pPara 返回焊缝跟踪功能需要的配置参数

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.1.40 配置当前相机为焊缝跟踪状态

int VzNL_SetToTrackingScanMode(VZNLHANDLE hDevice, SVzTrackingScanPara* pPara);

入参： hDevice 设备 Handle

入参： pPara (pPara!= nullptr)：返回实际的焊缝跟踪配置参数，外部可用于更新本地参数及软件显示；
(pPara== nullptr)：不返回参数

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.1.41 恢复当前相机为焊缝跟踪前的初始状态：按备份的参数恢复相机状态

int VzNL_RestoreToNonTrackingScanMode(VZNLHANDLE hDevice, SVzTrackingScanPara* pPara);

入参： hDevice 设备 Handle

入参： pPara (pPara!= nullptr)：返回操作后实际的配置参数，外部可用于更新本地参数及软件显示；(pPara== nullptr)：不返回参数；

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.1.42 获取焊缝跟踪用户 ROI 尺寸

int VzNL_GetTrackingScanUserROISize(VZNLHANDLE hDevice, unsigned int* pROIWidth, unsigned int* pROIHeight);

入参： hDevice 设备 Handle

入参： pROIWidth 用于输出默认 ROI 宽度

入参： pROIHeight 用于输出默认 ROI 高度

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.1.43 获取相机当前时间戳(单位 微妙)

unsigned long long VzNL_GetTimeStamp(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备 Handle

入参： pnErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回： 成功返回时间戳。

3.1.44 是否支持扫描传图信号功能

VzBool VzNL_IsSupportScanModeExtSignal(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备 Handle

入参： pnErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回： VzTrue 支持，否则不支持。

3.1.45 启用扫描传图信号功能

int VzNL_EnableScanModeExtSignal(VZNLHANDLE hDevice, VzBool bEnable);

入参： hDevice 设备 Handle

入参： bEnable VzTrue 启用 / VzFalse 禁用

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.1.46 获取是否启用扫描传图信号功能

VzBool VzNL_IsEnableScanModeExtSignal(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备 Handle

入参： pnErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回： VzTrue 启用，否则禁用。

3.2 VzNLSDK 基础配置接口

头文件：#include "VZNL_DetectConfig.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.2.1 检测区域设置

①配置检测区域

int VzNL_ConfigDetectROI(VZNLHANDLE hDevice, const SVzNLROIRect* pLeftROIArea, const SVzNLROIRect* pRightROI);

入参： hDevice 设备句柄

pLeftROIArea 左眼图像检测区域

pRightROI 右眼图像检测区域

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

②获取检测区域

int VzNL_GetConfigDetectROI(VZNLHANDLE hDevice, SVzNLROIRect* pLeftROI, SVzNLROIRect* pRightROI);

入参： hDevice 设备句柄

pLeftROI 左图像 ROI

pRightROI 右图像 ROI

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

③格式化 ROI

int VzNL_FormatDetectROI(VZNLHANDLE hDevice, SVzNLROIRect* pLeftROI, SVzNLROIRect* pRightROI, VzBool bKeepTop);

入参： hDevice 设备句柄

入参： pLeftROI 左侧 ROI

入参： pRightROI 右侧 ROI

入参： bKeepTop 保持高度

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

```
// 注意智光眼ROI 最小为64 * 256，在未开启所见即所得的时候，需要宽度32对齐，高度128对齐
// 开启所见即所得后，没有限制
SVzNLROIRect sLeftROI = {0, 64, 0, 1024};
SVzNLROIRect sRightROI = { 0, 64, 0, 1024 };

// 获取格式化后的ROI
Int nErrorCode = VzNL_FormatDetectROI(hDevice,  &sLeftROI, &sRightROI, VzTrue);
If (0 == nErrorCode)
{
    // 配置 ROI
    VzNL_ConfigDetectROI(hDevice, &sLeftROI, &sRightROI);
}

// 获取 ROI
SVzNLROIRect sRetLeftROI = {0, 64, 0, 1024};
SVzNLROIRect sRetRightROI = { 0, 64, 0, 1024 };
VzNL_ GetConfigDetectROI (hDevice, &sRetLeftROI, &sRetRightROI);
```

3.2.2 设置/获取眼睛曝光值

int VzNL_ConfigEyeExpose(VZNLHANDLE hDevice, EVzNLExposeMode eExposeMode, unsigned int nExposeTime);

入参: hDevice 设备句柄

入参：eExposeMode [keVzNLExposeMode_Fix=定值曝光 / keVzNLExposeMode_Auto=自动曝光]

入参：nExposeTime 曝光时间[20~100000]

返回：返回错误码

注意: 目前智光眼/星光眼仅支持定值曝光

int VzNL_GetConfigEyeExpose(VZNLHANDLE hDevice, EVzNLExposeMode* peExposeMode, unsigned int*

pnExposeTime);

```
// 配置相机曝光为 100
VzNL_ConfigEyeExpose(hDevice, keVzNLExposeMode_Fix， 100);
```

3.2.3 设置/获取相机增益

入参: hDevice 设备句柄

入参: eSensorType [keEyeSensorType_Left=左眼 / keEyeSensorType_Right=右眼]

入参: nCameraGain 曝光时间[0~255]

返回: 返回错误码

注意: 目前智光眼支持

int VzNL_SetCameraGain(VZNLHANDLE hDevice, EVzEyeSensorType eSensorType, unsigned short nCameraGain);

int VzNL_GetCameraGain(VZNLHANDLE hDevice, EVzEyeSensorType eSensorType, unsigned short* pnCameraGain);

```
// 设置左右目的增益为 1
VzNL_SetCameraGain(hDevice, keEyeSensorType_Left,1);
VzNL_SetCameraGain(hDevice, keEyeSensorType_Right,1);
```

3.2.4 设置 Trigger 模式

int VzNL_SetTriggerMode(VZNLHANDLE hDevice, EVzEyeTriggerMode eTriggerMode);

入参: hDevice 设备句柄

入参: eTriggerMode Trigger 模式[被动/主动/上升沿/下降沿/低电平使能]，默认使用 Master 主动触发模式

返回: 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.2.5 获取当前 Trigger 模式

EVzEyeTriggerMode VzNL_GetTriggerMode(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备句柄

入参： pnErrorCode 设置为 null 时不对其进行赋值，否则则返回错误码，0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

返回： Trigger 模式[被动/主动/上升沿/下降沿/低电平使能]

```
// 设置为主模式，主模式下由系统按帧率进行自动检测
VzNL_SetTriggerMode(hDevice, keEyeTriggerMode_Master);
```

3.2.6 设置触发极性模式

int VzNL_SetTriggerPolar(VZNLHANDLE hDevice, VzBool bNormal);

VzBool VzNL_GetTriggerPolar(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备句柄

入参： bNormal VzTrue 为默认极性/VzFalse 反向

返回： 返回错误码

```
// 触发极性，若修改则会有高电平触发变为低电平触发
SetTriggerPolar(hDevice, VzTrue);
```

3.2.7 设置触发分频

int VzNL_SetTriggerDivCoe(VZNLHANDLE hDevice, unsigned int nDivCoe);

unsigned int VzNL_GetTriggerDivCoe(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备句柄

入参： nDivCoe 分频数

返回： 返回错误码

```
// 默认为 1，表示每 1 个触发信号可以对相机进行触发一次
VzNL_SetTriggerDivCoe(hDevice, 1);
```

3.2.8 是否忽略外部使能

int VzNL_IgnoreTriggerExtSignal(VZNLHANDLE hDevice, VzBool bIgnore);

VzBool VzNL_IsIgnoreTriggerExtSignal(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备句柄

入参： bIgnore VzTrue 忽略（默认） VzFalse (关联外部使能信号)

返回： 返回错误码

```
// 配置忽略外部使能
VzNL_IgnoreTriggerExtSignal(hDevice, VzTrue);
```

3.2.9 启用外部信号使能

int VzNL_EnableTriggerHwExtEn(VZNLHANDLE hDevice, VzBool bHwExtEnable);

VzBool VzNL_IsEnableTriggerHwExtEn(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备句柄

入参： bHwExtEnable VzTrue 为使用硬信号 VzFalse 为使用软信号

返回： 返回错误码

```
// 需要配置忽略使能信号为 VzFalse 时才会起作用

// 配置为硬使能，接受硬件使能信号
VzNL_EnableTriggerHwExtEn(hDevice, VzTrue);

// 配置为软使能，接受软件生成的使能信号
VzNL_EnableTriggerHwExtEn(hDevice, VzTrue);
```

3.2.10 生成软外部使能信号

int VzNL_GenerateTriggerSoftExtEn(VZNLHANDLE hDevice, VzBool bHighLevel);

入参： hDevice 设备句柄

入参： bHighLevel 高电平信号

返回： 返回错误码

```
// 当使能信号为软使能时，配置 VzTrue 或者 VzFalse 可以模拟其高低电平信号
VzNL_GenerateTriggerSoftExtEn(hDevice, VzTrue);
```

3.2.11 获取参数设置范围

入参：hDevice 设备句柄

入参：eType 参数类型

入参：pnMinVal 最小值

入参：pnMaxVal 最大值

返回：返回错误码

int VzNL_QueryParamRange(VZNLHANDLE hDevice, EVzDeviceParamType eType, unsigned int* pnMinVal, unsigned int* pnMaxVal);

```
// 获取当前ROI和曝光下的帧率范围
unsigned int nMinFrameRate, nMaxFrameRate;
```

```
VzNL_QueryParamRange(hDevice,keDeviceParamType_FrameRate, &nMinFrameRate,
&nMaxFrameRate);

// 获取数据结构中带颜色数据的帧率范围
unsigned int nMinFrameRate, nMaxFrameRate;
VzNL_QueryParamRange(hDevice, keDeviceParamType_FrameRateWithColorData, &nMinFrameRate,
&nMaxFrameRate);
```

3.2.12 是否启用同步 RGB 设置

入参：hDevice 设备 handle

入参：bEnable true 表示同步设置 RGB，false 表示不进行同步

返回：返回错误码

int VzNL_EnableSyncRGBD(VZNLHANDLE hDevice, VzBool bEnable);

```
// 启用 RGB 同步,当 RGB 动态相机检测时，请启用此选项
VzNL_EnableSyncRGBD(hDevice, VzTrue);
```

3.2.13 是否启用低功耗模式

入参：hDevice 设备 handle

入参：bEnable true 表示开启低功耗,false 表示关闭低功耗

返回：返回错误码

int VzNL_EnableLowPowerMode(VZNLHANDLE hDevice, VzBool bEnable);

```
// 默认静态相机是启用低功耗状态的;动态相机没有此功能
VzNL_EnableLowPowerMode(hDevice, VzTrue);
```

3.2.14 停流后等待多长时间进入低功耗模式 单位：秒

入参：hDevice 设备 handle

入参：dDuration 时长：单位(秒)

返回：返回错误码

int VzNL_EnterLowPowerDuration(VZNLHANDLE hDevice, double dDuration);

```
// 设置 3 秒进入低功耗
VzNL_EnterLowPowerDuration(hDevice, 3.);
```

3.2.15 设置高斯模式

入参：hDevice 设备 handle

入参：bHGauss 设备横向高斯

入参：bVGauss 设备纵向高斯

返回：返回错误码

int VzNL_EnableGauss(VZNLHANDLE hDevice, VzBool bHGauss, VzBool bVGauss);

int VzNL_IsEnableGauss(VZNLHANDLE hDevice, VzBool* pbHGauss, VzBool* pbVGauss);

```
VzBool bHGauss = VzFalse; VzBool bVGauss = VzFalse;
int nErrorCode = VzNL_IsEnableGauss(hDevice, &bHGauss, &bVGauss);
nErrorCode = VzNL_EnableGauss(hDevice, bHGauss, bVGauss);
```

3.2.16 设置触发输出控制模式

入参：hDevice 设备 handle

入参：eTriggerOutMode 触发输出控制类型

keStrobeTriggerOutMode_None 默认不输出

keStrobeTriggerOutMode_CameraLink 当触发模式为 keEyeTriggerMode_Master 时,相机会跟随帧率进行一个信号的输出（此模式下，会导致帧率下降)

keStrobeTriggerOutMode_UserControl 用户控制，此模式下需要用 VzNL_GenerateTriggerOutSignal 生成一个控制信号

返回：返回错误码

```
int VzNL_SetStrobeTriggerOutMode(VZNLHANDLE hDevice, EVzStrobeTriggerOutMode eTriggerOutMode);
EVzStrobeTriggerOutMode VzNL_GetStrobeTriggerOutMode(VZNLHANDLE hDevice, int* pnErrorCode);
```

3.2.17 生成一个触发输出信号

入参：hDevice 设备 handle

入参：bTriggerOn VzTrue 时为高电平，VzFalse 时为低电平信号

返回：返回错误码

```
int VzNL_GenerateTriggerOutSignal(VZNLHANDLE hDevice, VzBool bTriggerOn);
VzBool VzNL_IsTriggerOutOnSignal(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
// 控制触发线生成触发信号，假设用此信号控制补光灯
VzNL_SetStrobeTriggerOutMode (hDevice, keStrobeTriggerOutMode_CameraLink );
// 打开补光灯
VzNL_GenerateTriggerOutSignal(hDevice, VzTrue);
// 关闭补光灯
VzNL_GenerateTriggerOutSignal(hDevice, VzFalse);
```

3.2.18 是否支持高帧率下降沿模式

入参：hDevice 设备 handle

出参: pnErrorCode 非空时返回错误码

返回：VzTrue 表示支持, VzFalse 表示不支持

```
VzBool VzNL_IsSupportHighFrequenceFallingEdge(VZNLHANDLE hDevice, int* pnErrorCode);
```

3.2.19 “全局降噪”功能控制和当前状态获取接口

入参：hDevice 设备 handle

入参：VzBool bEnable：使能/禁用全局降噪功能；

出参: VzBool* pbEnable：返回当前全局降噪功能使能状态

返回：返回错误码，VzNL_IsEnableGlobalDenoise() 返回错误码表示设备不支持此功能；

函数接口：

```
int VzNL_EnableGlobalDenoise(VZNLHANDLE hDevice, VzBool bEnable);
int VzNL_IsEnableGlobalDenoise(VZNLHANDLE hDevice, VzBool* pbEnable);
```

3.2.20 设置/获取扫描噪点过滤阈值接口

入参：hDevice 设备 handle

入参：nValue：扫描噪点过滤阈值,取值范围 0-255,其中 0 表示关闭此功能

返回：返回错误码

函数接口：

```
int VzNL_SetScanDenoiseTh(VZNLHANDLE hDevice, unsigned int nValue);
int VzNL_GetScanDenoiseTh(VZNLHANDLE hDevice, unsigned int* pnValue);
```

3.2.21 设置/获取点云平滑门限接口

入参：hDevice 设备 handle

入参：nValue： 平滑门限值

返回：返回错误码

函数接口：

int VzNL_SetSmoothingTh(VZNLHANDLE hDevice, unsigned int nValue);

int VzNL_GetSmoothingTh(VZNLHANDLE hDevice, unsigned int* pnValue);

3.2.22 设置/获取激光线检测模式接口

入参：hDevice 设备 handle

入参：nValue：检测模式值

返回：返回错误码

函数接口：

int VzNL_SetMLineMode(VZNLHANDLE hDevice, unsigned int nValue);

int VzNL_GetMLineMode(VZNLHANDLE hDevice, unsigned int* pnValue);

3.2.23 是否支持抽帧功能

入参：hDevice 设备 handle

出参: pnErrorCode 非空时返回错误码

返回：VzTrue 表示支持, VzFalse 表示不支持

VzBool VzNL_IsSupportDivFrame(VZNLHANDLE hDevice, int* pnErrorCode);

3.2.24 设置/获取抽帧数接口

入参：hDevice 设备 handle

入参：nDivFrame 抽帧数[1,10]

返回：返回错误码

函数接口：

int VzNL_SetDetectDivFrame(VZNLHANDLE hDevice, unsigned int nDivFrame);

unsigned int VzNL_GetDetectDivFrame(VZNLHANDLE hDevice, int* pnErrorCode);

3.3 静态相机配置接口

头文件：#include " VZNL_SwingMotor.h "

库文件：VzNLDetect.dll / libVzNLDetect.so

是否为静态相机

VzBool VzNL_IsSupportSwingMotor(VZNLHANDLE hDevice, int* pnErrorCode);

入参: hDevice 设备句柄

出参: pnErrorCode 非空时返回错误码

返回: 返回 VzTrue 表示支持摆动机构

```
// 获取是否支持静态摆动
VzBool bIsSupportSwing = VzNL_IsSupportSwingMotor(hDevice, nullptr);
```

3.3.1 设置工作范围

int VzNL_SetSwingMotorWorkRange(VZNLHANDLE hDevice, double dNearDistance, double dFarDistance);

入参: hDevice 设备句柄

入参: dNearDistance 近距离

入参: dFarDistance 远距离

返回:　非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.2 启用/禁用摆动机构

入参:　hDevice 设备句柄

入参:　bEnable 启用/禁用

返回:　返回 0 表示正确

int VzNL_EnableSwingMotor(VZNLHANDLE hDevice, VzBool bEnable);

3.3.3 是否启用了摆动机构

入参:　hDevice 设备句柄

出参: pnErrorCode 非空时返回错误码

返回:　返回是否启用摆动机构的标志

VzBool VzNL_IsEnableSwingMotor(VZNLHANDLE hDevice, int* pnErrorCode);

3.3.4 打开/关闭激光器

int VzNL_EnableLaserLight(VZNLHANDLE hDevice, VzBool bEnable);

入参:　hDevice 设备句柄

入参:　bEnable 启用/禁用

返回:　非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.5 是否启用了激光器

VzBool VzNL_IsEnableLaserLight(VZNLHANDLE hDevice);

入参:　hDevice 设备句柄

返回:　是否打开激光器

3.3.6 调节激光器亮度

int VzNL_SetLaserLight(VZNLHANDLE hDevice, unsigned int nLight);

入参:　hDevice 设备句柄

入参:　nLight 激光器亮度(0~3)

返回:　非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.7 获取激光器亮度

unsigned int VzNL_GetLaserLight(VZNLHANDLE hDevice, int* pnErrorCode);

入参:　hDevice 设备句柄

出参: pnErrorCode 非空时返回错误码

返回:　返回当前激光器亮度

3.3.8 设置角速度

int VzNL_SetSwingAngleSpeed(VZNLHANDLE hDevice, int nAngleSpeed);

入参:　hDevice 设备句柄

入参: nAngleSpeed 角速度

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.9 获取摆动机构角速度

int VzNL_GetSwingAngleSpeed(VZNLHANDLE hDevice, int* pnAngleSpeed);

入参: hDevice 设备句柄

出参: pnAngleSpeed 角速度

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.10 设置摆动机构起止角度

int VzNL_SetSwingMotorAngle(VZNLHANDLE hDevice, int nStartPos, int nEndPos);

入参: hDevice 设备句柄

入参: nStartPos 开始角度

入参: nEndPos 结束角度

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.11 获取电机采图起止角度

int VzNL_GetSwingMotorAngle(VZNLHANDLE hDevice, float* pnStartPos, float* pnEndPos);

入参: hDevice 设备句柄

出参: nStartPos 开始角度

出参: nEndPos 结束角度

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

```
// 获取开始结束角度
float fStartPos; float fEndPos;
VzNL_GetSwingMotorAngle(hDevice, &fStartPos, &fEndPos);

// 设置开启结束角度
VzNL_SetSwingMotorAngle(hDevice, fStartPos, fEndPos);
```

3.3.12 设置当前位置为开始/结束位置

int VzNL_SetSwingCurPosToMotorStartPos(VZNLHANDLE hDevice);

入参: hDevice 设备句柄

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.13 设置当前位置位结束角度

int VzNL_SetSwingCurPosToMotorEndPos(VZNLHANDLE hDevice);

入参: hDevice 设备句柄

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.14 旋转到某个位置

int VzNL_RotateSwing(VZNLHANDLE hDevice, float fAngle);

入参: hDevice 设备句柄

入参: fAngle 旋转到的角度(0~75°)

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.15 旋转到开始位置

int VzNL_RotateSwingToStartPos(VZNLHANDLE hDevice);

入参: hDevice 设备句柄

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.16 设置摆动机构扫描模式

int VzNL_SetSwingScanMode(VZNLHANDLE hDevice, EVzSwingMotorScanMode eScanMode);

入参: hDevice 设备句柄

入参: eScanMode 扫描模式

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.17 获取摆动机构扫描模式

EVzSwingMotorScanMode VzNL_GetSwingScanMode(VZNLHANDLE hDevice);

入参: hDevice 设备句柄

返回: 摆动机构扫描模式

```
//keSwingMotorScanMode_Once,    一次
//keSwingMotorScanMode_Loop,    轮询
// 摆动一次
VzNL_SetSwingScanMode(hDevice,  keSwingMotorScanMode_Once);
```

3.3.18 获取扫描机构信息

int VzNL_GetSwingMotorInfo(VZNLHANDLE hDevice, SVzSwingMotorDevInfo* psSwingMotorInfo);

入参: hDevice 设备句柄

出参: psSwingMotorInfo 电机信息

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.19 获取当前角度

float VzNL_GetSwingCurAngle(VZNLHANDLE hDevice, int* pnErrorCode);

入参: hDevice 设备句柄

出参: pnErrorCode 错误码(当不为 nullptr 返回)

返回: 返回当前摆动机构角度(0~75°)

3.3.20 设置停顿时间

int VzNL_SetSwingStopTime(VZNLHANDLE hDevice, unsigned int nMillionSecond);

入参: hDevice 设备句柄

入参: nMillionSecond 停顿时间(ms)

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.21 左移

int VzNL_SwingMoveLeft(VZNLHANDLE hDevice);

入参: hDevice 设备句柄

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.22 右移

int VzNL_SwingMoveRight(VZNLHANDLE hDevice);

入参: hDevice 设备句柄

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.23 设置相机摆动方向

int VzNL_SetSwingRotateDirect(VZNLHANDLE hDevice, EVzSwingRotateDirect eRotateDirect);

入参: hDevice 设备句柄

入参: eRotateDirect 摆动方向[keSwingRotateDirect_Clockwise 顺时针方向摆动,
keSwingRotateDirect_Counterclockwise 逆时针方向摆动]

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.24 获取相机摆动方向

int VzNL_GetSwingRotateDirect(VZNLHANDLE hDevice, EVzSwingRotateDirect* peRotateDirect);

入参: hDevice 设备句柄

入参: peRotateDirect 摆动方向

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.25 获取版本

unsigned int VzNL_GetSwingVersionCode(VZNLHANDLE hDevice);

入参: hDevice 设备句柄

返回: 摆动机构版本号

3.3.26 设置相机激光器停止位置

int VzNL_SetSwingStopAngle(VZNLHANDLE hDevice, int nAngle);

入参: hDevice 设备句柄

入参: nAngle 停止角度位置

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.27 获取相机激光器停止位置

int VzNL_GetSwingStopAngle(VZNLHANDLE hDevice, int* pnErrorCode);

入参: hDevice 设备句柄

入参: pnErrorCode 非空时返回错误码

返回: 停止角度位置

3.3.28 设置判断完成了一次扫描的角度阈值

int VzNL_SetSwingEndAngleThres(VZNLHANDLE hDevice, float fAngle);

入参: hDevice 设备句柄

入参: fAngle 角度阈值

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

3.3.29 获取判断完成了一次扫描的角度阈值

float VzNL_GetSwingEndAngleThres(VZNLHANDLE hDevice, int* pnErrorCode);

入参: hDevice 设备句柄

入参: pnErrorCode 非空时返回错误码

返回: 角度阈值

3.3.30 是否支持静态相机输出激光线时告知是否完成了一次扫描的功能

VzBool VzNL_IsSupportEndAngleThres(VZNLHANDLE hDevice, int* pnErrorCode);

入参: hDevice 设备句柄

入参: pnErrorCode 非空时返回错误码

返回: VzTrue 表示支持，否则不支持

3.3.31 查找摆动机构模块

VzBool VzNL_FindSwing(VZNLHANDLE hDevice, int* pnErrorCode);

入参: hDevice 设备句柄

入参: pnErrorCode 非空时返回错误码

返回: VzTrue 表示找到找到摆动机构，否则没有找到

3.3.32 重启摆动模块

int VzNL_RebootSwing(VZNLHANDLE hDevice);

入参: hDevice 设备句柄

入参: pnErrorCode 非空时返回错误码

返回: 非 0 表示失败,可以使用 VzNL_GetErrorInfo 获取

```
// 单次摆动并停止
#include <stdio.h>
#include <vector>
#include "VZNL_Common.h"
#include "VZNL_EyeConfig.h"
#include "VZNL_DetectLaser.h"
#include "VZNL_DetectConfig.h"
#include "VZNL_SwingMotor.h"
#include "windows.h"
#include <fstream>

static HANDLE s_hPauseEvent = NULL;

/// @name 自动检测回调函数
/// @{
void _AutoOutputLaserLineExCB(EVzResultDataType eDataType, SVzLaserLineData*
pLaserLinePoint, void* pParam)
{
```

```
    // 当需要的点云格式为SVzNLPointXYZRGBA时
    if (keResultDataType_PointXYZRGBA == eDataType)
    {
    }
}

// 显示错误信息
void _DisplayErrorInfo(const char* pszDesc, int nErrorCode)
{
    char szOutput[256];
    if (0 == nErrorCode)
    {
        sprintf_s(szOutput, 256, "%s\r\n", pszDesc);
    }
    else
    {
        char szErrorInfo[256] = { 0 };
        VzNL_GetErrorInfo(nErrorCode, szErrorInfo);

        sprintf_s(szOutput, 256, "%s [%d] %s\r\n", pszDesc, nErrorCode, szErrorInfo);
    }
    printf(szOutput);
}

// 搜索设备
void _ResearchDevice(std::vector<SVzNLEyeCBInfo>& vetDevice)
{
    // 搜索相机
    VzNL_ResearchDevice(keSearchDeviceFlag_EthLaserRobotEye);

    // 遍历相机
    int nDevCount = 0;
    VzNL_GetEyeCBDeviceInfo(nullptr, &nDevCount);
    if (nDevCount <= 0)
    {
        return;
    }

    vetDevice.resize(nDevCount);
    VzNL_GetEyeCBDeviceInfo(vetDevice.data(), &nDevCount);
}

// @brief 搜索且进行绑定设备
void _SearchAndBindDevice(std::vector<SVzNLEyeCBInfo>& vetDevice)
```

```
{
    int nErrorCode = 0;
    VzBool bCanResearch = VzFalse;
    do
    {
        bCanResearch = VzFalse;
        _ResearchDevice(vetDevice);
        std::vector<SVzNLEyeCBInfo>::iterator itDevice = vetDevice.begin();
        while (itDevice != vetDevice.end())
        {
            // 相机没有被识别
            if (VzFalse == itDevice->bValidDevice)
            {
                SVzNLEyeCBInfo sCurDevInfo = *itDevice;
                SVzNLEthernetEyeConfigInfo sOutputEthConfigInfo;
                nErrorCode = VzNL_GetEthernetEyeConfigInfo(&sCurDevInfo,
&sOutputEthConfigInfo);
                if (0 != nErrorCode)
                {
                    _DisplayErrorInfo("获取网络信息失败", nErrorCode);
                }

                //
                if (0 == memcmp(sOutputEthConfigInfo.byLocalIP,
sOutputEthConfigInfo.sNetCardInfo.byLocalIP, 4))
                {
                    _DisplayErrorInfo("请查看是否有网络权限和管理员权限\r\n", 0);
                }
                else
                {
                    nErrorCode = VzNL_BindEthernetEye(&sCurDevInfo);
                    if (0 != nErrorCode)
                    {
                        _DisplayErrorInfo("绑定相机失败", nErrorCode);
                    }
                    bCanResearch = VzTrue;
                }
            }
            itDevice++;
        }
    } while (bCanResearch);
}

/**
```

```
 * @brief 设备状态改变CallBack函数
 * @param eStatus [out] 状态
 * @param pInfoParam [out] 状态参数
 */
void _OnNotifyStatusCB(EVzDeviceWorkStatus eStatus, void* pInfoParam)
{
    if (keDeviceWorkStatus_Device_Auto_Stop == eStatus)
    {
        printf("收到停止信号\r\n");
        if (nullptr != s_hPauseEvent)
        {
            SetEvent(s_hPauseEvent);
        }
    }
}

int main()
{
    // 创建停止信号
    s_hPauseEvent = CreateEvent(NULL, TRUE, FALSE, NULL);

    // 初始化SDK
    SVzNLConfigParam sConfigParam;
    sConfigParam.nDeviceTimeOut = 0;
    int nRet = VzNL_Init(&sConfigParam);
    if (0 != nRet)
    {
        _DisplayErrorInfo("查看是否有其他程序在使用SDK?", nRet);
    }

    // 枚举相机
    std::vector<SVzNLEyeCBInfo> vetDevice;
    _SearchAndBindDevice(vetDevice);

    //
    int nErrorCode = 0;
    if (vetDevice.size() > 0)
    {
        _DisplayErrorInfo("显示当前可用相机\r\n", 0);
        std::vector<SVzNLEyeCBInfo>::iterator itDevice = vetDevice.begin();
        while (itDevice != vetDevice.end())
        {
            // 相机没有被识别
            if (VzFalse == itDevice->bValidDevice)
```

```
            {
                itDevice++;
            }

            char szOutput[256];
            sprintf_s(szOutput, 256, "----------------------准备打
开 %s----------------------\r\n", itDevice->byServerIP);
            _DisplayErrorInfo(szOutput, 0);

            itDevice++;
        }
    }

    //
    do
    {
        if (vetDevice.size() == 0)
        {
            break;
        }

        // 打开相机
        SVzNLOpenDeviceParam sOpenDevParam;
        VZNLHANDLE hDevice = VzNL_OpenDevice(&vetDevice[0], &sOpenDevParam, &nErrorCode);
        if (0 != nErrorCode)
        {
            _DisplayErrorInfo("打开设备失败", nErrorCode);
            break;
        }
        else
        {
            char szOutput[256];
            sprintf_s(szOutput, 256, "打开相机%s成功\r\n", vetDevice[0].byServerIP);
            _DisplayErrorInfo(szOutput, 0);
        }
        VzNL_SetDeviceStatusNotify(hDevice, _OnNotifyStatusCB, NULL);

        // 创建激光线检测工具
        nErrorCode = VzNL_BeginDetectLaser(hDevice);
        if (0 != nErrorCode)
        {
            _DisplayErrorInfo("创建检测激光线工具", nErrorCode);
            break;
        }
```

```
        else
        {
            _DisplayErrorInfo("创建激光线检测工具成功\r\n", 0);
        }

        // 设置到主模式
        nErrorCode = VzNL_SetTriggerMode(hDevice, keEyeTriggerMode_Master);
        if (0 != nErrorCode)
        {
            _DisplayErrorInfo("设置主模式失败", nErrorCode);
            break;
        }
        else
        {
            _DisplayErrorInfo("主模式设置成功\r\n", 0);
        }

        unsigned int nMinFrameRate, nMaxFrameRate;
        VzNL_QueryParamRange(hDevice, keDeviceParamType_FrameRate, &nMinFrameRate,
&nMaxFrameRate);

        VzNL_SetFrameRate(hDevice, nMaxFrameRate);

        ResetEvent(s_hPauseEvent);

        VzBool bIsSupportMotor = VzNL_IsSupportSwingMotor(hDevice, nullptr);
        if (bIsSupportMotor)
        {
            VzNL_EnableCalibROI(hDevice, VzFalse);

            VzNL_EnableSwingMotor(hDevice, VzTrue);
        }

        // 开始流模式检测
        nErrorCode = VzNL_StartAutoDetectEx(hDevice, keResultDataType_PointXYZRGBA,
keFlipType_None, _AutoOutputLaserLineExCB, nullptr);
        if (0 != nErrorCode)
        {
            _DisplayErrorInfo("开流失败", nErrorCode);
            break;
        }
        else
        {
            _DisplayErrorInfo("开流成功\r\n", 0);
```

```
        }

        // 等待摆动结束
        WaitForSingleObject(s_hPauseEvent, INFINITE);

        nErrorCode = VzNL_StopAutoDetect(hDevice);
        if (0 != nErrorCode)
        {
            _DisplayErrorInfo("关流失败", nErrorCode);
            break;
        }
        else
        {
            _DisplayErrorInfo("关流成功\r\n", 0);
        }

        VzNL_EndDetectLaser(hDevice);

        VzNL_CloseDevice(hDevice);

    } while (false);

    // 销毁SDK
    VzNL_Destroy();

    if (nullptr != s_hPauseEvent)
    {
        CloseHandle(s_hPauseEvent);
        s_hPauseEvent = nullptr;
    }

    return 0;
}
```

3.4 VzNLSDK 网络眼睛配置接口

头文件：#include "VZNL_EyeConfig.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.4.1 获取设备信息

int VzNL_GetEthernetEyeConfigInfo(const SVzNLEyeCBInfo* pEyeCBInfo, SVzNLEthernetEyeConfigInfo* psConfigInfo);

入参：　pEyeCBInfo　　设备信息

入参：　psConfigInfo　　眼睛参数

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

```
#include <stdio.h>
#include "VZNL_Common.h"

// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

if (0 < nDevCount)
{
    SVzNLEthernetEyeConfigInfo sOutputEthConfigInfo;
    int nErrorCode = VzNL_GetEthernetEyeConfigInfo(&pCBInfo[0], &sOutputEthConfigInfo);
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}

// 销毁SDK
VzNL_Destroy();
```

3.4.2 查询网络配置信息

入参： pEyeCBInfo 设备信息

出参： peIPType IP 类型[keNLEthernetEyeIPType_StaticIP:静态 IP keNLEthernetEyeIPType_DHCP:动态分配 IP]

出参： byEyeIP 眼睛 IP

出参： byMask 子网掩码

出参： byGateWay 网关

返回： 返回 0 表示成功

int VzNL_QueryEthernetInfo(const SVzNLEyeCBInfo* pEyeCBInfo,
EVzNLEthernetEyeIPType* peIPType,
unsigned char byEyeIP[4],
unsigned char byMask[4],
unsigned char byGateWay[4]);

```
#include <stdio.h>
#include "VZNL_Common.h"

// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

if (0 < nDevCount)
{
    EVzNLEthernetEyeIPType eIPType;
    unsigned char bySetEyeIP[4] = { 192, 168 , 10 , 113 };
    unsigned char byMask[4] = { 255, 255 , 255 , 0 };
    unsigned char byGW[4] = { 192, 168 ,10, 1 };
    int nEthRet = VzNL_QueryEthernetInfo(&pCBInfo[0], &eIPType, bySetEyeIP, byMask, byGW);
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}
```

```
// 销毁SDK
VzNL_Destroy();
```

3.4.3 解绑设备

int VzNL_UnBindEthernetEye(const SVzNLEyeCBInfo* pEyeCBInfo);

入参： pEyeCBInfo 设备信息

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.4.4 绑定设备

int VzNL_BindEthernetEye(const SVzNLEyeCBInfo* pEyeCBInfo);

入参： pEyeCBInfo 设备信息

返回： 检测到激光点的个数

```
#include <stdio.h>
#include "VZNL_Common.h"

// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

if (0 < nDevCount)
{
    SVzNLOpenDeviceParam sOpenDevParam;
    VZNLHANDLE hDevice = VzNL_OpenDevice(&pCBInfo[0], &sOpenDevParam, &nErrorCode);
    If ((hDevice != NULL) && (nErrorCode == 0))
    {
        if (VzFalse == pCBInfo[0].bValidDevice)
        {
            SVzNLEyeCBInfo sCurDevInfo = pCBInfo[0];
```

```
            SVzNLEthernetEyeConfigInfo sOutputEthConfigInfo;
            nErrorCode = VzNL_GetEthernetEyeConfigInfo(&sCurDevInfo,
&sOutputEthConfigInfo);
            if (0 != nErrorCode)
            {
                _DisplayErrorInfo("获取网络信息失败", nErrorCode);
            }

            //
            if (0 == memcmp(sOutputEthConfigInfo.byLocalIP,
sOutputEthConfigInfo.sNetCardInfo.byLocalIP, 4))
            {
                _DisplayErrorInfo("请查看是否有网络权限和管理员权限\r\n", 0);
            }
            else
            {
                nErrorCode = VzNL_BindEthernetEye(&sCurDevInfo);
                if (0 != nErrorCode)
                {
                    _DisplayErrorInfo("绑定相机失败", nErrorCode);
                }
                bCanResearch = VzTrue;
            }
        }
    }
    VzNL_CloseDevice(hDevice);
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}

// 销毁SDK
VzNL_Destroy();
```

3.5 VzNLSDK 激光检测接口

头文件：#include "VZNL_DetectLaser.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.5.1 开始激光检测

int VzNL_BeginDetectLaser(VZNLHANDLE hDevice);

入参： hDevice 设备句柄

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.5.2 结束激光检测

void VzNL_EndDetectLaser(VZNLHANDLE hDevice);

入参： hDevice 设备句柄

```
#include <stdio.h>
#include "VZNL_Common.h"

// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

if (0 < nDevCount)
{
    SVzNLOpenDeviceParam sOpenDevParam;
    VZNLHANDLE hDevice = VzNL_OpenDevice(&pCBInfo [0], &sOpenDevParam, &nErrorCode);
    if ((hDevice != NULL) && (nErrorCode == 0))
    {
        int nErrorCode = VzNL_BeginDetectLaser(hDevice);
        if (0 != nErrorCode)
        {
            return;
        }

        // 执行激光线检测相关函数
        VzNL_EndDetectLaser(hDevice);
    }
    VzNL_CloseDevice(hDevice);
```

```
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}

// 销毁SDK
VzNL_Destroy();
```

3.5.3 激光单线检测

int VzNL_DetectLaser(VZNLHANDLE hDevice, int nPointInterval);

入参：	hDevice	设备句柄
	nPointInterval	点的间隔
返回：	0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。	

3.5.4 获取激光线结果点的个数

int VzNL_GetLaserResultPointCount(VZNLHANDLE hDevice);

入参：	hDevice	设备句柄
返回：	检测到激光点的个数	

3.5.5 获取激光线 2D 结果

int VzNL_GetLaser2DResult(VZNLHANDLE hDevice, SVzNL2DPosition* p2DPoint, int* pnCount);

入参：	hDevice	设备句柄
	pnCount	2D 点的个数
出参：	p2DPoint	2D 点坐标首地址
返回：	0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。	

3.5.6 获取激光线 3D 结果

int VzNL_GetLaser3DResult(VZNLHANDLE hDevice, SVzNL3DPosition* p3DPoint, int* pnCount);

入参：	hDevice	设备句柄
	pnCount	3D 点的个数
出参：	p3DPoint	3D 点坐标首地址
返回：	0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。	

int VzNL_GetLaserImageResult(VZNLHANDLE hDevice, SVzNLImageData** ppLeftImageData, SVzNLImageData** ppRightImageData);

入参：	hDevice	设备句柄
出参：	ppLeftImageData	左图像地址
	ppRightImageData	右图像地址
返回：	0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。	

```
#include <stdio.h>
#include "VZNL_Common.h"

// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

if (0 < nDevCount)
{
    SVzNLOpenDeviceParam sOpenDevParam;
    VZNLHANDLE hDevice = VzNL_OpenDevice(&pCBInfo [0], &sOpenDevParam, &nErrorCode);
    if ((hDevice != NULL) && (nErrorCode == 0))
    {
        int nErrorCode = VzNL_BeginDetectLaser(hDevice);
        if (0 != nErrorCode)
        {
            return;
        }

        // 执行激光线检测相关函数
        SVzNL3DPosition* p3DPosition = NULL;
        Int n3DPointCount = 0;
        SVzNL2DPosition*    p2DPosition = NULL;
        int                 n2DPositionCount = 0;
        SVzNLImageData* pLeftImageData = NULL;
        SVzNLImageData* pRightImageData = NULL;
      nErrorCode = VzNL_DetectLaser(hDevice, 1);
    if (0 != nErrorCode)
    {
```

```
        _ShowErrorInfo(nErrorCode);
        return nErrorCode;
    }
    if (0 == VzNL_GetLaser3DResult(hDevice, NULL, &n3DPointCount))
    {
        p3DPosition = (SVzNL3DPosition*)malloc(n3DPointCount * sizeof(SVzNL3DPosition));
        VzNL_GetLaser3DResult(hDevice, p3DPosition, &n3DPointCount);
    }

    if (0 == VzNL_GetLaser2DResult(hDevice, NULL, &n2DPositionCount))
    {
        p2DPosition = (SVzNL2DPosition*)malloc(n2DPositionCount *
sizeof(SVzNL2DPosition));
        VzNL_GetLaser2DResult(hDevice, p2DPosition, &n2DPositionCount);
    }
    for (n = 0; n < n2DPositionCount && n < n3DPointCount; n++)
    {
        printf("index %d Left X:%d Y:%d Right X:%d Y:%d \t X:%.5f Y:%.5f Z:%.5f\n",
        p3DPosition[n].nPointIdx, p2DPosition[n].ptLeft2D.x, p2DPosition[n].ptLeft2D.y,
        p2DPosition[n].ptRight2D.x, p2DPosition[n].ptRight2D.y,
        p3DPosition[n].pt3D.x, p3DPosition[n].pt3D.y, p3DPosition[n].pt3D.z);
    }

    VzNL_ReleaseImage(&pLeftImageData);
    VzNL_ReleaseImage(&pRightImageData);

    if (NULL != p3DPosition)
    {
       free(p3DPosition);
        p3DPosition = NULL;
    }
     if (NULL != p2DPosition)
     {
        free(p2DPosition);
        p2DPosition = NULL;
     }

        nErrorCode = VzNL_EndDetectLaser(hDevice);
        if (0 != nErrorCode)
        {
            return;
        }
    }
    VzNL_CloseDevice(hDevice);
```

```
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}

// 销毁SDK
VzNL_Destroy();
```

3.5.7 设置波峰波谷凹槽最小深度

int VzNL_SetLaserMinSlotDeep(VZNLHANDLE hDevice, double dMinSlotDeep);

入参：	hDevice	设备句柄
出参：	dMinSlotDeep	凹槽最小深度
返回：	0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。	

3.5.8 取激光凹槽数据结果的波峰点的个数

int VzNL_GetLaserSlotResultPeakPonitCount(VZNLHANDLE hDevice);

入参：	hDevice	设备句柄
返回：	0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。	

3.5.9 获取激光凹槽数据结果的波谷点的个数

int VzNL_GetLaserSlotResultValleyPonitCount(VZNLHANDLE hDevice);

入参：	hDevice	设备句柄
出参：	dMinSlotDeep	凹槽最小深度
返回：	0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。	

3.5.10 获取激光凹槽 2D 结果

int VzNL_GetLaserSlot2DResult(VZNLHANDLE hDevice, SVzNL2DPosition* pPeakPoint, int* pnPeakPointCount, SVzNL2DPosition* pValleyPoint, int* pnValleyPointCount);

入参：	hDevice	设备句柄
出参：	dMinSlotDeep	凹槽最小深度
	pPeakPoint	波峰点，内存空间由用户分配
	pnPeakPointCount	pPeakPoint 的结构个数
	pValleyPoint	波谷点，内存空间由用户分配
	pnValleyPointCount	pValleyPoint 的结构个数
返回：	0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。	

3.5.11 获取激光凹槽 3D 结果

int VzNL_GetLaserSlot3DResult(VZNLHANDLE hDevice, SVzNL3DPosition* pPeakPoint, int* pnPeakPointCount, SVzNL3DPosition* pValleyPoint, int* pnValleyPointCount);

入参： hDevice 设备句柄
出参： dMinSlotDeep 凹槽最小深度
pPeakPoint 波峰点，内存空间由用户分配
pnPeakPointCount pPeakPoint 的结构个数
pValleyPoint 波谷点，内存空间由用户分配
pnValleyPointCount pValleyPoint 的结构个数
返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。
激光线自动检测并回调

3.5.12 开始自动检测

int VzNL_StartAutoDetect(VZNLHANDLE hDevice, EVzFlipType eFlipType, VzNL_GetAutoDtectResultCB pCB， void* pCBParam);

入参： hDevice 设备句柄
eFlipType 激光线方向
pCB 回调接口
pCBParam 对调中会原值返回
返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

回调函数声明方式

typedef int(*VzNL_GetAutoDtectResultCB)(SVzNL3DPosition* p3DPoint, SVzNL2DPosition* p2DPoint, int nCount, unsigned long long nTimeStamp, double dTotleOffset, double dStep,SVzNLImageData* pLeftEye, SVzNLImageData* pRightEye， void* pParam);

结束自动检测

int VzNL_StopAutoDetect(VZNLHANDLE hDevice);

入参： hDevice 设备句柄
返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

是否在自动检测中

VzBool VzNL_IsAutoDetecting(VZNLHANDLE hDevice, int* pnErrorCode);

```
#include <stdio.h>
#include "VZNL_Common.h"

// 激光线回调
int _GetAutoDetectResultCB(SVzNL3DPosition* p3DPosition, SVzNL2DPosition* p2DPosition,
int n3DPointCount, unsigned long long nTimeStamp, double dTotleStep, double dStep,
SVzNLImageData* pLeftImage, SVzNLImageData* pRightImage, void* pParam)
{
    if (n3DPointCount > 0)
    {
        printf("Auto Detect Count %.1f %.1f %.1f / %.11f\r\n", p3DPosition[0].pt3D.x,
p3DPosition[0].pt3D.y, p3DPosition[0].pt3D.z, dTotleStep);
    }

// 初始化SDK
```

```
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

if (0 < nDevCount)
{
    SVzNLOpenDeviceParam sOpenDevParam;
    VZNLHANDLE hDevice = VzNL_OpenDevice(&pCBInfo [0], &sOpenDevParam, &nErrorCode);
    if ((hDevice != NULL) && (nErrorCode == 0))
    {
        int nErrorCode = VzNL_BeginDetectLaser(hDevice);
        if (0 != nErrorCode)
        {
            return;
        }

        // 执行激光线检测相关函数
        nErrorCode = VzNL_StartAutoDetect(hDevice, keFlipType_Vertical,
_GetAutoDetectResultCB, hDevice);
         if (0 != nErrorCode)
         {
            return;
         }

        // 等待 5s(采集 5s 数据)
        Sleep(5000);

         VzNL_StopAutoDetect(hDevice);
         nErrorCode = VzNL_EndDetectLaser(hDevice);
         if (0 != nErrorCode)
```

```
        {
            return;
        }
    }
    VzNL_CloseDevice(hDevice);
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}

// 销毁SDK
VzNL_Destroy();
```

3.5.13 激光线自动检测函数

int VzNL_StartAutoDetectEx(VZNLHANDLE hDevice, EVzResultDataType eResultType, EVzFlipType eFlipType, VzNL_AutoOutputLaserLineExCB pCB, void* pCBParam);

入参： hDevice 设备句柄
eResultType 结果类型
eFlipType 激光线方向
pCB 回调函数
pCBParam 回调参数

返回： 0 为正确，其他值可通过 VzNL_GetErrorInfo 获取

回调函数：

typedef void(*VzNL_AutoOutputLaserLineExCB)(EVzResultDataType eDataType, SVzLaserLineData* pLaserLinePoint, void* pParam);

eDataType 数据类型，与开始检测入参相同

枚举值	3d 数据类型	2d 数据类型
keResultDataType_Position	SVzNL3DPosition	SVzNL2DPosition
keResultDataType_PointF	SVzNL3DPointF	SVzNL2DLRPoint
keResultDataType_PointXYZ,	SVzNLPointXYZ	SVzNL2DLRPoint
keResultDataType_PointXYZRGBA	SVzNLPointXYZRGBA	SVzNL2DLRPoint

pLaserLinePoint 输出激光线数据
pParam 返回回调时传入的参数

typedef struct
{
    void* p3DPoint; //< 3D 点 根据 ResultDataType 使用不同的数据结构
    void* p2DPoint; //< 2D 点 根据 ResultDataType 使用不同的数据结构
    int nPointCount; //< 2D 3D 数据个数

```
    double              dTotleOffset;   //< 总偏移
    double              dStep;          //< 距离上一条线的偏移
    unsigned long long  llFrameIdx;     //< 帧号
    unsigned long long  llTimeStamp;    //< 时间戳
    unsigned int        nEncodeNo;      //< 编码器值
} SVzLaserLineData;
```

结束自动检测

int VzNL_StopAutoDetect(VZNLHANDLE hDevice);

入参： hDevice 设备句柄

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

```
#include <stdio.h>
#include "VZNL_Common.h"

//
void _AutoOutputLaserLineExCB(EVzResultDataType eDataType, SVzLaserLineData*
pLaserLinePoint, void* pParam)
{
    if (nullptr == pLaserLinePoint)
    {
        return;
    }
     if (pLaserLinePoint->nPointCount > 0)
    {
        if (keResultDataType_Position == eDataType)      //< Position数据类型 使用
SVzNL3DPosition, SVzNL2DPosition
        {
            SVzNL3DPosition* p3DPosition = (SVzNL3DPosition*)pLaserLinePoint->p3DPoint;
            SVzNL2DPosition* p2DPosition = (SVzNL2DPosition*)pLaserLinePoint->p2DPoint;
            printf("{%.1lf, %.1lf, %.1lf}-{%d,%d}-{%d,%d}\r\n", p3DPosition->pt3D.x,
p3DPosition->pt3D.y, p3DPosition->pt3D.z,
                p2DPosition->ptLeft2D.x, p2DPosition->ptLeft2D.y,
p2DPosition->ptRight2D.x, p2DPosition->ptRight2D.y);
        }
        else if (keResultDataType_PointF == eDataType)       //< PointF数据类型     使用
SVzNL3DPointF SVzNL2DPointF
        {
            SVzNL3DPointF* p3DPoint = (SVzNL3DPointF*)pLaserLinePoint->p3DPoint;
            SVzNL2DLRPoint* p2DPoint = (SVzNL2DLRPoint*)pLaserLinePoint->p2DPoint;
            printf("{%.1lf, %.1lf, %.1lf}-{%d,%d}-{%d,%d}\r\n", p3DPoint->x,
p3DPoint->y, p3DPoint->z,
                p2DPoint->sLeft.x, p2DPoint->sLeft.y, p2DPoint->sRight.x,
p2DPoint->sRight.y);
        }
```

```
        else if (keResultDataType_PointXYZ == eDataType)      //< PointXYZ数据类型  使用
PointXYZ
        {
            SVzNLPointXYZ* p3DPoint = (SVzNLPointXYZ*)pLaserLinePoint->p3DPoint;
            SVzNL2DLRPoint* p2DPoint = (SVzNL2DLRPoint*)pLaserLinePoint->p2DPoint;
            printf("{%.1lf, %.1lf, %.1lf}-{%d,%d}-{%d,%d}\r\n", p3DPoint->x,
p3DPoint->y, p3DPoint->z,
                p2DPoint->sLeft.x, p2DPoint->sLeft.y, p2DPoint->sRight.x,
p2DPoint->sRight.y);
        }
        else if (keResultDataType_PointGray == eDataType)     //< PointXYZRGB数据  使用
SVzPointXYZRGB
        {
        }
        else if (keResultDataType_PointXYZRGBA == eDataType) //< PointXYZRGB数据  使用
SVzPointXYZRGB
        {
        }
    }
}

// 主函数相关片段
// 初始化SDK
SVzNLConfigParam sConfigParam;
sConfigParam.nDeviceTimeOut = 0;
int nRet = VzNL_Init(&sConfigParam);
if (0 != nRet)
{
   printf("SDK 多开\n")
}

SVzNLEyeCBInfo* pCBInfo = NULL;
int nDevIdx = 0; int nDevCount = 0;

VzNL_GetEyeCBDeviceInfo(NULL, &nDevCount);
if (nDevCount == 0)
    return;

pCBInfo = (SVzNLEyeCBInfo*)malloc(sizeof(SVzNLEyeCBInfo) * nDevCount);
VzNL_GetEyeCBDeviceInfo(pCBInfo, &nDevCount);

if (0 < nDevCount)
{
    SVzNLOpenDeviceParam sOpenDevParam;
```

```
    VZNLHANDLE hDevice = VzNL_OpenDevice(&pCBInfo [0], &sOpenDevParam, &nErrorCode);
    if ((hDevice != NULL) && (nErrorCode == 0))
    {
        int nErrorCode = VzNL_BeginDetectLaser(hDevice);
        if (0 != nErrorCode)
        {
            return;
        }

        nErrorCode = VzNL_StartAutoDetectEx(hDevice, keResultDataType_PointXYZRGBA,
keFlipType_None, _AutoOutputLaserLineExCB, hDevice);
        if(0 != nErrorCode)
        {
            return;
        }

        // 等待 5s(采集 5s 数据)
        Sleep(5000);

        VzNL_StopAutoDetect(hDevice);

        nErrorCode = VzNL_EndDetectLaser(hDevice);
        if (0 != nErrorCode)
        {
            return;
        }
    }
    VzNL_CloseDevice(hDevice);
}

if (NULL != pCBInfo)
{
    free(pCBInfo);
    pCBInfo = NULL;
}

// 销毁SDK
VzNL_Destroy();
```

3.5.14 设置过滤高度

int VzNL_ConfigLaserLineFilterHeight(VZNLHANDLE hDevice, double dFilterHeight);

入参： hDevice 设备句柄

入参： dFilterHeight 过滤高度（未高度标定时，大于此高度的数据不输出，高度标定后，

小于此数据的高度不输出）

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.5.15 获取过滤高度

int VzNL_GetLaserLineFilterHeight(VZNLHANDLE hDevice, double* pdFilterHeight);

入参： hDevice 设备句柄

出参： pdFilterHeight 过滤高度

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.5.16 启用/禁用过滤高度

int VzNL_EnableLaserLineFilterHeight(VZNLHANDLE hDevice, VzBool bEnable);

入参： hDevice 设备句柄

入参： bEnable 启用高度过滤

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.5.17 是否启用过滤高度

VzBool VzNL_IsEnableLaserLineFilterHeight(VZNLHANDLE hDevice, int* pnErrorCode);

入参： hDevice 设备句柄

出参： pnErrorCode 错误码

返回： 启用返回 VzTrue 失败返回 VzFalse

3.5.18 设置杂点过滤阈值

int VzNL_ConfigLaserLineMaxDeviation(VZNLHANDLE hDevice, double dMaxDeviation);

入参： hDevice 设备句柄

入参： dMaxDeviation 杂点过滤阈值

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息

3.5.19 设置偏移量

int VzNL_ConfigLaserBeginOffsetValue(VZNLHANDLE hDevice, double dOffsetValue);

入参： hDevice 设备句柄

入参： dOffsetValue 偏移位置

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息

3.5.20 设置传送带速度值

int VzNL_ConfigLaserObjRunSpeedValue(VZNLHANDLE hDevice, EVzObjRunDirect eDirect, double dSpeed);

入参： hDevice 设备句柄

eDirect keObjRunDirect_Plus 为正方向,

keObjRunDirect_Minus 为反方向

dSpeed 传送带移动速度

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.5.21 获取传送带速度值

int VzNL_GetLaserObjRunSpeedValue(VZNLHANDLE hDevice, double* pdSpeed);

入参： hDevice 设备句柄
出参： pdSpeed 传送带移动速度
返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.5.22 计算平均高度/最低高度/最高高度

void VzNL_CalcLaserZHeight(SVzNL3DPosition* p3DPoint, int nCount, double* pdAvgHeight, double* pdMinHeight, double* pdMaxHeight);

入参： p3DPoint 要计算的 3D 点【Array】
入参： nCount 点个数
入参： pdAvgHeight 平均高度
入参： pdMinHeight 最小高度
入参： pdMaxHeight 最大高度
返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.5.23 设置点云处理模式

入参： hDevice 设备句柄
入参： ePointCloudProcMode
*kePointCloudProcMode_Speed 速度计算模式
*kePointCloudProcMode_Encoder 编码器模式
*kePointCloudProcMode_FixedStep 固定步长模式
返回： 成功返回 0，否则为其他错误码

int VzNL_SetPointCloudProcMode(VZNLHANDLE hDevice, EVzPointCloudProcMode ePointCloudProcMode);

3.5.24 设置固定步长

设置前请先设置为 kePointCloudProcMode_FixedStep 模式

int VzNL_SetLaserLineFixedStep(VZNLHANDLE hDevice, double dStep);

3.5.25 配置转动轴半径

入参： hDevice 设备句柄
入参： dRadius 转动轴半径
返回： 成功返回 0，否则为其他错误码

int VzNL_ConfigLaserRollerRadius(VZNLHANDLE hDevice, double dRadius);

3.5.26 设置编码器脉冲精度

入参：hDevice 设备句柄
入参： nPulsePerRound 脉冲个数
返回： 成功返回 0，否则为其他错误

int VzNL_SetLaserEncoderResolution(VZNLHANDLE hDevice, unsigned int nPulsePerRound);

3.5.27 激光检测填充点模式

入参： hDevice 设备句柄
入参： bFillPoint VzTrue 为填充，VzFalse 为不填充(默认为 VzFalse)
返回： 成功返回 0，否则为其他错误码

int VzNL_EnableFillLaserPoint(VZNLHANDLE hDevice, VzBool bFillPoint);

3.5.28 激光检测扫描 RGB 原图的获取

入参： hDevice 设备句柄

出参： ppResultImage 输出 RGB 图像，需要用户调用 VzNL_ReleaseImage 进行数据释放

int VzNL_GetAutoDetectResultSurface(VZNLHANDLE hDevice, SVzNLImageData** ppResultImage);

```
#include <stdio.h>
#include <vector>
#include <list>
#include "VZNL_Common.h"
#include "VZNL_EyeConfig.h"
#include "VZNL_DetectLaser.h"
#include "VZNL_Graphics.h"
#include "VZNL_DetectConfig.h"
#include "VZNL_Utils.h"
#include "VZNL_SwingMotor.h"
#include "VZNL_RGBConfig.h"
#ifdef _WIN32
#include "windows.h"
#else
#include <memory.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#endif
#include <fstream>
#include <atomic>

/*****************************************************************************
 * File: vizumsdk_save_color_image_and_depth_map.cpp
 * Detail: 静态相机 第三目图像 点云图像 深度图 生成的 Sample
 * Author: Vizum
 * Date: 20210609
 *****************************************************************************/

static SVzVideoResolution g_sRGBVideoRes;
static SVzNLPointXYZRGBA* g_p2DToPointMap = nullptr;
static unsigned char* g_p2DRGBPic = nullptr;
static bool* g_pb2DInvalidPt = nullptr;
static std::atomic<bool> g_bPauseEvent(true);

/// @name 自动检测回调函数
/// @{
```

```
void _AutoOutputLaserLineExCB(EVzResultDataType eDataType, SVzLaserLineData* pLaserLinePoint, void*
pParam)
{
    // 当需要的点云格式为SVzNLPointXYZRGBA时
    if (keResultDataType_PointXYZRGBA == eDataType)
    {
        SVzNLPointXYZRGBA* p3DPoint = (SVzNLPointXYZRGBA*)pLaserLinePoint->p3DPoint;
        SVzNL2DLRPoint* p2DPoint = (SVzNL2DLRPoint*)pLaserLinePoint->p2DPoint;
        for (int nPtIdx = 0; nPtIdx < pLaserLinePoint->nPointCount; nPtIdx++)
        {
            if ((p2DPoint->sLeft.x < 0) || (p2DPoint->sLeft.x >= g_sRGBVideoRes.nFrameWidth))
            {
                p3DPoint++;
                p2DPoint++;
                continue;
            }

            if ((p2DPoint->sLeft.y < 0) || (p2DPoint->sLeft.y >= g_sRGBVideoRes.nFrameHeight))
            {
                p3DPoint++;
                p2DPoint++;
                continue;
            }

            int nPos = p2DPoint->sLeft.x + p2DPoint->sLeft.y * g_sRGBVideoRes.nFrameWidth;

            // 更新Depth Map
            g_p2DToPointMap[nPos] = *p3DPoint;
            g_pb2DInvalidPt[nPos] = true;

            // 更新点云RGB图
            unsigned char* pRGB = (unsigned char*)&p3DPoint->nRGB;
            unsigned char* pCurRGB = g_p2DRGBPic + nPos * 3;
            pCurRGB[0] = pRGB[0];
            pCurRGB[1] = pRGB[1];
            pCurRGB[2] = pRGB[2];
            p3DPoint++;
            p2DPoint++;
        }
    }
}

/// @brief
/// 获取文件纯路径
```

```
int _GetPurePath(const char* szFilePath, char szPurePath[VZ_MAXPATH])
{
	int nFindPos = -1;
	const char* pPos = szFilePath;
	while ('\0' != *pPos)
	{
		if ('/' == *pPos || '\\' == *pPos)
		{
			nFindPos = static_cast<int>(pPos - szFilePath);
		}
		pPos++;
	}

	if (-1 == nFindPos)
		return -1;

	memcpy(szPurePath, szFilePath, static_cast<unsigned int>(nFindPos));
	szPurePath[nFindPos] = '\0';
	return 0;
}

// 显示错误信息
void _DisplayErrorInfo(const char* pszDesc, int nErrorCode)
{
	char szOutput[256];
	if (0 == nErrorCode)
	{
#ifdef _WIN32
		sprintf_s(szOutput, 256, "%s\r\n", pszDesc);
#else
		sprintf(szOutput, "%s\r\n", pszDesc);
#endif
	}
	else
	{
		char szErrorInfo[256] = { 0 };
		VzNL_GetErrorInfo(nErrorCode, szErrorInfo);
#ifdef _WIN32
		sprintf_s(szOutput, 256, "%s [%d] %s\r\n", pszDesc, nErrorCode, szErrorInfo);
#else
		sprintf(szOutput, "%s [%d] %s\r\n", pszDesc, nErrorCode, szErrorInfo);
#endif
	}
	printf("%s", szOutput);
```

```
}

// 搜索设备
void _ResearchDevice(std::vector<SVzNLEyeCBInfo>& vetDevice)
{
	// 搜索相机
	VzNL_ResearchDevice(keSearchDeviceFlag_EthLaserRobotEye);

	// 遍历相机
	int nDevCount = 0;
	VzNL_GetEyeCBDeviceInfo(nullptr, &nDevCount);
	if (nDevCount <= 0)
	{
		return;
	}

	vetDevice.resize(nDevCount);
	VzNL_GetEyeCBDeviceInfo(vetDevice.data(), &nDevCount);
}

// @brief 搜索且进行绑定设备
void _SearchAndBindDevice(std::vector<SVzNLEyeCBInfo>& vetDevice)
{
	int nErrorCode = 0;
	VzBool bCanResearch = VzFalse;
	do
	{
		bCanResearch = VzFalse;
		_ResearchDevice(vetDevice);
		std::vector<SVzNLEyeCBInfo>::iterator itDevice = vetDevice.begin();
		while (itDevice != vetDevice.end())
		{
			// 相机没有被识别
			if (VzFalse == itDevice->bValidDevice)
			{
				SVzNLEyeCBInfo sCurDevInfo = *itDevice;
				SVzNLEthernetEyeConfigInfo sOutputEthConfigInfo;
				nErrorCode = VzNL_GetEthernetEyeConfigInfo(&sCurDevInfo, &sOutputEthConfigInfo);
				if (0 != nErrorCode)
				{
					_DisplayErrorInfo("获取网络信息失败", nErrorCode);
				}

				//
```

```
                if (0 == memcmp(sOutputEthConfigInfo.byLocalIP,
sOutputEthConfigInfo.sNetCardInfo.byLocalIP, 4))
                {
                    _DisplayErrorInfo("请查看是否有网络权限和管理员权限\r\n", 0);
                }
                else
                {
                    nErrorCode = VzNL_BindEthernetEye(&sCurDevInfo);
                    if (0 != nErrorCode)
                    {
                        _DisplayErrorInfo("绑定相机失败", nErrorCode);
                    }
                    bCanResearch = VzTrue;
                }
            }
            itDevice++;
        }
    } while (bCanResearch);
}

/**
 * @brief 设备状态改变CallBack函数
 * @param eStatus [out] 状态
 * @param pInfoParam [out] 状态参数
 */
void _OnNotifyStatusCB(EVzDeviceWorkStatus eStatus, void* pInfoParam)
{
    if (keDeviceWorkStatus_Device_Auto_Stop == eStatus)
    {
        printf("收到停止信号\r\n");
        g_bPauseEvent = true;
    }
}

int main()
{
    g_bPauseEvent = true;

    // 初始化SDK
    SVzNLConfigParam sConfigParam;
    sConfigParam.nDeviceTimeOut = 0;
    int nRet = VzNL_Init(&sConfigParam);
    if (0 != nRet)
    {
```

```
        _DisplayErrorInfo("查看是否有其他程序在使用SDK?", nRet);
        return 0;
    }

    VzNL_SetLogLevel(keNLLogLevel_Information, keNLLogType_File);

    // 枚举相机
    std::vector<SVzNLEyeCBInfo> vetDevice;
    _SearchAndBindDevice(vetDevice);

    //
    int nErrorCode = 0;
    std::vector<VZNLHANDLE>          vetHandles;
    if (vetDevice.size() > 0)
    {
        _DisplayErrorInfo("显示当前可用相机\r\n", 0);
        std::vector<SVzNLEyeCBInfo>::iterator itDevice = vetDevice.begin();
        while (itDevice != vetDevice.end())
        {
            // 相机没有被识别
            if (VzFalse == itDevice->bValidDevice)
            {
                itDevice++;
                continue;
            }

            char szOutput[256];
#ifdef _WIN32
            sprintf_s(szOutput, 256, "----------------------准备打开  %s----------------------\r\n",
itDevice->byServerIP);
#else
            sprintf(szOutput, "----------------------准备打开  %s----------------------\r\n",
itDevice->byServerIP);
#endif
            _DisplayErrorInfo(szOutput, 0);

            // 打开相机
            SVzNLOpenDeviceParam sOpenDevParam;
            VZNLHANDLE hDevice = VzNL_OpenDevice(&(*itDevice), &sOpenDevParam,
&nErrorCode);
            if (0 != nErrorCode)
            {
                _DisplayErrorInfo("打开设备失败", nErrorCode);
                break;
```

```
            }
            else
            {
                char szOutput[256];
#ifdef _WIN32
                sprintf_s(szOutput, 256, "打开相机%s成功\r\n", itDevice->byServerIP);
#else
                sprintf(szOutput, "打开相机%s成功\r\n", itDevice->byServerIP);
#endif
                _DisplayErrorInfo(szOutput, 0);
            }

            if (NULL != hDevice)
            {
                vetHandles.push_back(hDevice);
            }

            VzNL_SetOutputImageFormat(keVzNLImageType_BGR888);

            if (VzFalse == VzNL_IsSupportRGBCamera(hDevice, nullptr))
            {
                _DisplayErrorInfo("当前设备不支持RGB", nErrorCode);
                break;
            }

            if (VzFalse == VzNL_IsSupportSwingMotor(hDevice, nullptr))
            {
                break;
            }

            // 启用RGB Sensor
            VzNL_EnableRGB(hDevice, VzTrue);

            // 注册回调
            VzNL_SetDeviceStatusNotify(hDevice, _OnNotifyStatusCB, NULL);

            // 创建激光线检测工具
            nErrorCode = VzNL_BeginDetectLaser(hDevice);
            if (0 != nErrorCode)
            {
                _DisplayErrorInfo("创建检测激光线工具", nErrorCode);
                break;
            }
            else
```

```
                {
                    _DisplayErrorInfo("创建激光线检测工具成功\r\n", 0);
                }

                VzNL_GetRGBResolution(hDevice, &g_sRGBVideoRes);

                int nFrameSize = g_sRGBVideoRes.nFrameWidth * g_sRGBVideoRes.nFrameHeight;
                g_p2DToPointMap = new SVzNLPointXYZRGBA[nFrameSize];
                g_p2DRGBPic = new unsigned char[nFrameSize * 3];
                g_pb2DInvalidPt = new bool[nFrameSize];
                memset(g_pb2DInvalidPt, 0, sizeof(bool) * nFrameSize);

                // 设置到主模式
                nErrorCode = VzNL_SetTriggerMode(hDevice, keEyeTriggerMode_Master);
                if (0 != nErrorCode)
                {
                    _DisplayErrorInfo("设置主模式失败", nErrorCode);
                    break;
                }
                else
                {
                    _DisplayErrorInfo("主模式设置成功\r\n", 0);
                }

                _DisplayErrorInfo("启用计算中间2D图像模式", 0);

                // 采集激光线
                unsigned int nMinFrameRate, nMaxFrameRate;
                VzNL_QueryParamRange(hDevice, keDeviceParamType_FrameRate, &nMinFrameRate,
&nMaxFrameRate);

                VzNL_SetFrameRate(hDevice, nMaxFrameRate);

                g_bPauseEvent = false;

                VzNL_EnableSwingMotor(hDevice, VzTrue);
                // 开始流模式检测
                nErrorCode = VzNL_StartAutoDetectEx(hDevice, keResultDataType_PointXYZRGBA,
keFlipType_None, _AutoOutputLaserLineExCB, nullptr);
                if (0 != nErrorCode)
                {
                    _DisplayErrorInfo("开流失败", nErrorCode);
                    break;
                }
```

```
                else
                {
                    _DisplayErrorInfo("开流成功\r\n", 0);
                }

                // 等待结束信号
                while (false == g_bPauseEvent)
                {
#ifdef _WIN32
                    Sleep(1);
#else
                    usleep(1);
#endif
                }

                nErrorCode = VzNL_StopAutoDetect(hDevice);
                if (0 != nErrorCode)
                {
                    _DisplayErrorInfo("关流失败", nErrorCode);
                    break;
                }
                else
                {
                    _DisplayErrorInfo("关流成功\r\n", 0);
                }

                SVzNLImageData* psCenterImage = nullptr;
                VzNL_GetAutoDetectResultSurface(hDevice, &psCenterImage);

                // 存储灰度图
                char szPurePath[VZ_MAXPATH] = { 0 };
#ifdef _WIN32
                char szModule[VZ_MAXPATH] = { 0 };
                GetModuleFileNameA(nullptr, szModule, VZ_MAXPATH);
                _GetPurePath(szModule, szPurePath);
#else
                getcwd(szPurePath, VZ_MAXPATH);
#endif
                char szGrayFile[VZ_MAXPATH] = { 0 };
                if (psCenterImage)
                {
#ifdef _WIN32
                    sprintf_s(szGrayFile, VZ_MAXPATH, "%s\\Gray.png", szPurePath);
#else
```

```
                sprintf(szGrayFile, "%s\\Gray.png", szPurePath);
#endif
                VzNL_SaveImage(szGrayFile, psCenterImage);
            }

            if (g_p2DRGBPic)
            {
                // 根据点云生成的RGB图像
                SVzNLImageData sImageData;
                sImageData.byBitDepth = 8;
                sImageData.eImageType = keVzNLImageType_BGR888;
                sImageData.nBufferSize = nFrameSize * 3;
                sImageData.nChannels = 3;
                sImageData.nHeight = g_sRGBVideoRes.nFrameHeight;
                sImageData.nWidth = g_sRGBVideoRes.nFrameWidth;
                sImageData.pBuffer = g_p2DRGBPic;
                sImageData.ptOriPos.x = 0;
                sImageData.ptOriPos.y = 0;
#ifdef _WIN32
                sprintf_s(szGrayFile, VZ_MAXPATH, "%s\\PointRGB.png", szPurePath);
#else
                sprintf(szGrayFile, "%s\\PointRGB.png", szPurePath);
#endif
                VzNL_SaveImage(szGrayFile, &sImageData);
            }

            // 存储深度图
            if (nullptr != g_p2DToPointMap)
            {
                char szTifFile[VZ_MAXPATH] = { 0 };
#ifdef _WIN32
                sprintf_s(szTifFile, VZ_MAXPATH, "%s\\Depth.tif", szPurePath);
#else
                sprintf(szTifFile, "%s\\Depth.tif", szPurePath);
#endif
                VzNL_SaveDepthMapTiffImage(szTifFile, g_sRGBVideoRes.nFrameWidth,
g_sRGBVideoRes.nFrameHeight, keResultDataType_PointXYZRGBA, g_p2DToPointMap);
            }

            // 输出点云映射数据
            if (nullptr != g_p2DToPointMap)
            {
                delete[] g_p2DToPointMap;
                g_p2DToPointMap = nullptr;
```

```
                }
                if (nullptr != g_pb2DInvalidPt)
                {
                    delete[] g_pb2DInvalidPt;
                    g_pb2DInvalidPt = nullptr;
                }

                if (psCenterImage)
                {
                    VzNL_ReleaseImage(&psCenterImage);
                }
                VzNL_EndDetectLaser(hDevice);
                VzNL_CloseDevice(hDevice);

                itDevice++;
            }
        }

        // 销毁SDK
        VzNL_Destroy();

        getchar();

        return 0;
}
```

3.5.29 配置编码器使用参数类型【SDK3.6.20.10 新增】

入参: hDevice 设备句柄

入参: eParamType 编码器参数类型

使用 编码器分辨率 + 编码器半径参数 时配置 keEncodeParamType_RollerResolution

使用 编码器每两个脉冲的间距 时配置 keEncodeParamType_DistancePerPulse

返回: 成功时返回 0，失败返回错误码

```
int VzNL_SetLaserEncoderParamType(VZNLHANDLE hDevice, EVzEncodeParamType eParamType);
int VzNL_GetLaserEncoderParamType(VZNLHANDLE hDevice, EVzEncodeParamType* peParamType);
```

3.5.30 配置编码器间距【SDK3.6.20.10 新增】

入参: hDevice 设备句柄

入参: dDistancePerPulse 编码器脉冲间隔距离

注意：需要配置 VzNL_ SetLaserEncoderParamType 使用

返回: 成功时返回 0，失败返回错误码

```
int VzNL_SetLaserEncoderDistancePerPulse(VZNLHANDLE hDevice, double dDistancePerPulse);
int VzNL_GetLaserEncoderDistancePerPulse(VZNLHANDLE hDevice, double* pdDistancePerPulse);
```

```
// 设置到下降沿触发模式
```

```
int nErrorCode = VzNL_SetTriggerMode(hDevice, keEyeTriggerMode_FallingEdge);
if (0 != nErrorCode)
{
	_DisplayErrorInfo("设置下降沿触发失败", nErrorCode);
	break;
}
else
{
	_DisplayErrorInfo("下降沿触发设置成功\r\n", 0);
}

// 配置点云偏移计算为编码器计算
VzNL_SetPointCloudProcMode(hDevice, kePointCloudProcMode_Encoder);

// 编码器分辨率为 1000, 连接编码器的传动轮的周长为 500 时，计算当前距离
double dDistancePerPulse = 500. / 1000.;
VzNL_SetLaserEncoderParamType(hDevice, keEncodeParamType_DistancePerPulse);
VzNL_SetLaserEncoderDistancePerPulse(hDevice, dDistancePerPulse);
```

3.5.31 配置/获取激光线过滤范围

入参: hDevice 设备句柄

入参: eFilterType 过滤类型

入参: dRange 过滤范围

返回: 成功时返回 0，失败返回错误码

```
int VzNL_SetLaserLineFilterRange(VZNLHANDLE hDevice, EVzLaserPointFilterType eFilterType,
double dRange[2]);
int VzNL_GetLaserLineFilterRange(VZNLHANDLE hDevice, EVzLaserPointFilterType eFilterType,
double dRange[2]);
```

3.5.32 设置每次回调返回多少帧数据的数量

入参: hDevice 设备句柄

入参: nNum 一起返回的激光线数量（数值范围 0-10000，数组不宜过大，否则会造成内存分配失败）

返回: 成功时返回 0，失败返回错误码

```
int VzNL_SetOutputLaserLinesEachTime(VZNLHANDLE hDevice, unsigned int nNum);
int VzNL_GetOutputLaserLinesEachTime(VZNLHANDLE hDevice, unsigned int* nNum);
```

3.5.33 使能/禁止点云数据缓存功能

入参: hDevice 设备句柄

入参: bEnable 使能/禁用

返回: 成功时返回 0，失败返回错误码

```
int VzNL_EnablePointCloudCache(VZNLHANDLE hDevice, VzBool bEnable);
VzBool VzNL_IsEnablePointCloudCache(VZNLHANDLE hDevice, int* pnErrorCode);
```

3.5.34 查找输入 2D 点对应的 3D 点坐标(需使能点云数据缓存功能，并完成扫描)

入参: hDevice 设备句柄

入参: s2DPos 2d 点坐标

入参: p3DPoint 3d 点坐标

返回: 成功时返回 0，失败返回错误码

```
int VzNL_GetCached3DPointBy2DPos(VZNLHANDLE hDevice, SVzNL2DPoint s2DPos, SVzNL3DPoint* p3DPoint);
```

3.5.35 执行静态扫描点云拍照(阻塞执行)，输出完整扫描点云，如果为 RGBD 扫描则同步返回 RGB 图像

入参: hDevice 设备句柄

入参: eResultType 输出点云数据类型

入参: eFlipType 点云是否进行 Y 轴镜像

入参: bDoPointCloudMap是否将点云数据映射到 RGB 图像分辨率

入参: bEnableClip 是否按点云实际对应图像区域裁剪 RGB 图像，true：输出裁剪后的图像；false：输出原始图像；

出参: ppImage RGBD 扫描时输出 RGB 图像(参考 bEnableClip 标志)，非 RGBD 扫描时输出为 nullptr

出参: pPointCnt 输出点云数据点个数

出参: ppPointList 输出点云数据，数据类型为 eResultType

返回: 成功时返回 0，失败返回错误码

```
int VzNL_PointCloudSnapShort(VZNLHANDLE hDevice,
                             EVzResultDataType eResultType,
                             EVzFlipType eFlipType,
                             VzBool bDoPointCloudMap,
                             VzBool bEnableClip,
                             SVzNLImageData** ppImage,
                             unsigned int* pPointCnt,
                             void** ppPointList);
```

3.5.36 释放 VzNL_PointCloudSnapShort 输出的点云和图像数据

入参: ppImage 图像数据

入参: ppPointList2d 点云数据

返回: 成功时返回 0，失败返回错误码

```
int VzNL_ReleasePointCloudSnapShortResult(SVzNLImageData** ppImage, void** ppPointList);
```

3.5.37 执行焊缝跟踪点云扫描

入参: hDevice 设备句柄

入参: eResultType 结果类型

入参: eFlipType 激光线方向

入参: pCB 回调函数

入参: pCBParam 回调参数

返回: 成功时返回 0，失败返回错误码

```
int VzNL_StartAutoDetectForTracking(VZNLHANDLE hDevice, EVzResultDataType eResultType, EVzFlipType eFlipType, VzNL_AutoOutputLaserLineExCB pCB, void* pCBParam);
```

3.5.38 释放 VzNL_PointCloudSnapShort 输出的点云和图像数据

入参: hDevice 设备句柄

返回: 成功时返回 0，失败返回错误码

```
int VzNL_StopAutoDetectForTracking(VZNLHANDLE hDevice);
```

3.6 工具类接口

头文件：#include "VZNL_Utils.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.6.1 检测运行环境

出参: 环境信息

返回: 返回 VzTrue 说明一切正常，返回 VzFalse 表示有信息不匹配的地方

int VzNL_CheckPlatform(SVzCheckPlatformInfo* psPlatformInfo);

3.6.2 创建点云后处理工具 Handle

VZNLPOINTCLOUDHANDLE VzNL_CreateCloudPointAPTool();

当配置自适应参数时,绑定设备 Handle

int VzNL_AttachHandleForCloudPointAP(VZNLPOINTCLOUDHANDLE hCloudPointTool, VZNLHANDLE hDevice);

当配置自适应参数时,绑定设备 Handle

int VzNL_DetachHandleForCloudPointAP(VZNLPOINTCLOUDHANDLE hCloudPointTool);

3.6.3 设置 3D 点云基准激光线

(当生成灰度/深度图时需要配置), 配置后内部的图像参数会进行改变,若查看参数信息请调用 VzNL_GetCloudPointAPParam

入参: hCloudPointTool 点云工具句柄

入参: p3DPosition 3D 数据

入参: nPointCount 3D 点的个数

int VzNL_SetCloudPointAPBaseLine(VZNLPOINTCLOUDHANDLE hCloudPointTool, SVzNL3DPosition* p3DPosition, unsigned int nPointCount);

3.6.4 设置深度图/灰度图配置参数(当生成灰度/深度图时需要配置)

入参: hCloudPointTool 点云工具句柄

入参: psDepthMapParam 深度图配置参数

int VzNL_SetCloudPointAPParam(VZNLPOINTCLOUDHANDLE hCloudPointTool, SVzDepthMapImgCtlPara* psDepthMapParam);

3.6.5 启用基准线深度值计算模式(当生成灰度/深度图时需要配置)

入参: hCloudPointTool 点云工具句柄

入参: bEnable VzTrue 启用 / VzFalse 禁用

注意: 基准线模式:bEnable = VzTrue 当深度计算需要用当前一条线作为高度基准时,配置为此模式,例如激光线照射 U 型皮带，会形成高低不等的一条激光线，这个时候 U 型皮带上的各个点均为自身位置的 0 点位置

int VzNL_EnableCloudPointAPBaseLineMode(VZNLPOINTCLOUDHANDLE hCloudPointTool, VzBool bEnable);

VzBool VzNL_IsEnableCloudPointAPBaseLineMode(VZNLPOINTCLOUDHANDLE hCloudPointTool);

3.6.6 设置输出图像的最小宽度(当生成灰度 / 深度图时需要配置)

入参: hCloudPointTool 点云工具句柄

入参: nMinImageWidth 输出图像的最小宽度

int VzNL_SetCloudPointAPImageWidth(VZNLPOINTCLOUDHANDLE hCloudPointTool, unsigned int nMinImageWidth);

3.6.7 获取点云配置

入参: hCloudPointTool 点云工具句柄

出参: psDepthMapParam [out] 点云配置

int VzNL_GetCloudPointAPParam(VZNLPOINTCLOUDHANDLE hCloudPointTool, SVzDepthMapImgCtlPara* psDepthMapParam);

3.6.8 设置输出图像的最小宽度(当生成灰度 / 深度图时需要配置)

入参: hCloudPointTool 点云工具句柄

出参: pnImageHeight 输出图像高度

int VzNL_GetCloudPointAPImageHeight(VZNLPOINTCLOUDHANDLE hCloudPointTool, int* pnImageHeight);

3.6.9 开始点云处理(目前仅支持 keResultDataType_Position)

入参: hCloudPointTool 点云工具句柄

入参: eAfterProcType 图像后处理类型

* kePointAfterProcType_DepthMap 深度图

* kePointAfterProcType_GrayPic(灰度图)

入参: eDataType 激光线数据 SVzLaserLineData 的数据类型

入参: eDirect 激光线扫描方向

入参: pDepthMap 深度图的回调

入参: pParam 回调函数的参数

返回:返回 0 则为正确，返回非 0 为错误，可通过 VzNL_GetErrorInfo 获取错误信息

int VzNL_BeginCloudPointAfterProc(VZNLPOINTCLOUDHANDLE hCloudPointTool, EVzPointAfterProcType eAfterProcType, EVzResultDataType eDataType, EVzObjRunDirect eDirect, VzNL_OnOutputCloudPointAPData pDepthMap, void* pParam);

3.6.10 点云处理(目前仅支持 keResultDataType_Position)

入参: hCloudPointTool 点云工具句柄

入参: psLineData 激光线数据

int VzNL_CloudPointAfterProc(VZNLPOINTCLOUDHANDLE hCloudPointTool, SVzLaserLineData* psLineData);

3.6.11 结束点云处理

入参: hCloudPointTool 点云工具句柄

int VzNL_EndCloudPointAfterProc(VZNLPOINTCLOUDHANDLE hCloudPointTool);

3.6.12 销毁点云处理工具

入参: hCloudPointTool 点云工具句柄

int VzNL_DestroyCloudPointAPTool(VZNLPOINTCLOUDHANDLE hCloudPointTool);

3.6.13 创建图像生成工具

入参: hDevice 设备句柄

VZNLIMAGEGENERATOR VzNL_CreateImageGenerator(VZNLHANDLE hDevice);

3.6.14 开始送入点云数据

入参:hImageGenerator 图像生成工具

入参:eDataType 送入数据的类型,目前仅支持 keResultDataType_PointXYZRGBA 与 keResultDataType_PointGray

* keResultDataType_PointGray 输出图像为 8 bit 灰度图

* keResultDataType_PointGray 输出图像为 24 bit 彩色图

int VzNL_BeginPushPointToImageGenerator(VZNLIMAGEGENERATOR hImageGenerator, EVzResultDataType eDataType);

3.6.15 送入点云数据进行处理

入参: hImageGenerator 图像生成工具

入参: psLineData 激光线数据

int VzNL_PushPointToImageGenerator(VZNLIMAGEGENERATOR hImageGenerator, SVzLaserLineData* psLineData);

3.6.16 结束点云传送

入参: hImageGenerator 图像生成工具

int VzNL_EndPushPointToImageGenerator(VZNLIMAGEGENERATOR hImageGenerator);

3.6.17 通过图像生成工具，生成灰度图像

入参: hImageGenerator 图像生成工具

入参: psLineData 图像数据

int VzNL_GetImageFromImageGenerator(VZNLIMAGEGENERATOR hImageGenerator, SVzNLImageData** ppImageData);

3.6.18 通过图像生成工具，生成 Tif 图像

入参: hImageGenerator 图像生成工具

入参: lpszFile 文件名 tif

int VzNL_SaveDepthMapImageFromImageGenerator(VZNLIMAGEGENERATOR hImageGenerator, const char* lpszFile);

3.6.19 通过 2D 位置找 3D 点

入参: hImageGenerator 图像生成工具

入参: psLineData 3D 数据

int VzNL_Find3DPointFrom2DPosForImageGenerator(VZNLIMAGEGENERATOR hImageGenerator, int nX, int nY, SVzNLPointXYZ* p3DPoint);

3.6.20 销毁图像创建工具

入参: hImageGenerator 图像生成工具

int VzNL_DestroyImageGenerator(VZNLIMAGEGENERATOR hImageGenerator);

3.6.21 使用固定点云数据生成深度图

入参: eDataType 数据类型

入参: p3DPoint 3D 点云数组

入参: nPointCount 点的个数

入参: sDepthMapParam 深度图参数

入参: ppImageData 图像

返回: 返回 0 表示正常

int VzNL_CreateDepthMapImage(EVzResultDataType eDataType, void* p3DPoint, unsigned int nPointCount, SVzDepthMapImgCtlPara* sDepthMapParam, SVzNLImageData** ppImageData);

3.6.22 使用固定点云生成灰度图

入参: eDataType 数据类型

入参: p3DPoint 3D 点云数组

入参: nPointCount 点的个数

入参: fXYScale XY 比例

入参: ppImageData 图像

返回: 返回 0 表示正常

int VzNL_CreateGrayImage(EVzResultDataType eDataType, void* p3DPoint, unsigned int nPointCount, float fXYScale, SVzNLImageData** ppImageData);

3.6.23 读取图像

入参: szFileName [in] 要读取的文件全路径

入参: ppImageData [out] 无压缩图像实例

返回: 返回 0 表明正确

int VzNL_LoadImageFromFile(const char* szFileName, SVzNLImageData** ppImageData);

3.6.24 从 tif 文件中加载 3D 点云

入参: lpszFile 文件全路径

入参: nImageWidth 图像宽

入参: nImageHeight 图像高

入参: eDataType 数据类型

入参: p3DPoint 3D 点指针

返回: 返回 0 表示正常

int VzNL_LoadDepthMapTiffImage(const char* lpszFile, unsigned int* nImageWidth, unsigned int* nImageHeight, SVzNL3DPointF* p3DPoint, unsigned int* nPointCount);

3.6.25 将 3D 点云数据存储为 Tif 深度图

入参: lpszFile 文件全路径
入参: nImageWidth 图像宽
入参: nImageHeight 图像高
入参: eDataType 数据类型
入参: p3DPoint 3D 点指针
返回: 返回 0 表示正常

int VzNL_SaveDepthMapTiffImage(const char* lpszFile, unsigned int nImageWidth, unsigned int nImageHeight, EVzResultDataType eDataType, void* p3DPoint);

3.6.26 将输入的点云数据转换为指定分辨率的 2D 伪彩图像输出

入参: eDataType 输入点数据类型，内部根据数据类型解析数据
入参: nPointCnt 输入 pPointList 中点数据个数
入参: pPointList 输入点数据列表
入参: nImageWidth 输入希望输出的图像宽度
入参: nImageHeight 输入希望输出的图像高度
入参: nColorGradient 输出点数据颜色梯度
入参: nEdgeMargin 输入图像边缘的留白宽度(像素值，四边留白)
入参: pImage 输出的图像数据
返回: 返回 0 表示正常

int VzNL_PointCloudTo2DFixedSizeImage(EVzResultDataType eDataType,unsigned int nPointCnt,void* pPointList,unsigned int nImageWidth,unsigned int nImageHeight,unsigned int nColorGradient,unsigned int nEdgeMargin,SVzNLImageData* pImage);

3.6.27 将输入的点云数据转换为指定像素精度的 2D 伪彩图像输出

入参: eDataType 输入点数据类型，内部根据数据类型解析数据
入参: nPointCnt 输入 pPointList 中点数据个数
入参: pPointList 输入点数据列表
入参: fWHPixelResolution 输入图像像素精度，代表每个像素对应的实际物理距离，例如：0.1 表示像素间隔为 0.1 毫米
入参: nColorGradient 输出点数据颜色梯度
入参: nEdgeMargin 输入图像边缘的留白宽度(像素值，四边留白)
入参: pImage 输出的图像数据
返回: 返回 0 表示正常

int VzNL_PointCloudTo2DFixedResolutionImage(EVzResultDataType eDataType,unsigned int nPointCnt, void* pPointList, float fWHPixelResolution, unsigned int nColorGradient,
unsigned int nEdgeMargin,SVzNLImageData* pImage);

3.6.28 释放输出的图像数据

入参: pImage 图像数据
返回: 返回 0 表示正常

int VzNL_PointCloudTo2DImageRelease(SVzNLImageData* pImage);

3.6.29 创建 RGB 点云处理工具

入参: hDevice 设备句柄

返回: 返回 RGB 点云处理工具句柄

VZNLRGBCLOUDPOINTTOOL VzNL_CreateRGBCloudPointTool(VZNLHANDLE hDevice);

3.6.30 开始送入点云数据到 RGB 点云处理工具

入参: hRGBCloudPointTool RGB 点云处理工具句柄

入参: eDataType 送入数据的类型,目前仅支持 keResultDataType_PointXYZRGBA 、keResultDataType_PointRGBA_D

返回: 返回 0 表示正常

int VzNL_BeginPushPointToRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool, EVzResultDataType eDataType);

3.6.31 送入点云数据到 RGB 点云处理工具

入参: hRGBCloudPointTool RGB 点云处理工具句柄

入参: psLineData 激光线数据

返回: 返回 0 表示正常

int VzNL_PushPointToRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool, SVzLaserLineData* psLineData);

3.6.32 通过 2D 位置找 3D 点

入参: hRGBCloudPointTool RGB 点云处理工具句柄

入参: s2DPoint 2D 位置

出参: p3DPoint 3D 点

返回: 返回 0 表示正常

int VzNL_Find3DPointFrom2DPosForRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool, const SVzNL2DPoint s2DPoint, SVzNL3DPoint* p3DPoint);

3.6.33 通过 3D 点找 2D 位置

入参: hRGBCloudPointTool RGB 点云处理工具句柄

入参: s3DPoint 3D 点

出参: p2DPoint 2D 位置

返回: 返回 0 表示正常

int VzNL_Find2DPosFrom3DPointForRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool, const SVzNL3DPoint s3DPoint, SVzNL2DPoint* p2DPoint);

3.6.34 结束 RGB 点云处理

入参: hRGBCloudPointTool RGB 点云处理工具句柄

返回: 返回 0 表示正常

int VzNL_EndPushPointToRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool);

3.6.35 销毁 RGB 点云工处理工具

入参: hRGBCloudPointTool RGB 点云处理工具句柄

返回: 返回 0 表示正常

int VzNL_DestroyRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool);

3.6.36 通过双目校正图 2D 坐标计算 3D 点坐标，根据当前设备内部参数计算

入参: hDevice 设备句柄

入参: nPointCnt 输入点个数

入参: p2DPointList 输入双目校正图 2D 点坐标列表

入参: p3DPointList 输出计算得到的 3D 点列表

返回: 返回 0 表示正常

int VzNL_Cal3DPointsFromStereo2DPoints(VZNLHANDLE hDevice, unsigned int nPointCnt,
SVzNL2DLRPoint* p2DPointList, SVzNL3DPoint* p3DPointList);

3.6.37 通过双目校正图 2D 浮点坐标计算 3D 点坐标，根据当前设备内部参数计算

入参: hDevice 设备句柄

入参: nPointCnt 输入点个数

入参: p2DPointList 输入双目校正图 2D 点坐标列表

入参: p3DPointList 输出计算得到的 3D 点列表

返回: 返回 0 表示正常

int VzNL_Cal3DPointsFromStereo2DFPoints(VZNLHANDLE hDevice, unsigned int nPointCnt,
SVzNL2DLRPointF* p2DPointList, SVzNL3DPoint* p3DPointList);

3.6.38 输入 2D RGB 图像数据 + 扫描点云数据(静态 RGBD 扫描)，获取映射后的 2D RGB 图像数据和同分辨率的点云序列

入参: hDevice 设备句柄

入参: pRGBImage 输入与点云匹配的 RGB 图像

入参: eDataType 输入点云数据类型，同样也为输出的点云数据类型

入参: nLineCnt 输入激光线数据条数

入参: pLineDataList 输入激光线数据列表

入参: bEnableClip 输入是否进行有效图像裁剪，true：输出点云实际对应的有效图像区域及映射点云；false：输出原始图像及映射点云；

入参: ppOutImage 输出处理后的图像(参考 bEnableClip 标志决定是否裁剪图像)

入参: pPointCnt 输出点云数据中的点个数

入参: ppMappedPointList 输出映射后与 ppOutImage 相同分辨率的点云数据，点数据类型为 eDataType

返回: 返回 0 表示正常

int VzNL_GetMappedRGBImageAndPointCloud(VZNLHANDLE hDevice,
SVzNLImageData* pRGBImage,
EVzResultDataType eDataType,
unsigned int nLineCnt,
SVzLaserLineData* pLineDataList,

bool bEnableClip,
SVzNLImageData** ppOutImage,
unsigned int* pPointCnt,
void** ppMappedPointList);

3.6.39 输入左目或右目图像数据 + 扫描点云数据(静态非 RGBD 扫描)，获取映射后的 2D 左目或右目图像数据和同分辨率的点云序列

入参: hDevice 设备句柄
入参: bIsRightImage 输入当前图像是否为右目图像
入参: pImage 输入与点云匹配的左目或右目图像数据
入参: eDataType 输入点云数据类型，同样也为输出的点云数据类型
入参: nLineCnt 输入激光线数据条数
入参: pLineDataList 输入激光线数据列表
入参: bEnableClip 输入是否进行有效图像裁剪，true：输出点云实际对应的有效图像区域及映射点云；false：输出原始图像及映射点云;
入参: ppOutImage 输出处理后的图像(参考 bEnableClip 标志决定是否裁剪图像)
入参: pPointCnt 输出点云数据中的点个数
入参: ppMappedPointList 输出映射后与 ppOutImage 相同分辨率的点云数据，点数据类型为 eDataType
返回: 返回 0 表示正常

int VzNL_GetMappedLRImageAndPointCloud(VZNLHANDLE hDevice,
bool bIsRightImage,
SVzNLImageData* pImage,
EVzResultDataType eDataType,
unsigned int nLineCnt,
SVzLaserLineData* pLineDataList,
bool bEnableClip,
SVzNLImageData** ppOutImage,
unsigned int* pPointCnt,
void** ppMappedPointList);

3.6.40 释放 VzNL_GetMappedRGBImageAndPointCloud()/VzNL_GetMappedLRImageAndPointCloud() 函数返回的图像和 3D 点数据

入参: ppOutImage 图像数据
入参: ppPointList 点云数据
返回: 返回 0 表示正常

int VzNL_ReleaseImageAndPointCloudMappedResult(SVzNLImageData** ppImage, void** ppPointList);

3.6.41 USB 串口继电器设备搜索

入参: pDeviceNum 返回搜索到的设备个数
入参: ppDeviceInfoList 返回搜索到的设备名称列表
返回: 返回 0 表示正常

int VzNL_SearchUSBSerialRelay(unsigned int* pDeviceNum, SVzSerialPortInfo** ppDeviceInfoList);

3.6.42 获取指定 USB 串口继电器设备的配置信息

入参：pDeviceName 输入设备名称(搜索函数返回)
入参：pCfgPara 返回设备配置信息
返回： 返回 0 表示正常
int VzNL_GetUSBSerialRelayCfgPara(char* pDeviceName, SVzUSBSerialRelayCfgPara* pCfgPara);

3.6.43 查看指定 USB 串口继电器设备是否打开

入参：pDeviceName 输入设备名称(搜索函数返回)
返回： 返回 true 表示已经打开;false 为未打开
bool VzNL_IsUSBSerialRelayOpen(char* pDeviceName);

3.6.44 打开指定端口的 USB 串口继电器设备，按配置参数打开指定设备

入参：pDeviceName 输入设备名称(搜索函数返回)
入参：pCfgPara 返回设备配置信息
返回： 返回 0 表示正常
int VzNL_OpenUSBSerialRelay(char* pDeviceName, SVzUSBSerialRelayCfgPara sCfgPara);

3.6.45 关闭指定端口的 USB 串口继电器设备

入参：pDeviceName 输入设备名称(搜索函数返回)
返回： 返回 0 表示正常
int VzNL_CloseUSBSerialRelay(char* pDeviceName);

3.6.46 获取所有打开的 USB 串口继电器设备端口列表

入参：pDeviceName 输入设备名称(搜索函数返回)
入参：ppDeviceInfoList 返回已打开设备的名称列表
返回： 返回 0 表示正常
int VzNL_GetOpenedUSBSerialRelayList(unsigned int* pDeviceNum, SVzSerialPortInfo** ppDeviceInfoList);

3.6.47 获取指定 USB 继电器通道当前开关状态

入参：pDeviceName 输入设备名称(搜索函数返回)
入参：nChannelIdx 输入继电器通道索引
入参：pState 返回该通道当前开关状态
返回： 返回 0 表示正常
int VzNL_GetUSBSerialRelayChannelOnOffState(char* pDeviceName, unsigned int nChannelIdx, bool* pState);

3.6.48 打开/关闭指定继电器通道，pState 返回操作后该通道状态

入参：pDeviceName 输入设备名称(搜索函数返回)
入参：bOn 输入需要设置的开关状态
入参：nChannelIdx 输入继电器通道索引

入参：pState　　不为 null 时返回该通道当前开关状态

返回：　返回 0 表示正常

int VzNL_SetUSBSerialRelayChannelOnOff(char* pDeviceName, bool bOn, unsigned int nChannelIdx, bool* pState);

3.6.49　获取所有继电器通道开关状态：unsigned int* pState：低位开始，每位代表一个通道；

入参：pDeviceName　　输入设备名称(搜索函数返回)

入参：pState　　返回该通道当前开关状态

返回：　返回 0 表示正常

int VzNL_GetUSBSerialRelayAllChannelOnOffState(char* pDeviceName, unsigned int* pState);

3.6.50　打开/关闭所有继电器通道，pState 返回操作后所有继电器通道状态(低位开始，每位代表一个通道)

入参：pDeviceName　　输入设备名称(搜索函数返回)

入参：bOn　　输入需要设置的开关状态

入参：pState　　不为 null 时返回所有通道当前开关状态

返回：　返回 0 表示正常

int VzNL_SetUSBSerialRelayAllChannelOnOff(char* pDeviceName, bool bOn, unsigned int* pState);

```
#include <stdio.h>
#include <vector>
#include <list>
#include "VZNL_Common.h"
#include "VZNL_EyeConfig.h"
#include "VZNL_DetectLaser.h"
#include "VZNL_Graphics.h"
#include "VZNL_DetectConfig.h"
#include "VZNL_Utils.h"
#include "VZNL_SwingMotor.h"
#include "windows.h"
#include <fstream>

/*******************************************************************************
 * File: vizumsdk_save_gray_depth_map_sample.cpp
 * Detail: 静态相机 保存灰度图像 和 深度图的 Sample
 * Author: Vizum
 * Date: 20210609
 *******************************************************************************/

static SVzVideoResolution g_sVideoRes;
static SVzNLPointXYZRGBA* g_p2DToPointMap = nullptr;
static bool* g_pb2DInvalidPt = nullptr;
```

```
static HANDLE hPauseEvent = NULL;
static SVzNLROIRect g_sImageLeftROI = { 0, 1536, 0, 2048 };
static SVzNLROIRect g_sImageRightROI = { 0, 1536, 0, 2048 };

/// @name  自动检测回调函数
/// @{
void _AutoOutputLaserLineExCB(EVzResultDataType eDataType, SVzLaserLineData* pLaserLinePoint, void*
pParam)
{
	//  当需要的点云格式为 SVzNLPointXYZRGBA 时
	if (keResultDataType_PointXYZRGBA == eDataType)
	{
		SVzNLPointXYZRGBA* p3DPoint = (SVzNLPointXYZRGBA*)pLaserLinePoint->p3DPoint;
		SVzNL2DLRPoint* p2DPoint = (SVzNL2DLRPoint*)pLaserLinePoint->p2DPoint;
		for (int nPtIdx = 0; nPtIdx < pLaserLinePoint->nPointCount; nPtIdx++)
		{
			if ((p2DPoint->sLeft.x < g_sImageLeftROI.left) || (p2DPoint->sLeft.x >=
g_sImageLeftROI.right))
			{
				p3DPoint++;
				p2DPoint++;
				continue;
			}

			if ((p2DPoint->sLeft.y < g_sImageLeftROI.top) || (p2DPoint->sLeft.y >=
g_sImageLeftROI.bottom))
			{
				p3DPoint++;
				p2DPoint++;
				continue;
			}

			int nPos = (p2DPoint->sLeft.x - g_sImageLeftROI.left) + (p2DPoint->sRight.y -
g_sImageLeftROI.top) * (g_sImageLeftROI.right - g_sImageLeftROI.left);

			g_p2DToPointMap[nPos] = *p3DPoint;
			g_pb2DInvalidPt[nPos] = true;
			p3DPoint++;
			p2DPoint++;
		}
	}
}

/// @brief
```

```
/// 获取文件纯路径
int _GetPurePath(const char* szFilePath, char szPurePath[VZ_MAXPATH])
{
	int nFindPos = -1;
	const char* pPos = szFilePath;
	while ('\0' != *pPos)
	{
		if ('/' == *pPos || '\\' == *pPos)
		{
			nFindPos = static_cast<int>(pPos - szFilePath);
		}
		pPos++;
	}

	if (-1 == nFindPos)
		return -1;

	memcpy(szPurePath, szFilePath, static_cast<unsigned int>(nFindPos));
	szPurePath[nFindPos] = '\0';
	return 0;
}

// 显示错误信息
void _DisplayErrorInfo(const char* pszDesc, int nErrorCode)
{
	char szOutput[256];
	if (0 == nErrorCode)
	{
		sprintf_s(szOutput, 256, "%s\r\n", pszDesc);
	}
	else
	{
		char szErrorInfo[256] = { 0 };
		VzNL_GetErrorInfo(nErrorCode, szErrorInfo);

		sprintf_s(szOutput, 256, "%s [%d] %s\r\n", pszDesc, nErrorCode, szErrorInfo);
	}
	printf(szOutput);
}

// 搜索设备
void _ResearchDevice(std::vector<SVzNLEyeCBInfo>& vetDevice)
{
	// 搜索相机
```

```
	VzNL_ResearchDevice(keSearchDeviceFlag_EthLaserRobotEye);

	// 遍历相机
	int nDevCount = 0;
	VzNL_GetEyeCBDeviceInfo(nullptr, &nDevCount);
	if (nDevCount <= 0)
	{
		return;
	}

	vetDevice.resize(nDevCount);
	VzNL_GetEyeCBDeviceInfo(vetDevice.data(), &nDevCount);
}

// @brief 搜索且进行绑定设备
void _SearchAndBindDevice(std::vector<SVzNLEyeCBInfo>& vetDevice)
{
	int nErrorCode = 0;
	VzBool bCanResearch = VzFalse;
	do
	{
		bCanResearch = VzFalse;
		_ResearchDevice(vetDevice);
		std::vector<SVzNLEyeCBInfo>::iterator itDevice = vetDevice.begin();
		while (itDevice != vetDevice.end())
		{
			// 相机没有被识别
			if (VzFalse == itDevice->bValidDevice)
			{
				SVzNLEyeCBInfo sCurDevInfo = *itDevice;
				SVzNLEthernetEyeConfigInfo sOutputEthConfigInfo;
				nErrorCode = VzNL_GetEthernetEyeConfigInfo(&sCurDevInfo, &sOutputEthConfigInfo);
				if (0 != nErrorCode)
				{
					_DisplayErrorInfo("获取网络信息失败", nErrorCode);
				}

				//
				if (0 == memcmp(sOutputEthConfigInfo.byLocalIP,
sOutputEthConfigInfo.sNetCardInfo.byLocalIP, 4))
				{
					_DisplayErrorInfo("请查看是否有网络权限和管理员权限\r\n", 0);
				}
				else
```

```
                {
                    nErrorCode = VzNL_BindEthernetEye(&sCurDevInfo);
                    if (0 != nErrorCode)
                    {
                        _DisplayErrorInfo("绑定相机失败", nErrorCode);
                    }
                    bCanResearch = VzTrue;
                }
            }
            itDevice++;
        }
    } while (bCanResearch);
}

/**
 * @brief 设备状态改变 CallBack 函数
 * @param eStatus [out] 状态
 * @param pInfoParam [out] 状态参数
 */
void _OnNotifyStatusCB(EVzDeviceWorkStatus eStatus, void* pInfoParam)
{
    if (keDeviceWorkStatus_Device_Auto_Stop == eStatus)
    {
        printf("收到停止信号\r\n");
        if (nullptr != hPauseEvent)
        {
            SetEvent(hPauseEvent);
        }
    }
}

int main()
{
    hPauseEvent = CreateEvent(NULL, TRUE, FALSE, NULL);

    // 初始化 SDK
    SVzNLConfigParam sConfigParam;
    sConfigParam.nDeviceTimeOut = 0;
    int nRet = VzNL_Init(&sConfigParam);
    if (0 != nRet)
    {
        _DisplayErrorInfo("查看是否有其他程序在使用 SDK?", nRet);
    }
```

```
	VzNL_SetLogLevel(keNLLogLevel_Information, keNLLogType_File);

	// 枚举相机
	std::vector<SVzNLEyeCBInfo> vetDevice;
	_SearchAndBindDevice(vetDevice);

	//
	int nErrorCode = 0;
	std::vector<VZNLHANDLE>		vetHandles;
	if (vetDevice.size() > 0)
	{
		_DisplayErrorInfo("显示当前可用相机\r\n", 0);
		std::vector<SVzNLEyeCBInfo>::iterator itDevice = vetDevice.begin();
		while (itDevice != vetDevice.end())
		{
			// 相机没有被识别
			if (VzFalse == itDevice->bValidDevice)
			{
				itDevice++;
			}

			char szOutput[256];
			sprintf_s(szOutput, 256, "----------------------准备打开  %s----------------------\r\n",
itDevice->byServerIP);
			_DisplayErrorInfo(szOutput, 0);

			// 打开相机
			SVzNLOpenDeviceParam sOpenDevParam;
			VZNLHANDLE hDevice = VzNL_OpenDevice(&(*itDevice), &sOpenDevParam,
&nErrorCode);
			if (0 != nErrorCode)
			{
				_DisplayErrorInfo("打开设备失败", nErrorCode);
				break;
			}
			else
			{
				char szOutput[256];
				sprintf_s(szOutput, 256, "打开相机%s 成功\r\n", itDevice->byServerIP);
				_DisplayErrorInfo(szOutput, 0);
			}

			if (NULL != hDevice)
			{
```

```
            vetHandles.push_back(hDevice);
        }

        VzNL_SetOutputImageFormat(keVzNLImageType_BGR888);

        VzNL_SetDeviceStatusNotify(hDevice, _OnNotifyStatusCB, NULL);

        // 创建激光线检测工具
        nErrorCode = VzNL_BeginDetectLaser(hDevice);
        if (0 != nErrorCode)
        {
            _DisplayErrorInfo("创建检测激光线工具", nErrorCode);
            break;
        }
        else
        {
            _DisplayErrorInfo("创建激光线检测工具成功\r\n", 0);
        }

        VzNL_EnableCalibROI(hDevice, VzFalse);

        unsigned int nOldExpose = 0;
        EVzNLExposeMode eExposeMode = keVzNLExposeMode_Fix;
        VzNL_GetConfigEyeExpose(hDevice, &eExposeMode, &nOldExpose);

        SVzNLROIRect sOldLeftROI, sOldRightROI;
        VzNL_GetConfigDetectROI(hDevice, &sOldLeftROI, &sOldRightROI);

        VzNL_EnableCalibROI(hDevice, VzTrue);

        SVzNLEyeDeviceInfoEx sDeviceInfoEx;
        sDeviceInfoEx.sEyeCBInfo.nSize = sizeof(SVzNLEyeDeviceInfoEx);
        VzNL_GetDeviceInfo(hDevice, &sDeviceInfoEx.sEyeCBInfo);
        g_sVideoRes = sDeviceInfoEx.sVideoRes;

        g_sImageLeftROI.right = g_sVideoRes.nFrameWidth;
        g_sImageLeftROI.bottom = g_sVideoRes.nFrameHeight;
        g_sImageRightROI.right = g_sVideoRes.nFrameWidth;
        g_sImageRightROI.bottom = g_sVideoRes.nFrameHeight;

        int nFrameSize = (g_sImageRightROI.right - g_sImageRightROI.left) *
(g_sImageRightROI.bottom - g_sImageRightROI.top);
        g_p2DToPointMap = new SVzNLPointXYZRGBA[nFrameSize];
        g_pb2DInvalidPt = new bool[nFrameSize];
```

```
memset(g_pb2DInvalidPt, 0, sizeof(bool) * nFrameSize);

// 配置曝光
nErrorCode = VzNL_ConfigEyeExpose(hDevice, keVzNLExposeMode_Fix, 10000);
if (0 != nErrorCode)
{
    _DisplayErrorInfo("曝光配置失败", nErrorCode);
}

// 配置 ROi
nErrorCode = VzNL_ConfigDetectROI(hDevice, &g_sImageLeftROI, &g_sImageRightROI);
if (0 != nErrorCode)
{
    _DisplayErrorInfo("ROI 检测配置失败", nErrorCode);
}

SVzNLImageData* pLeftImage = nullptr; SVzNLImageData* pRightImage = nullptr;
nErrorCode = VzNL_GetEyeImage(hDevice, &pLeftImage, &pRightImage, 5000);
if (0 != nErrorCode)
{
    _DisplayErrorInfo("获取图像失败", nErrorCode);
}

VzNL_EnableCalibROI(hDevice, VzFalse);

nErrorCode = VzNL_ConfigEyeExpose(hDevice, keVzNLExposeMode_Fix, nOldExpose);
if (0 != nErrorCode)
{
    _DisplayErrorInfo("曝光配置失败", nErrorCode);
}

// 配置 ROi
nErrorCode = VzNL_ConfigDetectROI(hDevice, &sOldLeftROI, &sOldRightROI);
if (0 != nErrorCode)
{
    _DisplayErrorInfo("ROI 检测配置失败", nErrorCode);
}

// 设置到主模式
nErrorCode = VzNL_SetTriggerMode(hDevice, keEyeTriggerMode_Master);
if (0 != nErrorCode)
{
    _DisplayErrorInfo("设置主模式失败", nErrorCode);
    break;
```

```
        }
        else
        {
            _DisplayErrorInfo("主模式设置成功\r\n", 0);
        }

        unsigned int nMinFrameRate, nMaxFrameRate;
        VzNL_QueryParamRange(hDevice, keDeviceParamType_FrameRate, &nMinFrameRate,
&nMaxFrameRate);

        VzNL_SetFrameRate(hDevice, nMaxFrameRate);

        ResetEvent(hPauseEvent);

        VzBool bIsSupportMotor = VzNL_IsSupportSwingMotor(hDevice, nullptr);
        if (bIsSupportMotor)
        {
            VzNL_EnableSwingMotor(hDevice, VzTrue);
        }

        // 开始流模式检测
        nErrorCode = VzNL_StartAutoDetectEx(hDevice, keResultDataType_PointXYZRGBA,
keFlipType_None, _AutoOutputLaserLineExCB, nullptr);
        if (0 != nErrorCode)
        {
            _DisplayErrorInfo("开流失败", nErrorCode);
            break;
        }
        else
        {
            _DisplayErrorInfo("开流成功\r\n", 0);
        }

        // 等待摆动停止
        WaitForSingleObject(hPauseEvent, INFINITE);

        nErrorCode = VzNL_StopAutoDetect(hDevice);
        if (0 != nErrorCode)
        {
            _DisplayErrorInfo("关流失败", nErrorCode);
            break;
        }
        else
        {
```

```
                    _DisplayErrorInfo("关流成功\r\n", 0);
                }
                VzNL_EndDetectLaser(hDevice);
                VzNL_CloseDevice(hDevice);
                // 存储灰度图
                char szModule[VZ_MAXPATH] = { 0 }; char szPurePath[VZ_MAXPATH] = { 0 };
                GetModuleFileNameA(nullptr, szModule, VZ_MAXPATH);
                _GetPurePath(szModule, szPurePath);
                if (nullptr != pLeftImage)
                {
                    char szGrayFile[VZ_MAXPATH] = { 0 };
                    sprintf_s(szGrayFile, VZ_MAXPATH, "%s\\Gray.png", szPurePath);
                    VzNL_SaveImage(szGrayFile, pLeftImage);
                }

                // 存储深度图
                if (nullptr != g_p2DToPointMap)
                {
                    char szTifFile[VZ_MAXPATH] = { 0 };
                    sprintf_s(szTifFile, VZ_MAXPATH, "%s\\Depth.tif", szPurePath);
                    VzNL_SaveDepthMapTiffImage(szTifFile, g_sVideoRes.nFrameWidth,
g_sVideoRes.nFrameHeight, keResultDataType_PointXYZRGBA, g_p2DToPointMap);
                }

                // 输出点云映射数据
                if (nullptr != g_p2DToPointMap)
                {
                    delete[] g_p2DToPointMap;
                    g_p2DToPointMap = nullptr;
                }
                if (nullptr != g_pb2DInvalidPt)
                {
                    delete[] g_pb2DInvalidPt;
                    g_pb2DInvalidPt = nullptr;
                }
                VzNL_ReleaseImage(&pLeftImage);
                VzNL_ReleaseImage(&pRightImage);

                itDevice++;
            }
        }
        // 销毁 SDK
        VzNL_Destroy();
        getchar();
```

```
    return 0;
}
```

3.7 VzNLSDK 图像操作接口

头文件：#include "VZNL_Graphics.h"

库文件：VzNL_Graphics.dll / lib VzNL_Graphics.so

3.7.1 开始绘图

VZNLHANDLE VzNL_BeginPaint(SVzNLImageData* pImageData);

入参：　pImageData　　图像结构体地址

返回：　绘图句柄，NULL 为创建失败

3.7.2 绘制直线

int VzNL_DrawLine(VZNLHANDLE hHandle, const SVzNL2DPoint sPoint[2]);

入参：　hDevice　　设备句柄

　　dwFrameColor　　边框颜色

　　dwFillColor　　填充色

　　nBorder　　边框宽度

返回：　0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.7.3 绘制矩形

int VzNL_DrawRect(VZNLHANDLE hHandle, const SVzNL2DPoint sRect[4], unsigned long dwFrameColor, unsigned long dwFillColor, unsigned int nBorder);

入参：　hDevice　　设备句柄

　　sRect　　矩形四个点的坐标

　　dwFrameColor　　边框颜色

　　dwFillColor　　填充色

　　nBorder　　边框宽度

返回：　0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.7.4 绘制多边形

int VzNL_DrawPolygon(VZNLHANDLE hHandle, const SVzNL2DPoint* psPoint, const unsigned int nPtCount, unsigned long dwFrameColor, unsigned long dwFillColor, unsigned int nBorder);

入参：　hDevice　　设备句柄

　　psPoint　　多变形顶点坐标数组

　　nPtCount　　顶点个数

　　dwFrameColor　　边框颜色

　　dwFillColor　　填充色

　　nBorder　　边框宽度

返回：　0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.7.5 绘制圆形

int VzNL_DrawCircle(VZNLHANDLE hHandle, const SVzNL2DPoint sPoint, int nRadius, unsigned long dwFrameColor, unsigned long dwFillColor, unsigned int nBorder);

入参： hDevice 设备句柄
sPoint 圆心坐标
nRadius 边框颜色
dwFrameColor 边框颜色
dwFillColor 填充色
nBorder 边框宽度

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.7.6 绘制椭圆

int VzNL_DrawEllipe(VZNLHANDLE hHandle, const SVzNL2DPoint sPoint, unsigned long dwFrameColor, unsigned long dwFillColor, unsigned int nLongAxes, unsigned int nShortAxes, double angle, unsigned int nBorder);

入参： hDevice 设备句柄
sPoint 圆心坐标
dwFrameColor 边框颜色
dwFillColor 填充色
nLongAxes 长轴长度
nShortAxes 短轴长度
angle 旋转角度
nBorder 边框宽度

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.7.7 结束绘图

void VzNL_EndPaint(VZNLHANDLE hHandle);

入参： hDevice 设备句柄

返回： 无

3.7.8 显示图像

void VzNL_ShowImage(const char* szWindowName, SVzNLImageData* pImageData);

入参： szWindowName 窗口名称
pImageData 图像数据

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

注：该接口在 linux 下无法显示图像。（linux 下显示库版本/种类多，无法做到通用）

3.7.9 存储图像

int VzNL_SaveImage(const char* szFileName, SVzNLImageData* pImageData);

入参： szFileName 存储文件的名称
pImageData 图像数据

返回： 0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。

3.7.10 压缩图片

int VzNL_CompressImage(EVzNLImageCompress eCompress, SVzNLImageData* pSrc, VzNLCompressData** pDst);

入参：	handle	绘图工具 handle
	eCompress	压缩类型
	pSrc	数据源（无压缩）
	pDst	压缩后的数据
返回：	0 为正确，其他错误码可通过 VzNL_GetErrorInfo 获取错误信息。	

3.7.11 释放压缩图片

void VzNL_ReleaseCompressImage(SVzNLCompressData** pDst);

入参：	pDst	所要释放图像的地址
返回：	无返回	

3.8 文件相关接口

头文件：#include "VZNL_FileUtils.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.8.1 创建文件工具

入参: eFileType 文件类型

* keLaserFileType_PureTxt	纯文本
* keLaserFileType_Txt	本公司自用
* keLaserFileType_Pcd	PCD
* keLaserFileType_Las	LAS
* keLaserFileType_VzBinaryData	Binary File

返回: 返回文件句柄

VZNLFILE VzNL_CreateLaserFile(EVzLaserFileType eFileType);

3.8.2 打开文件

入参: hFile 文件句柄

入参: lpszFile 文件名

入参: eFileMode 文件打开模式

入参: eDataType 数据类型

入参: dSpeed 速度 mm/s

返回: 返回值为 0 表示正确

int VzNL_OpenLaserFile(VZNLFILE hFile, const char* lpszFile, EVzFileMode eFileMode, EVzResultDataType eDataType, double dSpeed);

3.8.3 写入文件

入参: hFile 文件句柄

入参: psLaserLinePoint 激光线数据

返回:返回值为 0 表示正确

int VzNL_WriteLaserFile(VZNLFILE hFile, const SVzLaserLineData* psLaserLinePoint);

3.8.4 关闭文件

入参: hFile [in] 文件句柄

返回: 返回值为 0 表示正确

int VzNL_CloseLaserFile(VZNLFILE hFile);

```
static VZNLFILE s_hFile = nullptr;

// 数据回调函数
void _AutoOutputLaserLineExCB(EVzResultDataType eDataType, SVzLaserLineData*
pLaserLinePoint, void* pParam)
{
    // 当需要的点云格式为SVzNLPointXYZRGBA时
    if (keResultDataType_PointXYZRGBA == eDataType)
    {
        if (nullptr != s_hFile)
        {
            VzNL_WriteLaserFile(s_hFile, pLaserLinePoint);
        }
    }
}

// 创建文件句柄
void _SaveFile()
{
         // 创建激光线检测工具
        int nErrorCode = VzNL_BeginDetectLaser(hDevice);
        if (0 != nErrorCode)
        {
            return;
        }

         s_hFile = VzNL_CreateLaserFile(keLaserFileType_Pcd);
         if (NULL != s_hFile)
         {
               VzNL_OpenLaserFile(s_hFile, "D:/test.pcd", keFileModeWrite,
keResultDataType_PointXYZRGBA, 0.);
         }

       // 开始流模式检测
        nErrorCode = VzNL_StartAutoDetectEx(hDevice, keResultDataType_PointXYZRGBA,
keFlipType_None, _AutoOutputLaserLineExCB, nullptr);
        if (0 != nErrorCode)
        {
            break;
```

```
        }

        // 保存5s的数据
        Sleep(5000);

        nErrorCode = VzNL_StopAutoDetect(hDevice);
        if (0 != nErrorCode)
        {
            break;
        }

         if (NULL != s_hFile)
         {
               VzNL_CloseLaserFile(s_hFile);
         }

         // 销毁激光线检测工具
         VzNL_EndDetectLaser(hDevice);
}
```

3.9 RGB 相关配置接口

头文件：#include "VZNL_RGBConfig.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.9.1 当前相机是否支持 RGB

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue 表示支持 VzFalse 表示不支持

VzBool VzNL_IsSupportRGBCamera(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.2 获取 RGB 图像分辨率

入参: hDevice [in] 设备 Handle

返回: 关闭成功返回 0，否则为错误码

int VzNL_GetRGBResolution(VZNLHANDLE hDevice, SVzVideoResolution* psVideoRes);

3.9.3 启用/获取 RGB 功能

入参: hDevice 当前设备句柄

入参: bEnable VzTrue 启用 / VzFalse 禁用

返回: 正确返回 0， 失败返回其他值

int VzNL_EnableRGB(VZNLHANDLE hDevice, VzBool bEnable);

VzBool VzNL_IsEnableRGB(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.4 格式化彩色 ROI

入参: hDevice 当前设备句柄

入参: psROI[in/out] 想要格式化的 ROI 大小

返回: 正确返回 0， 失败返回其他值

int VzNL_FormatRGBROI(VZNLHANDLE hDevice, SVzNLROIRect* psROI);

3.9.5 配置 RGB ROI

入参: hDevice 当前设备句柄

入参: psROI 想要设置的 ROI 大小

返回: 正确返回 0， 失败返回其他值

int VzNL_SetRGBROI(VZNLHANDLE hDevice, const SVzNLROIRect* psROI);

VZNLAPI int VzNL_GetRGBROI(VZNLHANDLE hDevice, SVzNLROIRect* psROI);

3.9.6 启用/获取 自动白平衡

入参: hDevice 当前设备句柄

入参: bEnable 启用 VzTrue /禁用 VzFalse 白平衡

返回: 正确返回 0， 失败返回其他值

int VzNL_EnableRGBAWB(VZNLHANDLE hDevice, VzBool bEnable);

VzBool VzNL_IsEnableRGBAWB(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.7 设置/获取 自动曝光状态

入参: hDevice 当前设备句柄

入参: bEnable 启用 VzTrue / 禁用 VzFalse

返回: 正确返回 0， 失败返回其他值

int VzNL_EnableRGBAutoExpose(VZNLHANDLE hDevice, VzBool bEnable);

VzBool VzNL_IsEnableRGBAutoExpose(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.8 设置 R Value

入参: hDevice 当前设备句柄

入参: fValue R 分量值

返回: 正确返回 0， 失败返回其他值

int VzNL_SetRGBRValue(VZNLHANDLE hDevice, float fValue);

float VzNL_GetRGBRValue(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.9 设置 G Value

入参: hDevice 当前设备句柄

入参: fValue G 分量值

返回: 正确返回 0， 失败返回其他值

int VzNL_SetRGBGValue(VZNLHANDLE hDevice, float fValue);

float VzNL_GetRGBGValue(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.10 设置 B Value

入参: hDevice 当前设备句柄

入参: fValue B 分量值

返回： 正确返回 0， 失败返回其他值

int VzNL_SetRGBBValue(VZNLHANDLE hDevice, float fValue);

float VzNL_GetRGBBValue(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.11 设置彩色 Sensor 帧率

入参： hDevice 当前设备句柄

入参： nValue 帧率

返回： 正确返回 0， 失败返回其他值

int VzNL_SetRGBFrameRate(VZNLHANDLE hDevice, unsigned int nValue);

unsigned int VzNL_GetRGBFrameRate(VZNLHANDLE hDevice, int* pErrorCode);

3.9.12 设置彩色 Sensor 曝光

入参： hDevice 当前设备句柄

入参： nValue 曝光值[20~100000]

返回： 正确返回 0， 失败返回其他值

int VzNL_SetRGBExpose(VZNLHANDLE hDevice, unsigned int nValue);

unsigned int VzNL_GetRGBExpose(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.13 设置彩色 sensor 增益

入参： hDevice 当前设备句柄

入参： nValue 设置增益值

返回： 正确返回 0， 失败返回其他值

int VzNL_SetRGBGain(VZNLHANDLE hDevice, unsigned int nValue);

unsigned int VzNL_GetRGBGain(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.14 设置 RGBSensor 触发方式

入参： hDevice 当前设备句柄

入参： eTriggerMode 触发模式

返回： 正确返回 0， 失败返回其他值

int VzNL_SetRGBTriggerMode(VZNLHANDLE hDevice, EVzEyeTriggerMode eTriggerMode);

EVzEyeTriggerMode VzNL_GetRGBTriggerMode(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.15 设置彩色 Sensor 期望值

入参： hDevice 当前设备句柄

入参： fExposeThres 期望值[1~255]

返回： 正确返回 0， 失败返回其他值

int VzNL_SetRGBAutoExposeThres(VZNLHANDLE hDevice, float fExposeThres);

float VzNL_GetRGBAutoExposeThres(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.16 生成一个 RGB 信号

入参： hDevice 当前设备句柄

返回： 正确返回 0， 失败返回其他值

int VzNL_GenRGBSoftSignal(VZNLHANDLE hDevice);

3.9.17 设置 CCM

入参: hDevice 当前设备句柄

入参: dCCMVal CCM 值

返回: 正确返回 0， 失败返回其他值

int VzNL_SetRGBColorCortectMatrix(VZNLHANDLE hDevice, double dCCMVal[3][3]);

int VzNL_GetRGBColorCortectMatrix(VZNLHANDLE hDevice, double dCCMVal[3][3]);

3.9.18 启用是否输出 RGB 2D 数据

入参: hDevice 当前设备句柄

入参: bEnable [in] 启用为 VzTrue,关闭为 VzFalse

返回: 正确返回 0， 失败返回其他值

提示: 当配置为 VzTrue 时，开始检测输出的 2D 左目数据为 RGB 图像的位置。

int VzNL_EnableOutputColor2D(VZNLHANDLE hDevice, VzBool bEnable);

3.9.19 获取中间相机单帧 RGB 图像

入参: hDevice 当前设备句柄

出参： ppRGBImage 中间相机 RGB 图像的结构地址

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_GetRGBImage(VZNLHANDLE hDevice,SVzNLImageData** ppRGBImage);

3.9.20 通过 3d 点获取 RGB 图像的 2d 点坐标

入参: hDevice 当前设备句柄

出参： s3DPoint [in] 3d 点

出参： ps2DPoint [in/out] 2d 点

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_CalcColor2DFrom3D(VZNLHANDLE hDevice, const SVzNL3DPoint s3DPoint, SVzNL2DPoint* ps2DPoint);

3.9.21 是否支持 RGB HDR 功能

入参: hDevice 当前设备句柄

出参： pnErrorCode 错误码,为 nullptr 时不返回

返回： 返回 VzTrue 表示支持，否则为不支持。

VzBool VzNL_IsSupportRGBHDR(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.22 使能彩色 sensor hdr 功能

入参: hDevice 当前设备句柄

入参： bEnable VzTrue 启用 / VzFalse 禁用

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_EnableRGBHDR(VZNLHANDLE hDevice, VzBool bEnable);

VzBool VzNL_IsEnableRGBHDR(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.23 设置彩色 Sensor hdr 高曝光

入参: hDevice 当前设备句柄

入参： nValue 曝光值[20~100000]

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_SetRGBHDRHighExpose(VZNLHANDLE hDevice, unsigned int nValue);

unsigned int VzNL_GetRGBHDRHighExpose(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.24 设置彩色 Sensor hdr 低曝光

入参: hDevice 当前设备句柄

入参： nValue 曝光值[20~100000]

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_SetRGBHDRLowExpose(VZNLHANDLE hDevice, unsigned int nValue);

unsigned int VzNL_GetRGBHDRLowExpose(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.25 设置熔池检测区域

入参: hDevice 当前设备句柄

入参： pROIArea RGB ROI 区域

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_ConfigMoltenPoolROI(VZNLHANDLE hDevice, const SVzNLROIRect* pROIArea);

int VzNL_GetConfigMoltenPoolROI(VZNLHANDLE hDevice, SVzNLROIRect* pROIArea);

3.9.26 设置 RGBHDR 出图模式

入参: hDevice 当前设备句柄

入参：nMode 输出图像模式：原图，HDR 图，背景图，熔池图

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_SetRGBHDRImageMode(VZNLHANDLE hDevice, EVzRGBHDROutImageMode eMode);

EVzRGBHDROutImageMode VzNL_GetRGBHDRImageMode(VZNLHANDLE hDevice, int* pnErrorCode);

3.9.27 开始实时采集 RGB 图像

入参: hDevice 当前设备句柄

入参：pImageCB 图像回调接口

入参：pCBParam 回调接口参数

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_StartRGBCapture(VZNLHANDLE hDevice, VzNL_OutputRGBImageCB pImageCB, void* pCBParam);

3.9.28 停止实时采集 RGB 图像

入参: hDevice 当前设备句柄

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_StopRGBCapture(VZNLHANDLE hDevice);

3.9.29 设置熔池检测区域尺寸

入参: hDevice 当前设备句柄

入参： sROISize ROI 区域尺寸

返回： 0 为正确，失败返回错误码，错误码可通过 VzNL_GetErrorInfo 获取错误信息。

int VzNL_SetMoltenPoolROISize(VZNLHANDLE hDevice, const SVzNLSize sROISize[4]);

int VzNL_GetMoltenPoolROISize(VZNLHANDLE hDevice, SVzNLSize sROISize[4]);

3.10 防尘盖相关配置接口

头文件：#include "VZNL_DustCover.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.10.1 是否支持防飞溅翻盖的功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回： VzTrue 表示支持防尘盖功能,返回 VzFalse 表示不支持防尘盖功能。

VzBool VzNL_IsSupportCoverCamera(VZNLHANDLE hDevice, int* pnErrorCode);

3.10.2 关闭/打开防飞溅翻盖

入参: hDevice 当前设备句柄

入参: bCover [in] 关闭防尘盖为 VzTrue, 打开防尘盖为 VzFalse

返回: 正确返回 0， 失败返回其他值

int VzNL_CoverCamera(VZNLHANDLE hDevice, VzBool bCover);

3.10.3 是否关闭了防飞溅翻盖

入参: hDevice 当前设备句柄

入参: pnErrorCode 当 pnErrorCode 不为 nullptr 时,输出错误码

返回： VzTrue 表示关闭了防尘盖,返回 VzFalse 表示打开了防尘盖。

VzBool VzNL_IsCoverCamera(VZNLHANDLE hDevice, int* pnErrorCode);

3.10.4 是否开启了防飞溅翻盖控制的功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue 表示启用 VzFalse 表示不启用

VzBool VzNL_IsEnableAutoCoverControl(VZNLHANDLE hDevice, int* pnErrorCode);

3.10.5 开启/关闭防飞溅翻盖自动控制功能

入参: hDevice 当前设备句柄

入参: bEnable[in] 自动控制防飞溅翻盖为 VzTrue, 否则为 VzFalse

返回: 正确返回 0， 失败返回其他值

int VzNL_EnableAutoCoverControl(VZNLHANDLE hDevice, VzBool bEnable);

3.10.6 是否支持 IO 控制防飞溅翻盖的功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue 表示支持 VzFalse 表示不支持

VzBool VzNL_IsSupportIOControl(VZNLHANDLE hDevice, int* pnErrorCode);

3.10.7 开启/关闭 IO 控制防飞溅翻盖的功能

入参: hDevice 当前设备句柄

入参: bEnable VzTrue 表示开启 VzFalse 表示关闭

返回: 正确返回 0， 失败返回其他值

int VzNL_EnableIOControl(VZNLHANDLE hDevice, VzBool bEnable);

3.10.8 是否开启了 IO 控制防飞溅翻盖的功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue 表示启用 VzFalse 表示不启用

VzBool VzNL_IsEnableIOControl(VZNLHANDLE hDevice, int* pnErrorCode);

3.10.9 配置 IO 控制消抖参数

入参: hDevice 当前设备句柄

入参: nDbcTh 消抖参数

返回: 正确返回 0， 失败返回其他值

int VzNL_SetIOControlDbcTh(VZNLHANDLE hDevice, unsigned int nDbcTh);

3.10.10 获取 IO 控制消抖参数

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 消抖参数

unsigned int VzNL_GetIOControlDbcTh(VZNLHANDLE hDevice, int* pnErrorCode);

3.10.11 是否支持应用防飞溅翻盖大中小配置的功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue 表示支持 VzFalse 表示不支持

VzBool VzNL_IsSupportApplyDustCoverConfig(VZNLHANDLE hDevice, int* pnErrorCode);

3.10.12 根据类型使用防飞溅翻盖配置

入参: hDevice 当前设备句柄

入参: eConfigType 保存的配置类型

返回: 正确返回 0， 失败返回其他值

int VzNL_ApplyDustCoverConfigType(VZNLHANDLE hDevice, EVzDustCoverConfigType eConfigType);

3.10.13 获取当前使用防飞溅翻盖配置的类型

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 当前使用的配置类型

EVzDustCoverConfigType VzNL_QueryDustCoverConfigType(VZNLHANDLE hDevice, int* pnErrorCode);

3.11 统一内外部激光器配置接口

头文件：#include "VZNL_ExtLaserLight.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.11.1 是否支持关闭激光器

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 返回 VzTrue 表示支持，否则为不支持

VzBool VzNL_IsSupportTurnOffLaserLight(VZNLHANDLE hDevice, int* pnErrorCode);

3.11.2 关闭激光器

入参: hDevice 当前设备句柄

入参: bOff VzTrue:关闭，VzFalse:点亮

返回: 正确返回 0， 失败返回其他值

int VzNL_TurnOffLaserLight(VZNLHANDLE hDevice, VzBool bOff);

3.11.3 激光器是否已关闭

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 返回 VzTrue 表示已关闭，否则为未关闭

VzBool VzNL_IsSupportTurnOffLaserLight(VZNLHANDLE hDevice, int* pnErrorCode);

3.11.4 是否支持激光器亮度调节功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 返回 VzTrue 表示支持，否则为不支持

VzBool VzNL_IsSupportChangeLaserLightLevel(VZNLHANDLE hDevice, int* pnErrorCode);

3.11.5 获取外部激光器功率控制范围

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 返回控制范围

SVzNLRange VzNL_GetExtLaserLightRange(VZNLHANDLE hDevice, int* pnErrorCode);

3.11.6 配置功率等级

入参: hDevice 当前设备句柄

入参: nLevel 功率等级

返回: 成功返回 0，否则为错误码。

int VzNL_SetLaserLightLevel(VZNLHANDLE hDevice, int nLevel);

3.11.7 获取功率等级

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 当前激光功率等级

int VzNL_GetLaserLightLevel(VZNLHANDLE hDevice, int* pnErrorCode);

3.12 手眼标定转换接口

头文件：#include "VZNL_EyeHandPosConvert.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.12.1 初始化算法库

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 返回转换算法句柄

VZEHPCHANDLE VzNL_CreateEyeHandPosConvertHandle(VZNLHANDLE hDevice, int* pErrorCode);

3.12.2 释放算法库资源

入参: hConvertHandle 转换句柄

返回: 正确返回 0， 失败返回其他值

```
int VzNL_ReleaseEyeHandPosConvert(VZEHPCHANDLE hConvertHandle);
```

3.12.3 三轴标定算法函数接口

入参: hConvertHandle 转换句柄

入参: pCameraPoint 点数组

入参: nCount 点个数,目前仅支持 1 个点的转换

入参: dCalibMatrix 标定矩阵

入参: eCalibType 标定方式

入参: pRobotPoint 输出的机械臂坐标

返回: 正确返回 0， 失败返回其他值

```
int VzNL_EyeHandPosConvert3(VZEHPCHANDLE hConvertHandle,
    SVzNL3DPoint* pCameraPoints,
    int nCount,
    double dCalibMatrix[16],
    EVzHandEyeConvertType eConvertType,
    SVzNL3DPoint* pRobotPoint);
```

3.12.4 六轴标定算法函数接口：输入点坐标+三轴矢量

入参: hConvertHandle 转换句柄

入参: pCameraPoint 点数组

入参: pRobpose 拍摄时机械臂坐标

入参: nCount 点个数,目前仅支持 1 个点的转换

入参: dCalibMatrix 标定矩阵

入参: eConvertType 标定方式

入参: ePosSeqType 姿态顺序

入参: pRobotPoint 输出的机械臂坐标

返回: 正确返回 0， 失败返回其他值

```
int VzNL_EyeHandPosConvert6WithNormal(VZEHPCHANDLE hConvertHandle,
    SVzNLPCameraPointinfo* pCameraPoints,
    SVzNLRobotPointD* pRobpose,
    int nCount,
    double dCalibMatrix[16],
    EVzHandEyeConvertType eConvertType,
    EVzRPYSeqConvertType ePosSeqType,
    SVzNLRobotPointD* pRobotPoint);
```

3.12.5 六轴标定算法函数接口：输入点坐标+姿态角

入参: hConvertHandle 转换句柄

入参: pCameraPoint 点数组

入参: pRobpose 拍摄时机械臂坐标

入参: nCount 点个数,目前仅支持 1 个点的转换
入参: dCalibMatrix 标定矩阵
入参: eConvertType 标定方式
入参: ePosSeqType 姿态顺序
入参: pRobotPoint 输出的机械臂坐标
返回: 正确返回 0， 失败返回其他值

```
int VzNL_EyeHandPosConvert6WithEuler(VZEHPCHANDLE hConvertHandle,
    SVzNLRobotPointD* pCameraPoints,
    SVzNLRobotPointD* pRobpose,
    int nCount,
    double dCalibMatrix[16],
    EVzHandEyeConvertType eConvertType,
    EVzRPYSeqConvertType ePosSeqType,
    SVzNLRobotPointD* pRobotPoint);
```

3.12.6 R2RPY 旋转矩阵转 RPY 角

入参: hConvertHandle 转换句柄
入参: R 旋转矩阵 3*3
入参: RPY RPY 角,顺序为 Roll-Pitch-Yaw. 角度制
返回: 正确返回 0， 失败返回其他值

```
int VzNL_R2RPY(VZEHPCHANDLE hConvertHandle, double R[9], double RPY[3], const
EVzRPYSeqConvertType ePosSeqType);
```

3.12.7 R2RPY RPY 角转旋转矩阵

入参: hConvertHandle 转换句柄
入参: R 旋转矩阵 3*3
入参: RPY RPY 角,顺序为 Roll-Pitch-Yaw. 角度制
返回: 正确返回 0， 失败返回其他值

```
int VzNL_R2RPY(VZEHPCHANDLE hConvertHandle, double R[9], double RPY[3], const
EVzRPYSeqConvertType ePosSeqType);
```

3.13 C 版激光器接口

头文件：#include "VZNL_ExtStrobeLaser.h"
库文件：VzNLDetect.dll / libVzNLDetect.so

3.13.1 是否支持 C 版功能

入参: hDevice 当前设备句柄
入参: pnErrorCode 错误信息，如果不需要可填 NULL
返回: 返回 VzTrue 表示支持，否则为不支持

```
VzBool VzNL_IsSupportStrobe(VZNLHANDLE hDevice, int* pnErrorCode);
```

3.13.2 开启/关闭 C 版功能

入参: hDevice 当前设备句柄

入参: bEnable VzTrue:开启，VzFalse:不开启

返回: 成功返回 0，否则为错误码。

int VzNL_EnableStrobe(VZNLHANDLE hDevice, VzBool bEnable);

3.13.3 激光器是否开启 C 版功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 返回 VzTrue 表示支持，否则为不支持

VzBool VzNL_IsEnableStrobe(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.4 点亮激光器

入参: hDevice 当前设备句柄

入参: bOn VzTrue:点亮，VzFalse:关闭

返回: 成功返回 0，否则为错误码。

int VzNL_TurnOnStrobeLaser(VZNLHANDLE hDevice, VzBool bOn);

3.13.5 激光器是否已点亮

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue:点亮，VzFalse:关闭

VzBool VzNL_IsTurnOnStrobeLaser(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.6 配置 C 版激光器点亮时长（微妙）

入参: hDevice 当前设备句柄

入参: nTime 时长

返回: 成功返回 0，否则为错误码。

int VzNL_SetStrobeTime(VZNLHANDLE hDevice, unsigned int nTime);

3.13.7 获取 C 版激光器点亮时长（微妙）

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 时长

unsigned int VzNL_GetStrobeTime(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.8 配置 C 版激光器点亮延迟时长（微妙）

入参: hDevice 当前设备句柄

入参: nTime 时长

返回: 成功返回 0，否则为错误码。

int VzNL_SetStrobeTimeOffset(VZNLHANDLE hDevice, unsigned int nTime);

3.13.9 获取 C 版激光器点亮延迟时长（微妙）

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 延迟时长

unsigned int VzNL_GetStrobeTimeOffset(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.10 是否支持激光器 PWM 调节功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue:支持，VzFalse:不支持

VzBool VzNL_IsSupportPwmControl(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.11 配置 PWM 频率

入参: hDevice 当前设备句柄

入参: nFreq 频率

返回: 成功返回 0，否则为错误码。

int VzNL_SetPwmFreq(VZNLHANDLE hDevice, unsigned int nFreq);

3.13.12 获取 PWM 频率

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 频率

unsigned int VzNL_GetPwmFreq(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.13 配置 PWM 占空比

入参: hDevice 当前设备句柄

入参: nRatio 占空比 例如：50 ，代表占空比 50%

返回: 成功返回 0，否则为错误码。

int VzNL_SetPwmRatio(VZNLHANDLE hDevice, unsigned int nRatio);

3.13.14 获取 PWM 占空比

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 占空比

unsigned int VzNL_GetPwmRatio(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.15 是否支持补光灯

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue:支持，VzFalse:不支持

VzBool VzNL_IsSupportExtLight(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.16 开启/关闭补光灯

入参: hDevice 当前设备句柄

入参: bOn VzTrue:开启，VzFalse:不开启

返回: 成功返回 0，否则为错误码。

int VzNL_TurnOnExtLight(VZNLHANDLE hDevice, VzBool bOn);

3.13.17 是否开启了补光灯

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue:开启，VzFalse:未开启

VzBool VzNL_IsTurnOnExtLight(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.18 设置抗反光增益

入参: hDevice 当前设备句柄

入参: eAntiReflectGainType 抗反光增益类型

返回: 成功返回 0，否则为错误码

int VzNL_SetAntiReflectGainType(VZNLHANDLE hDevice, EVzAntiReflectGainType eAntiReflectGainType);

3.13.19 获取抗反光增益

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 抗反光增益类型

EVzAntiReflectGainType VzNL_GetAntiReflectGainType(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.20 获取 C 版激光器占空比值

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 当前 C 版激光器点亮时间占空比（浮点数，取值范围[0.0, 100.0]），只为查看占空比值，无需设置；

float VzNL_GetStrobeTimeRatio(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.21 开启/关闭补光灯自动控制

入参: hDevice 当前设备句柄

入参: bEnable VzTrue:开启，VzFalse:不开启

返回: 成功返回 0，否则为错误码

int VzNL_EnableAutoControlExtLight(VZNLHANDLE hDevice, VzBool bEnable);

3.13.22 是否开启补光灯自动控制

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue:开启，VzFalse:未开启

VzBool VzNL_IsEnableAutoControlExtLight(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.23 配置自动控制补光灯开启后的等待时间

入参: hDevice 当前设备句柄

入参: nWaitTime 等待时间

返回: 成功返回 0，否则为错误码

int VzNL_SetAutoControlWaitTime(VZNLHANDLE hDevice, unsigned int nWaitTime);

3.13.24 获取自动控制补光灯开启后的等待时间

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 等待时间

unsigned int VzNL_GetAutoControlWaitTime(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.25 是否支持 ch3 功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue:支持，VzFalse:不支持

VzBool VzNL_IsEnableCh3(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.26 开启/关闭 ch3 功能

入参: hDevice 当前设备句柄

入参: bEnable VzTrue:开启，VzFalse:不开启

返回: 成功返回 0，否则为错误码

int VzNL_EnableCh3(VZNLHANDLE hDevice, VzBool bEnable);

3.13.27 激光器是否 ch3 功能

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: VzTrue:开启，VzFalse:未开启

VzBool VzNL_IsEnableCh3(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.28 配置 ch3 频率

入参: hDevice 当前设备句柄

入参: nFreq 频率

返回: 成功返回 0，否则为错误码

int VzNL_SetCh3Freq(VZNLHANDLE hDevice, unsigned int nFreq);

3.13.29 获取 ch3 频率

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 频率

unsigned int VzNL_GetCh3Freq(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.30 配置 ch3 占空比

入参: hDevice 当前设备句柄

入参: nRatio 频率

返回: 成功返回 0，否则为错误码

int VzNL_SetCh3Ratio(VZNLHANDLE hDevice, unsigned int nRatio);

3.13.31 获取 ch3 占空比

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 占空比

unsigned int VzNL_GetCh3Ratio(VZNLHANDLE hDevice, int* pnErrorCode);

3.13.32 配置 C 版激光自动关闭的时间

入参: hDevice 当前设备句柄

入参: nTime 时间，单位分钟，默认 1 分钟

返回: 成功返回 0，否则为错误码

int VzNL_SetAutoTurnOffStrobeLaserTime(VZNLHANDLE hDevice, unsigned int nTime);

3.13.33 获取 C 版激光自动关闭的时间

入参: hDevice 当前设备句柄

入参: pnErrorCode 错误信息，如果不需要可填 NULL

返回: 时间

unsigned int VzNL_GetAutoTurnOffStrobeLaserTime(VZNLHANDLE hDevice, int* pnErrorCode);

3.14 相机配置文件相关接口

头文件：#include "VZNL_PetaLinux.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.14.1 保存用户配置到文件

入参: hDevice 当前设备句柄

入参: lpszFile 文件

返回: 成功返回 0，否则为错误码

int VzNL_SaveUserSettingToFile(VZNLHANDLE hDevice, const char* lpszFile);

3.14.2 恢复用户配置

入参: hDevice 当前设备句柄

入参: lpszFile 文件

返回: 成功返回 0，否则为错误码

int VzNL_RestoreUserSettingFromFile(VZNLHANDLE hDevice, const char* lpszFile);

3.14.3 读取用户文件列表

入参: hDevice 当前设备句柄

入参: szUserProfileName 获取用户配置名

入参: nProfileCount 获取文件个数

返回: 成功返回 0，否则为错误码

int VzNL_ReadProfileNameList(VZNLHANDLE hDevice, char szUserProfileName[VZ_USERPROFILE_MAX_COUNT][VZ_USERPROFILE_NAME_LENGTH]);

3.14.4 保存用户配置(共 10 个)

入参: hDevice 当前设备句柄

入参: nProfileIdx 文件 Index

入参: lpszName 名称

返回: 成功返回 0，否则为错误码

int VzNL_SaveUserSetting(VZNLHANDLE hDevice, unsigned int nProfileIdx, const char* lpszName);

3.14.5 恢复用户配置

入参: hDevice 当前设备句柄

入参: nProfileIdx 文件 Index

返回: 成功返回 0，否则为错误码

int VzNL_RestoreUserSettingFromIndex(VZNLHANDLE hDevice, unsigned int nProfileIdx);

3.14.6 恢复用户配置

入参: hDevice 当前设备句柄

入参: lpszName 名称

返回: 成功返回 0，否则为错误码

int VzNL_RestoreUserSettingFromName(VZNLHANDLE hDevice, const char* lpszName);

3.14.7 恢复系统默认配置

入参: hDevice 当前设备句柄

返回: 成功返回 0，否则为错误码

int VzNL_RestoreDefaultSetting(VZNLHANDLE hDevice);

3.15 PetaLinux 相关接口

头文件：#include "VZNL_PetaLinux.h"

库文件：VzNLDetect.dll / libVzNLDetect.so

3.15.1 使能 PetaLinux 通道

入参: hDevice 当前设备句柄

入参: bEnable VzTrue:使能，VzFalse:不使能

返回: 成功返回 0，否则为错误码

VZNLAPI int VzNL_EnablePetaLinux(VZNLHANDLE hDevice, VzBool bEnable);

使用 PetaLinux 通道使用流程示例：

1. VzNL_OpenDevice
2. VzNL_BeginDetectLaser
3. VzNL_SetDeviceStatusNotifyEx(m_hDevice, ReadDataCallBack, nullptr);
4. VzNL_EnablePetaLinux(m_hDevice, VzTrue)
5. VzNL_StartAutoDetect 后有触发的话 ReadDataCallBack 就有回调

```
void ReadDataCallBack(EVzDeviceWorkStatus eStatus, void* pExtData, unsigned int nDataLength, void*
pInfoParam)
{
    if(keDeviceWorkStatus_PetaMsg == eStatus)
    {
        int nType = *(int *)(pExtData);

        if (4 == nType)
        {
            int nCount = *(int *)((char *)pExtData + 4);
            printf("count : %d\n", nCount);
        }

    }
}
```

4 接口改动记录

4.1 版本 3.6.20.32

4.1.1 添加支持 x 轴点云过滤(适用于所有相机)

```
/// @brief
/// 配置激光线过滤范围
/// <param name = "hDevice">[in]设备</param>
/// <param name = "eFilterType">[in]过滤类型</param>
/// <param name = "dRange">[in]过滤范围</param>
/// @return 成功返回0，失败返回错误码
VZNLAPI int VzNL_SetLaserLineFilterRange(VZNLHANDLE hDevice, EVzLaserPointFilterType eFilterType, double dRange[2]);

/// @brief
/// 获取激光线过滤范围
/// <param name = "hDevice">[in]设备</param>
/// <param name = "eFilterType">[in]过滤类型</param>
/// <param name = "dRange">[in]过滤范围</param>
/// @return 成功返回0，失败返回错误码
VZNLAPI int VzNL_GetLaserLineFilterRange(VZNLHANDLE hDevice, EVzLaserPointFilterType eFilterType, double dRange[2]);
```

4.1.2 移除激光器在相机上安装(适用于所有相机)

```
/**
* @brief 激光器在相机上安装？
* @param hDevice [in] 设备Handle
* @param bOnCamera [in] VzTrue在相机上/VzFalse不在相机上
* @return 关闭成功返回0，否则为错误码
*/
VZNLAPI int VzNL_LaserEmitterOnCamera(VZNLHANDLE hDevice, VzBool bOnCamera);
VZNLAPI VzBool VzNL_HasLaserEmitterOnCamera(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.1.3 移除当前相机检测触发信号接口(适用于所有相机)

```
/**
 * @brief 当前相机是否有触发信号进来？
 * @param hDevice [in] 设备Handle
 * @param pnErrorCode [out] 错误码
 * @return VzTrue有触发信号/VzFalse无触发信号
 */
VZNLAPI VzBool VzNL_HasTriggerSignalToCamera(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 获取当前相机连接的编码器值
 * @param hDevice [in] 设备Handle
 * @param pnErrorCode [out] 错误码
 * @return VzTrue有触发信号/VzFalse无触发信号
 */
VZNLAPI int VzNL_GetCameraTriggerEncoderData(VZNLHANDLE hDevice, SVzEncoderData* pEncoderData);
```

4.1.4 移除相机是否做过面标定接口(适用于所有相机)

```
/**
 * @brief 相机是否做过面标定
 * @param hDevice 当前设备句柄
 * @param pnErrorCode 错误信息，如果不需要可填NULL
 * @return VzTrue 表示做过 VzFalse 表示没做过
 */
VZNLAPI VzBool VzNL_IsSupportStaticCalibParam(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 眼睛是否标定过
 * @param [in] hDevice          设备句柄
 * @param [in] pErrorCode       ErrorCode
 * @return 如果标定过返回VzTrue
 */
VZNLAPI VzBool VzNL_LaserIsCalibration(VZNLHANDLE hDevice, int* pErrorCode);
```

4.1.5 移除设置激光门限值接口(适用于所有相机)

```
/**
 * @brief 设置激光门限值(只支持EyeCB和星光眼)
 * @param [in] hDevice 设备句柄
 * @param [in] nLaserThres 激光门限值[Range:1~255]
 * @return 返回0表示正确
 * @retval 0 表示成功
 * @retval 非0 表示失败,可以使用VzNL_GetErrorInfo获取
 */
VZNLAPI int VzNL_SetLaserThres(VZNLHANDLE hDevice, int nLaserThres);
VZNLAPI int VzNL_GetLaserThres(VZNLHANDLE hDevice, int* pnLaserThres);
```

4.1.6 移除设置数据扫描长度,当扫描达到指定长度后，自动停止扫描接口(适用于所有相机)

```
/**
 * @brief 设置数据扫描长度,当扫描达到指定长度后，自动停止扫描。
 * @param [in] hDevice          设备句柄
 * @param [in] nDistance        扫描长度,单位毫米(mm)
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetLaserCaptureDistance(VZNLHANDLE hDevice, double dDistance);
VZNLAPI int VzNL_GetLaserCaptureDistance(VZNLHANDLE hDevice, double* dnDistance);
```

4.1.7 移除设置网络配置信息接口(适用于所有相机)

```
/**
 * @brief 重置网络极光眼的配置信息
 * @param [in] pEyeCBInfo   设备信息
 * @param [in] byEyeIP      眼睛IP
 * @return 返回0表示成功
 */
VZNLAPI int VzNL_ConfigEthernetEyeConfigInfo(const SVzNLEyeCBInfo* pEyeCBInfo,
    unsigned char byEyeIP[4]);

/**
* @brief 重置网络配置信息
* @param [in] pEyeCBInfo   设备信息
* @param [in] byEyeIP      眼睛IP
* @param [in] byMask       子网掩码
* @param [in] byGateWay    网关
* @return 返回0表示成功
*/
VZNLAPI int VzNL_ConfigEthernetInfo(const SVzNLEyeCBInfo* pEyeCBInfo,
    unsigned char byEyeIP[4],
    unsigned char byMask[4],
    unsigned char byGateWay[4]);

/**
* @brief 重置网络配置信息
* @param [in] pEyeCBInfo   设备信息
* @param [in] eEyeIPType   眼睛IP 类型
* @param [in] byEyeIP      眼睛IP
* @param [in] byMask       子网掩码
* @param [in] byGateWay    网关
* @return 返回0表示成功
*/
VZNLAPI int VzNL_ConfigEthernetInfoEx(const SVzNLEyeCBInfo* pEyeCBInfo,
    EVzNLEthernetEyeIPType eEyeIPType,
    unsigned char byEyeIP[4],
    unsigned char byMask[4],
    unsigned char byGateWay[4]);

/**
 * @brief 设置IP类型
 * @param [in] pEyeCBInfo   设备信息
 * @param [in] eIPType      IP类型[keNLEthernetEyeIPType_StaticIP:静态IP keNLEthernetEyeIPType_DHCP:动态分配IP]
 * @return 返回0表示成功
 */
VZNLAPI int VzNL_ChangeEthernetIPType(const SVzNLEyeCBInfo* pEyeCBInfo,
    EVzNLEthernetEyeIPType eIPType);
```

4.1.8 移除获取 RGB 异步接口(适用于动态 RGBD 相机)

```
/**
 * @brief 获取RGBD所在位置
 * @param hDevice 当前设备句柄
 * @param dCCMVal CCM值
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_GetRGBAsyncRect(VZNLHANDLE hDevice, SVzNLROIRect pROIRect[2]);


/**
 * @brief 获取RGB镜头的X方向视野大小(mm)
 * @param hDevice 当前设备句柄
 * @param nViewDx X方向视野大小(mm)
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_GetRGBViewXSize(VZNLHANDLE hDevice, unsigned int& nViewDx);
```

4.1.9 移除设置是否输出 ROI 图像接口(适用于所有相机)

```
////////////////////////////////////////////////////////////////////////////
/**
 * @brief 设置是否输出ROI图像
 * @param [in] bOutputROI 是否输出ROI图像
 */
VZNLAPI void VzNL_SetOutputROIImage(VzBool bOutputROI);
```

4.1.10 移除多系统相关函数接口(适用于所有相机)

```
/// @name 多系统相关函数(gtx专用接口)
/// @{
/**
 * @brief 获取当前系统列表
 * @param hDevice 设备Handle
 * @param pnCount 输出系统的个数
 * @return 系统列表数组
 */
VZNLAPI SVzSystemProjectInfo* VzNL_GetSystemPrjArray(VZNLHANDLE hDevice, int* pnCount);

/**
 * @brief 设置当前系统索引
 * @param hDevice 设备Handle
 * @param nCurSel 要配置为第几个系统为运行时的系统
 * @return 返回0为正确，其他错误码请用GetErrorString。
 */
VZNLAPI int VzNL_SwitchSystemPrj(VZNLHANDLE hDevice, int nCurSel);

/**
 * @brief 获取当前系统索引
 * @param hDevice 设备Handle
 * @return 当前系统的索引
 */
VZNLAPI int VzNL_GetCurrentSystemPrj(VZNLHANDLE hDevice);

/**
 * @brief 查询运行时间(s)
 * @param hDevice 设备Handle
 * @param pnErrorCode 错误码
 * @return 返回运行时间
 */
VZNLAPI double VzNL_GetSystemRuntime(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.1.11 移除激光标定函数接口(适用于所有相机)

```
/**
 * @brief 设置激光标定。
 * @param [in] hDevice              设备句柄
 * @param [in] dStandardObjectHeight 设置标定高度
 * @param [in] pTopROI              顶部ROI数组指针
 * @param [in] nTopROICount         顶部ROI的数组个数
 * @param [in] pBottomROI           底部ROI数组指针
 * @param [in] nBottomROICount      底部ROI数组个数
 * @return 返回0为正确，失败返回错误码
 */
VZNLAPI int VzNL_SetLaserStandard(VZNLHANDLE hDevice, double dStandardObjectHeight,
    const SVzNLROIRect* pTopROI, const int nTopROICount,
    const SVzNLROIRect* pBottomROI, const int nBottomROICount);

/**
 * @brief 清除标定信息。
 * @param [in] hDevice              设备句柄
 * @return 返回0为正确，失败返回错误码
 */
VZNLAPI int VzNL_ClearLaserStandard(VZNLHANDLE hDevice);
```

4.1.12 移除设置自动调节超时时间函数接口(适用于所有相机)

```
/**
 * @brief 设置自动调节超时时间
 * @param hDevice 当前设备句柄
 * @param nTimeOut 设置自动调节超时时间
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_SetRGBAutoAdjustTimeOut(VZNLHANDLE hDevice, unsigned int nTimeOut);
VZNLAPI unsigned int VzNL_GetRGBAutoAdjustTimeOut(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.1.13 移除启用存储参数函数接口(适用于所有相机)

```
/**
 * @brief 启用存储参数
 * @param [in] hDevice 设备句柄
 * @return 返回0表示正确
 * @retval 0 表示成功
 * @retval 非0 表示失败,可以使用VzNL_GetErrorInfo获取
 */
VZNLAPI int VzNL_EnableSaveSwingParam(VZNLHANDLE hDevice, VzBool bEnable);
```

4.2 版本 3.6.20.33（没有接口变化）

4.3 版本 3.6.20.34

4.3.1 添加开启/关闭防飞溅翻盖自动控制功能(适用于静态相机)

```
/** @brief 开启/关闭防飞溅翻盖自动控制功能
 *  @param hDevice      [in] 设备Handle
 *  @param bCover       [in] 是否自动控制防飞溅翻盖
 *  @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_EnableAutoCoverControl(VZNLHANDLE hDevice, VzBool bEnable);

/**
 * @brief 是否支持防飞溅翻盖控制的功能
 * @param hDevice 当前设备句柄
 * @param pnErrorCode 错误信息，如果不需要可填NULL
 * @return VzTrue 表示支持 VzFalse 表示不支持
 */
VZNLAPI VzBool VzNL_IsEnableAutoCoverControl(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.3.2 添加舵机摆动速度范围的接口(适用于静态相机)

```
/**
* @brief 设置/获取角速度
* @param [in] hDevice 设备句柄
* @param [in] bEnable 启用/禁用
* @return 返回0表示正确
* @retval 0 表示成功
* @retval 非0 表示失败,可以使用VzNL_GetErrorInfo获取
*/
VZNLAPI int VzNL_SetSwingAngleSpeed(VZNLHANDLE hDevice, float fAngleSpeed);
VZNLAPI int VzNL_GetSwingAngleSpeed(VZNLHANDLE hDevice, float* pfdAngleSpeed);
VZNLAPI int VzNL_GetSwingAngleSpeedRange(VZNLHANDLE hDevice, float* pfMinAngleSpeed, float* pfMaxAngleSpeed);
```

4.4 版本 3.6.20.35

4.4.1 添加开启/关闭快速采图模式功能(适用于静态相机)

```
/**
 * @brief 开启/关闭快速采图模式
 * @param hDevice 当前设备句柄
 * @param [in] bEnableQuickMode 是否启用
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableQuickMode(VZNLHANDLE hDevice, VzBool bEnableQuickMode);
VZNLAPI VzBool VzNL_IsEnableQuickMode(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.5 版本 3.6.20.36(没有接口变化)

4.6 版本 3.6.20.37

4.6.1 修改摆动速度的数据类型由浮点型为整形(适用于静态相机)

```
/**
 * @brief 设置/获取角速度
 * @param [in] hDevice 设备句柄
 * @param [in] bEnable 启用/禁用
 * @return 返回0表示正确
 * @retval 0 表示成功
 * @retval 非0 表示失败,可以使用VzNL_GetErrorInfo获取
 */
VZNLAPI int VzNL_SetSwingAngleSpeed(VZNLHANDLE hDevice, int nAngleSpeed);
VZNLAPI int VzNL_GetSwingAngleSpeed(VZNLHANDLE hDevice, int* pnAngleSpeed);
VZNLAPI int VzNL_GetSwingAngleSpeedRange(VZNLHANDLE hDevice, int* pnMinAngleSpeed, int* pnMaxAngleSpeed);
```

```
/**
 * @brief 设置/获取角速度
 * @param [in] hDevice 设备句柄
 * @param [in] bEnable 启用/禁用
 * @return 返回0表示正确
 * @retval 0 表示成功
 * @retval 非0 表示失败,可以使用VzNL_GetErrorInfo获取
 */
VZNLAPI int VzNL_SetSwingMotorAngle(VZNLHANDLE hDevice, int nStartPos, int nEndPos);
VZNLAPI int VzNL_GetSwingMotorAngle(VZNLHANDLE hDevice, int* pnStartPos, int* pnEndPos);
```

```
/**
 * @brief 设置相机激光器停止位置
 * @param [in] hDevice 设备句柄
 * @param [in] fAngle 停止角度
 * @return 返回0表示正确
 * @retval 0 表示成功
 * @retval 非0 表示失败,可以使用VzNL_GetErrorInfo获取
 */
VZNLAPI int VzNL_SetSwingStopAngle(VZNLHANDLE hDevice, int nAngle);
VZNLAPI int VzNL_GetSwingStopAngle(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.6.2 增加瞳距数据的获取的接口(适用于所以相机)

```
/**
 * @brief 获取相机瞳距信息
 * @param hDevice 当前设备句柄
 * @param [in] nPupilDistance 瞳距
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_GetPupilDistance(VZNLHANDLE hDevice, unsigned int& nPupilDistance);
```

4.6.3 统一动/静态激光器开关控制和亮度控制逻辑(适用于所以相机)

```
/**
 * @brief 是否支持关闭激光器
 * @param [in] hDevice 设备句柄
 * @param [in] bOn VzTrue:点亮, VzFalse:关闭
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI VzBool VzNL_IsSupportTurnOffLaserLight(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 关闭激光器
 * @param [in] hDevice 设备句柄
 * @param [in] bOff VzTrue:关闭, VzFalse:点亮
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_TurnOffLaserLight(VZNLHANDLE hDevice, VzBool bOff);

/**
 * @brief 激光器是否已关闭
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 成功返回0，否则为错误码。
 * @return VzTrue:关闭, VzFalse:点亮。
 */
VZNLAPI VzBool VzNL_IsTurnOffLaserLight(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 是否支持激光器亮度调节功能
 * @param [in] hDevice 设备句柄
 * @param [in] bOn VzTrue:点亮, VzFalse:关闭
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI VzBool VzNL_IsSupportChangeLaserLightLevel(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
/**
 * @brief 获取外部激光器功率控制范围
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return 成功返回功率范围
 */
VZNLAPI SVzNLRange VzNL_GetExtLaserLightRange(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置功率等级
 * @param [in] hDevice 设备句柄
 * @param [in] nLevel 功率等级
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetLaserLightLevel(VZNLHANDLE hDevice, int nLevel);

/**
 * @brief 获取功率等级
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return 成功返回当前激光功率等级
 */
VZNLAPI int VzNL_GetLaserLightLevel(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.7 版本 3.6.20.38(没有接口变化)

4.8 版本 3.6.20.39

4.8.1 添加多帧数据一起回调的接口(适用于所以相机)

```
/**
 * @brief 设置每次回调返回多少帧数据的数量
 * @param [in] hDevice  设备句柄
 * @param [in] nCount   多少帧回调一次数据
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetOutputLaserLinesEachTime(VZNLHANDLE hDevice, unsigned int nNum);
VZNLAPI int VzNL_GetOutputLaserLinesEachTime(VZNLHANDLE hDevice, unsigned int* nNum);
```

4.9 版本 3.6.20.40(没有接口变化)

4.10 版本 3.6.20.41

4.10.1 添加获取左右目分辨率的接口(适用于所以相机)

```
/**
 * @brief 获取左右目分辨率
 * @param hDevice [in] 设备Handle
 * @return 关闭成功返回0，否则为错误码
 */
VZNLAPI int VzNL_GetResolution(VZNLHANDLE hDevice, SVzVideoResolution* psVideoRes);
```

4.11 版本 3.6.20.42

4.11.1 添加是否支持 IO 控制相机舵机功能的接口(适用于带有翻盖的静态相机)

```
/**
 * @brief 是否支持IO控制相机舵机的功能
 * @param hDevice 当前设备句柄
 * @param pnErrorCode 错误信息，如果不需要可填NULL
 * @return VzTrue 表示支持 VzFalse 表示不支持
 */
VZNLAPI VzBool VzNL_IsSupportIOControl(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.11.2 添加使能 IO 控制相机舵机功能的接口(适用于带有翻盖的静态相机)

```
/**
 * @brief 开启/关闭IO控制相机舵机的功能
 * @param hDevice 当前设备句柄
 * @param bEnable VzTrue 表示开启 VzFalse 表示关闭
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_EnableIOControl(VZNLHANDLE hDevice, VzBool bEnable);
```

4.11.3 添加是否使能了 IO 控制相机舵机功能的接口(适用于带有翻盖的静态相机)

```
/**
 * @brief 是否开启了IO控制相机舵机的功能
 * @param hDevice 当前设备句柄
 * @param pnErrorCode 错误信息，如果不需要可填NULL
 * @return VzTrue VzTrue 表示开启 VzFalse 表示关闭
 */
VZNLAPI VzBool VzNL_IsEnableIOControl(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.11.4 添加配置/获取 IO 控制消抖参数的接口(适用于带有翻盖的静态相机)

```
/**
 * @brief 配置/获取IO控制消抖参数
 * @param hDevice 当前设备句柄
 * @param nDbcTh  消抖参数
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_SetIOControlDbcTh(VZNLHANDLE hDevice, unsigned int nDbcTh);
VZNLAPI unsigned int VzNL_GetIOControlDbcTh(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.12 版本 3.6.20.43(没有接口变化)

4.13 版本 3.6.20.44(没有接口变化)

4.14 版本 3.6.20.45

4.14.1 添加将输入的点云数据转换 2D 伪彩图像输出的接口(适用于带有翻盖的静态相机)

```
/*
* 功能：将输入的点云数据转换为指定分辨率的 2D 伪彩图像输出
* 参数说明：
*   EVzResultDataType eDataType: 输入点数据类型，内部根据数据类型解析数据；
*   unsigned int nPointCnt: 输入 pPointList 中点数据个数；
*   void* pPointList: 输入点数据列表。
*   unsigned int nImageWidth: 输入希望输出的图像宽度；
*   unsigned int nImageHeight: 输入希望输出的图像高度；
*   unsigned int nColorGradient: 输出点数据颜色梯度
*   unsigned int nEdgeMargin: 输入图像边缘的留白宽度(像素值，四边留白)；
*   SVzNLImageData* pImage: 输出的图像数据；
*/
VZNLAPI int VzNL_PointCloudTo2DFixedSizeImage(EVzResultDataType eDataType,
    unsigned int nPointCnt,
    void* pPointList,
    unsigned int nImageWidth,
    unsigned int nImageHeight,
    unsigned int nColorGradient,
    unsigned int nEdgeMargin,
    SVzNLImageData* pImage);

/*
* 功能：将输入的点云数据转换为指定像素精度的 2D 伪彩图像输出
* 参数说明：
*   EVzResultDataType eDataType: 输入点数据类型，内部根据数据类型解析数据；
*   unsigned int nPointCnt: 输入 pPointList 中点数据个数；
*   void* pPointList: 输入点数据列表。
*   float fWHPixelResolution: 输入图像像素精度，代表每个像素对应的实际物理距离，例如：0.1 表示像素间隔为0.1毫米；
*   unsigned int nColorGradient: 输出点数据颜色梯度
*   unsigned int nEdgeMargin: 输入图像边缘的留白宽度(像素值，四边留白)；
*   SVzNLImageData* pImage: 输出的图像数据；
*/
VZNLAPI int VzNL_PointCloudTo2DFixedResolutionImage(EVzResultDataType eDataType,
    unsigned int nPointCnt,
    void* pPointList,
    float fWHPixelResolution,
    unsigned int nColorGradient,
    unsigned int nEdgeMargin,
    SVzNLImageData* pImage);
```

```
/*
* 释放输出的图像数据
*/
VZNLAPI int VzNL_PointCloudTo2DImageRelease(SVzNLImageData* pImage);
```

4.15 版本 3.6.20.46(没有接口变化)

4.16 版本 3.6.20.47

4.16.1 添加防尘盖大中小配置的功能的接口(适用于带有翻盖的静态相机)

```
/**
 * @brief 是否支持应用防尘盖大中小配置的功能
 * @param hDevice 当前设备句柄
 * @param pnErrorCode 错误信息，如果不需要可填NULL
 * @return VzTrue 表示支持 VzFalse 表示不支持
 */
VZNLAPI VzBool VzNL_IsSupportApplyDustCoverConfig(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 根据类型使用舵机配置
 * @param hDevice 当前设备句柄
 * @param [in] eConfigType 保存的配置类型
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_ApplyDustCoverConfigType(VZNLHANDLE hDevice, EVzDustCoverConfigType eConfigType);
/**
 * @brief 获取使用舵机配置的类型
 * @param hDevice 当前设备句柄
 * @param [out] pnErrorCode 成功返回0，否则为错误码。
 * @return 当前使用的配置类型
 */
VZNLAPI EVzDustCoverConfigType VzNL_QueryDustCoverConfigType(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.17 版本 3.6.20.48

4.17.1 添加开启/关闭以左目坐标为原点来点云计算方式的接口(适用于所有相机)

```
/**
 * @brief 开启/关闭以左目坐标为原点来点云计算方式
 * @param hDevice 当前设备句柄
 * @param [in] bEnable 是否启用
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableLeftEyeOrigin(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnableLeftEyeOrigin(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.17.2 添加获取左目内参的接口(适用于所有相机)

```
/**
 * @brief 获取左目内参
 * @param hDevice 当前设备句柄
 * @param [in] dMatrix 左目内参矩阵 3*3 + 5*1
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_QueryLeftEyeCalibData(VZNLHANDLE hDevice, double dLeftMatrix[14]);
```

4.17.3 添加获取左目到 3D 点云的转换矩阵的接口(适用于所有相机)

```
/**
 * @brief 获取左目内参
 * @param hDevice 当前设备句柄
 * @param [in] dMatrix 左目内参矩阵 3*3 + 5*1
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_QueryLeftEyeCalibData(VZNLHANDLE hDevice, double dLeftMatrix[14]);
```

4.17.4 添加获取 3D 点云到左目的转换矩阵的接口(适用于所有相机)

```
/**
 * @brief 获取3D点云到左目的转换矩阵
 * @param hDevice 当前设备句柄
 * @param [in] dLeft2Stereo 左目到3D的转换矩阵 4*4
 * @param [in] dStereo2Left 3D到左目的转换矩阵 4*4
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_QueryStereo2LeftEyeMatrix(VZNLHANDLE hDevice, double dMatrix[16]);
```

4.18 版本 3.6.20.49

4.18.1 添加手眼标定转换接口(适用于所有相机)

```
/**
 * @brief 初始化算法库
 * @param hDevice 当前设备句柄
 * @param [out] pErrorCode          返回错误码，为 NULL时，不返回
 * @return 转换句柄
 */
VZNLAPI VZEHPCHANDLE VzNL_CreateEyeHandPosConvertHandle(VZNLHANDLE hDevice, int* pErrorCode);

/**
 * @brief 释放算法库资源
 * @param hConvertHandle 转换句柄
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_ReleaseEyeHandPosConvert(VZEHPCHANDLE hConvertHandle);

/* 三轴标定算法函数接口
    * hConvertHandle 转换句柄
    * pCameraPoint  点数组
    * nCount   点个数,目前仅支持1个点的转换
    * dCalibMatrix 标定矩阵
    * eConvertType 标定方式
    * pRobotPoint 输出的机械臂坐标
    * @return 正确返回0， 失败返回其他值
    */
VZNLAPI int VzNL_EyeHandPosConvert3(VZEHPCHANDLE hConvertHandle,
    SVzNL3DPoint* pCameraPoints,
    int nCount,
    double dCalibMatrix[16],
    EVzHandEyeConvertType eConvertType,
    SVzNL3DPoint* pRobotPoint);
```

```
/* 六轴标定算法函数接口：输入点坐标+三轴矢量
 * hConvertHandle 转换句柄
 * pCameraPoints 相机点数组
 * pRobpose  拍摄时机械臂坐标
 * nCount   点个数,目前仅支持1个点的转换
 * dCalibMatrix 标定矩阵
 * eConvertType 标定方式
 * ePosSeqType 姿态顺序
 * pRobotPoint 输出的机械臂坐标
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_EyeHandPosConvert6WithNormal(VZEHPCHANDLE hConvertHandle,
    SVzNLPCameraPointinfo* pCameraPoints,
    SVzNLRobotPointD* pRobpose,
    int nCount,
    double dCalibMatrix[16],
    EVzHandEyeConvertType eConvertType,
    EVzRPYSeqConvertType ePosSeqType,
    SVzNLRobotPointD* pRobotPoint);

/* 六轴标定算法函数接口：输入点坐标+姿态角
 * hConvertHandle 转换句柄
 * pCameraPoints  相机点数组// xyzabc
 * pRobpose  拍摄时机械臂坐标
 * nCount   点个数,目前仅支持1个点的转换
 * dCalibMatrix 标定矩阵
 * eConvertType 标定方式
 * ePosSeqType 姿态顺序
 * pRobotPoint 输出的机械臂坐标
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_EyeHandPosConvert6WithBuler(VZEHPCHANDLE hConvertHandle,
    SVzNLRobotPointD* pCameraPoints,
    SVzNLRobotPointD* pRobpose,
    int nCount,
    double dCalibMatrix[16],
    EVzHandEyeConvertType eConvertType,
    EVzRPYSeqConvertType ePosSeqType,
    SVzNLRobotPointD* pRobotPoint);
```

```
/**
 * @brief R2RPY 旋转矩阵转RPY角
 * @param hConvertHandle 转换句柄
 * @param R 旋转矩阵3*3
 * @param RPY RPY角,顺序为Roll-Pitch-Yaw. 角度制
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_R2RPY(VZEHPCHANDLE hConvertHandle, double  R[9], double RPY[3], const EVzRPYSeqConvertType ePosSeqType);

/**
 * @brief R2RPY RPY角转旋转矩阵
 * @param hConvertHandle 转换句柄
 * @param R 旋转矩阵3*3
 * @param RPY RPY角,顺序为Roll-Pitch-Yaw.角度制
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_RPY2R(VZEHPCHANDLE hConvertHandle, double RPY[3], double R[9], const EVzRPYSeqConvertType ePosSeqType);
```

4.19 版本 3.6.20.50

4.19.1 添加激光器闪频接口(适用于所有相机)

```
/**
 * @brief 是否支持频闪功能
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return 返回VzTrue表示支持，否则为不支持。
 */
VZNLAPI VzBool VzNL_IsSupportStrobe(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 开启/关闭频闪功能
 * @param [in] hDevice 设备句柄
 * @param [in] bEnable VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableStrobe(VZNLHANDLE hDevice, VzBool bEnable);

/**
 * @brief 激光器是否开启频闪功能
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:开启，VzFalse:未开启。
 */
VZNLAPI VzBool VzNL_IsEnableStrobe(VZNLHANDLE hDevice, int* pnErrorCode);


/**
 * @brief 点亮激光器
 * @param [in] hDevice 设备句柄
 * @param [in] bOn VzTrue:关闭，VzFalse:点亮
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_TurnOnStrobeLaser(VZNLHANDLE hDevice, VzBool bOn);

/**
 * @brief 激光器是否已点亮
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:关闭，VzFalse:点亮。
 */
VZNLAPI VzBool VzNL_IsTurnOnStrobeLaser(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置频闪激光器点亮时长（微妙）
 * @param [in] hDevice 设备句柄
 * @param [in] nTime   时长
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetStrobeTime(VZNLHANDLE hDevice, unsigned int nTime);
VZNLAPI unsigned int VzNL_GetStrobeTime(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
/**
 * @brief 配置频闪激光器点亮延迟时长（微妙）
 * @param [in] hDevice 设备句柄
 * @param [in] nTimeOffset   时长偏移
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetStrobeTimeOffset(VZNLHANDLE hDevice, int nTimeOffset);
VZNLAPI int VzNL_GetStrobeTimeOffset(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.19.2 添加激光器 PWM 功率调节接口(适用于所有相机)

```
/**
 * @brief 是否支持激光器PWM调节功能
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:支持，VzFalse:不支持
 */
VZNLAPI VzBool VzNL_IsSupportPwmControl(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置PWM频率
 * @param [in] hDevice 设备句柄
 * @param [in] nFreq 频率
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetPwmFreq(VZNLHANDLE hDevice, unsigned int nFreq);
VZNLAPI unsigned int VzNL_GetPwmFreq(VZNLHANDLE hDevice, int* pnErrorCode);


/**
 * @brief 配置PWM占空比
 * @param [in] hDevice 设备句柄
 * @param [in] nRatio 例如: 50 ，代表占空比 50%
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetPwmRatio(VZNLHANDLE hDevice, unsigned int nRatio);
VZNLAPI unsigned int VzNL_GetPwmRatio(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.19.3 添加补光灯接口(适用于所有相机)

```
/**
 * @brief 是否支持补光灯
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return 返回VzTrue表示支持，否则为不支持。
 */
VZNLAPI VzBool VzNL_IsSupportExtLight(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 开启/关闭补光灯
 * @param [in] hDevice 设备句柄
 * @param [in] bOn VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_TurnOnExtLight(VZNLHANDLE hDevice, VzBool bOn);

/**
 * @brief 激光器是否开启补光灯
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:开启，VzFalse:未开启。
 */
VZNLAPI VzBool VzNL_IsTurnOnExtLight(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.20 版本 3.6.20.51

4.20.1 添加配置检测物体环境接口(适用于所有相机)

```
/**
 * @brief 配置检测物体环境参数
 * @param hDevice 当前设备句柄
 * @param eEnvType 环境类型
 * @param nLevel 参数等级
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_ConfigDetectEnvLevel(VZNLHANDLE hDevice, EVzEnvType eEnvType,unsigned int nLevel);
VZNLAPI SVzEnvLevel VzNL_GetConfigDetectEnvLevel(VZNLHANDLE hDevice, int* pnErrorCode);
/**
 * @brief 获取配置检测物体环境参数范围
 * @param hDevice 当前设备句柄
 * @param eEnvType 环境类型
 * @param [out] pnErrorCode 当pnErrorCode不为nullptr时,输出错误码
 * @return 参数等级范围
 */
VZNLAPI SVzNLRange VzNL_GetDetectEnvLevelRange(VZNLHANDLE hDevice, EVzEnvType eEnvType, int* pnErrorCode);
```

4.21 版本 3.6.20.52 (没有接口变化)

4.22 版本 3.6.20.53

4.22.1 添加配置抗反光增益接口(适用于所有相机)

```
/**
 * @brief 设置/获取抗反光增益
 * @param [in] hDevice 设备句柄
 * @param [out] eAntiReflectGainType 抗反光增益类型
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetAntiReflectGainType(VZNLHANDLE hDevice, EVzAntiReflectGainType eAntiReflectGainType);
VZNLAPI EVzAntiReflectGainType VzNL_GetAntiReflectGainType(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.23 版本 3.6.20.54 (没有接口变化)

4.24 版本 3.6.20.55 (没有接口变化)

4.25 版本 3.6.20.56

4.25.1 添加全局降噪功能配置接口(适用于所有相机)

```
/**
 * @brief 激光检测全局降噪功能接口
 * @param [in] hDevice    设备句柄
 * @param [in] bEnable    VzTrue为使能全局降噪，VzFalse为不使能全局降噪
 * @return 成功返回0，否则为其他错误码，返回错误可认为不支持此功能
 */
VZNLAPI int VzNL_EnableGlobalDenoise(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI int VzNL_IsEnableGlobalDenoise(VZNLHANDLE hDevice, VzBool* pbEnable);
```

4.25.2 添加 C 版激光器点亮时间占空比获取接口(适用于所有相机)

```
/**
 * @brief 配置频闪激光器亮度值(点亮时间与单帧采集时间的比例值：浮点百分比：[0,100])
 * @param [in] hDevice 设备句柄
 * @param [in] nTimeRatio: 点亮时间与单帧采集时间(1/nFrameRate)的比例值
 * @return 成功返回0，否则为错误码，VzNL_GetStrobeTimeRatio() 函数返回错误可认为当前设备不支持此功能；
 */
VZNLAPI int VzNL_SetStrobeTimeRatio(VZNLHANDLE hDevice, float nTimeRatio);
VZNLAPI float VzNL_GetStrobeTimeRatio(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.26 版本 3.6.20.57(没有接口变化)

4.27 版本 3.6.20.58(没有接口变化)

4.28 版本 3.6.20.59(没有接口变化)

4.29 版本 3.6.20.60(没有接口变化)

4.30 版本 3.6.20.61(没有接口变化)

4.31 版本 3.6.20.62

4.31.1 添加开启/关闭补光灯自动控制接口(适用于含有补光灯的相机)

```
/**
 * @brief 开启/关闭补光灯自动控制
 * @param [in] hDevice 设备句柄
 * @param [in] bEnable VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableAutoControlExtLight(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnableAutoControlExtLight(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.31.2 添加配置自动控制补光灯开启后的等待时间接口(适用于含有补光灯的相机)

```
/**
 * @brief 配置自动控制补光灯开启后的等待时间
 * @param [in] hDevice 设备句柄
 * @param [in] nWaitTime  微秒
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetAutoControlWaitTime(VZNLHANDLE hDevice, unsigned int nWaitTime);
VZNLAPI unsigned int VzNL_GetAutoControlWaitTime(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.32 版本 3.6.20.63(没有接口变化)

4.33 版本 3.6.20.64

4.33.1 添加使能 PetaLinux 通道接口(适用于支持 PetaLinux 的相机)

```
/// @brief
/// 使能PetaLinux通道
/// <param name = "hDevice">[in]设备</param>
/// <param name = "bEnable">[in]VzTrue:使能，VzFalse:不使能</param>
/// @return 成功返回0，失败返回错误码
VZNLAPI int VzNL_EnablePetaLinux(VZNLHANDLE hDevice, VzBool bEnable);
```

4.34 版本 3.6.20.65(没有接口变化)

4.35 版本 3.6.20.66(没有接口变化)

4.36 版本 3.6.20.67

4.36.1 添加启用数据研所传输的接口(适用于所有相机)

```
/**
 * @brief 激光检测是否启用数据压缩传输
 * @param [in] hDevice   设备句柄
 * @param [in] bEnable   VzTrue为使能压缩，VzFalse为不使能压缩
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_EnableCompress(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnableCompress(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.37 版本 3.6.20.68

4.37.1 添加通过 3d 点获取 RGB 图像的 2d 点坐标的接口(适用于静态 RGBD 相机)

```
/**
 * @brief  通过3d点获取RGB图像的2d点坐标
 * @param hDevice [in] 设备Handle
 * @param s3DPoint [in] 3d点
 * @param ps2DPoint [in/out] 2d点
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_CalcColor2DFrom3D(VZNLHANDLE hDevice, const SVzNL3DPoint s3DPoint, SVzNL2DPoint* ps2DPoint);
```

4.38 版本 3.6.20.69

4.38.1 添加 RGB 点云处理工具相关的接口(适用于静态 RGBD 相机)

```
/**
 * @brief 创建RGB点云处理工具
 * @param hDevice   设备句柄
 */
VZNLAPI VZNLRGBCLOUDPOINTTOOL VzNL_CreateRGBCloudPointTool(VZNLHANDLE hDevice);


/**
 * @brief 开始送入点云数据
 * @param hRGBCloudPointTool    RGB点云处理工具
 * @param eDataType             送入数据的类型,目前仅支持 keResultDataType_PointXYZRGBA 与keResultDataType_PointRGBA_D
 */
VZNLAPI int VzNL_BeginPushPointToRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool, EVzResultDataType eDataType);

/**
 * @brief 送入点云数据进行处理
 * @param hRGBCloudPointTool    RGB点云处理工具
 * @param psLineData            激光线数据
 */
VZNLAPI int VzNL_PushPointToRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool, SVzLaserLineData* psLineData);
```

```
/**
 * @brief 送入点云数据进行处理
 * @param hRGBCloudPointTool    RGB点云处理工具
 * @param psLineData            激光线数据
 */
VZNLAPI int VzNL_PushPointToRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool, SVzLaserLineData* psLineData);

/**
 * @brief 通过2D位置找3D点
 * @param hRGBCloudPointTool    RGB点云处理工具
 * @param s2DPoint              2D数据
 * @param p3DPoint              3D数据
 */
VZNLAPI int VzNL_Find3DPointFrom2DPosForRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool, const SVzNL2DPoint s2DPoint, SVzNL3DPoint* p3DPoint);


/**
 * @brief 通过3D点找2D位置
 * @param hRGBCloudPointTool    RGB点云处理工具
 * @param s3DPoint              3D数据
 * @param p2DPoint              2D数据
 */
VZNLAPI int VzNL_Find2DPosFrom3DPointForRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool, const SVzNL3DPoint s3DPoint, SVzNL2DPoint* p2DPoint);

/**
 * @brief 结束点云处理
 * @param hRGBCloudPointTool RGB点云工处理工具
 */
VZNLAPI int VzNL_EndPushPointToRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool);

/**
 * @brief 销毁RGB点云工处理工具
 * @param hRGBCloudPointTool  RGB点云工处理工具
 */
VZNLAPI int VzNL_DestroyRGBCloudPointTool(VZNLRGBCLOUDPOINTTOOL hRGBCloudPointTool);
```

4.39 版本 3.6.20.70(没有接口变化)

4.40 版本 3.6.20.71(没有接口变化)

4.41 版本 3.6.20.72(没有接口变化)

4.42 版本 3.6.20.73(没有接口变化)

4.43 版本 3.6.20.74

4.43.1 添加获取温度信息的接口(适用于所有相机)

```
/*
 * @brief 获取温度信息
 * @param hDevice [in] 设备句柄
 * @param eChipInfo [in] 芯片信息
 * @param pfMin [out] 最低温度
 * @param pfMax [out] 最高温度
 * @param pfCur [out] 当前温度
 * @return 失败返回非0
 */
VZNLAPI int VzNL_QueryChipInfo(VZNLHANDLE hDevice, EVzDevChipInfo eChipInfo, float* pfMin, float* pfMax, float* pfCur);
```

4.44 版本 3.6.20.75

4.44.1 添加指定设备 IP 列表的接口(适用于所有相机)

```
/**
 * @brief 添加设备IP列表
 * @param [in] 设备IP列表
 * @param [in] 个数
 * @return 添加成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetDeviceIpList(SVzNLDeviceIp* pDeviceList,unsigned int nCount);
```

4.45 版本 3.6.20.76(没有接口变化)

4.46 版本 3.6.20.77(没有接口变化)

4.47 版本 3.6.20.78

4.47.1 添加重启摆动模块的接口(适用于静态相机)

```
/// @brief 重启摆动模块
VZNLAPI int VzNL_RebootSwing(VZNLHANDLE hDevice);
```

4.48 版本 3.6.20.79(没有接口变化)

4.49 版本 3.6.20.80(没有接口变化)

4.50 版本 3.6.20.81

4.50.1 添加设置数据扫描长度,当扫描达到指定长度后，自动停止扫描接口(适用于所有相机)

```
/**
 * @brief 设置数据扫描长度,当扫描达到指定长度后，自动停止扫描。
 * @param [in] hDevice          设备句柄
 * @param [in] nDistance        扫描长度,单位毫米(mm)
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetLaserCaptureDistance(VZNLHANDLE hDevice, double dDistance);
VZNLAPI int VzNL_GetLaserCaptureDistance(VZNLHANDLE hDevice, double* dnDistance);
```

4.51 版本 3.6.20.82

4.51.1 添加设置扫描噪点过滤阈值的接口(适用于所有相机)

```
/*
* @brief 设置/获取扫描噪点过滤阈值
* @param name="hDevice"[in]设备handle</param>
* @param name="nDebounce"[in]噪点过滤阈值,取值范围0-255,0:关闭此功能</param>
* @return 返回错误码
*/
VZNLAPI int VzNL_SetScanDenoiseTh(VZNLHANDLE hDevice, unsigned int nValue);
VZNLAPI int VzNL_GetScanDenoiseTh(VZNLHANDLE hDevice, unsigned int* pnValue);
```

4.52 版本 3.6.20.83(没有接口变化)

4.53 版本 3.6.20.84(没有接口变化)

4.53.1 添加启用无限制帧率的接口(适用于所有相机)

```
/**
* @brief 启用无限制帧率
* @param hDevice 当前设备句柄
* @param [in] bEnable 是否启用
* @return 成功返回0，否则为错误码。
*/
VZNLAPI int VzNL_EnableFreeFrameRate(VZNLHANDLE hDevice, VzBool bEnable);
```

4.54 版本 3.6.20.85(没有接口变化)

4.55 版本 3.6.20.86

4.55.1 添加点云数据的缓存相关接口(适用于所有相机)

```
/**
* @brief 使能/禁止点云数据缓存功能
* @param [in] bEnable 使能/禁用，使能后扫描点云过程中会将数据缓存到本地，可进行后续的2D点到3D点查找
* @return 返回0表示正确
* @retval 0 表示成功
* @retval 非0 表示失败,可以使用VzNL_GetErrorInfo获取
*/
VZNLAPI int VzNL_EnablePointCloudCache(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnablePointCloudCache(VZNLHANDLE hDevice, int* pnErrorCode);

/**
* @brief 查找输入2D点对应的3D 点坐标，需使能点云数据缓存功能，并完成扫描。
* @param [in] s2DPos: 输入从图上选中的2D点坐标，RGBD扫描：RGB图像的坐标；双目RGBD扫描，左目图像坐标；
* @param [in] p3DPoint: 输出匹配的3D 点坐标；
* @return 返回0表示正确
* @retval 0 表示成功
* @retval 非0 表示失败,可以使用VzNL_GetErrorInfo获取
*/
VZNLAPI int VzNL_GetCached3DPointBy2DPos(VZNLHANDLE hDevice, SVzNL2DPoint s2DPos, SVzNL3DPoint* p3DPoint);
```

4.56 版本 3.6.20.87(没有接口变化)

4.57 版本 3.6.20.88(没有接口变化)

4.58 版本 3.6.20.89(没有接口变化)

4.59 版本 3.6.20.90

4.59.1 添加获取双目固定视差接口(适用于所有相机)

```
/** @brief 获取双目固定视差
*  @param[in] hDevice           设备Handle
*  @param[out] pFixDisparity    返回固定视差
*/
VZNLAPI int VzNL_GetFixDisparity(VZNLHANDLE hDevice, int* pFixDisparity);
```

4.59.2 添加通过双目校正图 2D 坐标计算 3D 点坐标接口(适用于所有相机)

```
/**
* @brief 通过双目校正图2D 坐标计算3D 点坐标，根据当前设备内部参数计算
* @param hDevice        输入设备句柄
* @param nPointCnt      输入点个数
* @param p2DPointList   输入双目校正图2D 点坐标列表
* @param p3DPointList   输出计算得到的3D 点列表
*/
VZNLAPI int VzNL_Cal3DPointsFromStereo2DPoints(VZNLHANDLE hDevice, unsigned int nPointCnt, SVzNL2DLRPoint* p2DPointList, SVzNL3DPoint* p3DPointList);
```

4.59.3 移除 C 版功能相关控制接口(适用于所有相机)

```
/**
 * @brief 是否支持C版功能
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return 返回VzTrue表示支持，否则为不支持。
 */
VZNLAPI VzBool VzNL_IsSupportStrobe(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 开启/关闭C版功能
 * @param [in] hDevice 设备句柄
 * @param [in] bEnable VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableStrobe(VZNLHANDLE hDevice, VzBool bEnable);

/**
 * @brief 激光器是否开启C版功能
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:开启，VzFalse:未开启。
 */
VZNLAPI VzBool VzNL_IsEnableStrobe(VZNLHANDLE hDevice, int* pnErrorCode);


/**
 * @brief 点亮激光器
 * @param [in] hDevice 设备句柄
 * @param [in] bOn VzTrue:点亮，VzFalse:关闭
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_TurnOnStrobeLaser(VZNLHANDLE hDevice, VzBool bOn);
```

```
/**
 * @brief 激光器是否已点亮
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return  VzTrue:点亮，VzFalse:关闭
 */
VZNLAPI VzBool VzNL_IsTurnOnStrobeLaser(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置C版激光器点亮时长（微妙）
 * @param [in] hDevice 设备句柄
 * @param [in] nTime   时长
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetStrobeTime(VZNLHANDLE hDevice, unsigned int nTime);
VZNLAPI unsigned int VzNL_GetStrobeTime(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置C版激光器点亮延迟时长（微妙）
 * @param [in] hDevice 设备句柄
 * @param [in] nTimeOffset   时长偏移
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetStrobeTimeOffset(VZNLHANDLE hDevice, int nTimeOffset);
VZNLAPI int VzNL_GetStrobeTimeOffset(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置C版激光器亮度值(点亮时间与单帧采集时间的比例值：浮点百分比：[0,100])
 * @param [in] hDevice 设备句柄
 * @param [in] nTimeRatio: 点亮时间与单帧采集时间(1/nFrameRate)的比例值
 * @return 成功返回0，否则为错误码，VzNL_GetStrobeTimeRatio() 函数返回错误可认为当前设备不支持此功能；
 */
VZNLAPI int VzNL_SetStrobeTimeRatio(VZNLHANDLE hDevice, float nTimeRatio);
VZNLAPI float VzNL_GetStrobeTimeRatio(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
/**
 * @brief 是否支持激光器PWM调节功能
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:支持，VzFalse:不支持
 */
VZNLAPI VzBool VzNL_IsSupportPwmControl(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置PWM频率
 * @param [in] hDevice 设备句柄
 * @param [in] nFreq 频率
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetPwmFreq(VZNLHANDLE hDevice, unsigned int nFreq);
VZNLAPI unsigned int VzNL_GetPwmFreq(VZNLHANDLE hDevice, int* pnErrorCode);


/**
 * @brief 配置PWM占空比
 * @param [in] hDevice 设备句柄
 * @param [in] nRatio 占空比 例如: 50 ，代表占空比 50%
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetPwmRatio(VZNLHANDLE hDevice, unsigned int nRatio);
VZNLAPI unsigned int VzNL_GetPwmRatio(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 是否支持补光灯
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return 返回VzTrue表示支持，否则为不支持。
 */
VZNLAPI VzBool VzNL_IsSupportExtLight(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
/**
 * @brief 开启/关闭补光灯
 * @param [in] hDevice 设备句柄
 * @param [in] bOn VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_TurnOnExtLight(VZNLHANDLE hDevice, VzBool bOn);

/**
 * @brief 是否开启了补光灯
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:开启，VzFalse:未开启。
 */
VZNLAPI VzBool VzNL_IsTurnOnExtLight(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 设置/获取抗反光增益
 * @param [in] hDevice 设备句柄
 * @param [out] eAntiReflectGainType 抗反光增益类型
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetAntiReflectGainType(VZNLHANDLE hDevice, EVzAntiReflectGainType eAntiReflectGainType);
VZNLAPI EVzAntiReflectGainType VzNL_GetAntiReflectGainType(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 开启/关闭补光灯自动控制
 * @param [in] hDevice 设备句柄
 * @param [in] bEnable VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableAutoControlExtLight(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnableAutoControlExtLight(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
/**
 * @brief 配置自动控制补光灯开启后的等待时间
 * @param [in] hDevice 设备句柄
 * @param [in] nWaitTime  微秒
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetAutoControlWaitTime(VZNLHANDLE hDevice, unsigned int nWaitTime);
VZNLAPI unsigned int VzNL_GetAutoControlWaitTime(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.60 版本 3.6.20.91(没有接口变化)

4.61 版本 3.6.20.92

4.61.1 添加使能双目/RGB Map 本地缓存/加载功能，提升设备连接速度的接口(适用于所有相机)

```
/**
 * @brief 使能双目/RGB Map本地缓存/加载功能，提升设备连接速度
 * @param hDevice 当前设备句柄
 * @param [in] bEnable 是否启用
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableLocalMapDataLoad(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnableLocalMapDataLoad(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.62 版本 3.6.20.93(没有接口变化)

4.63 版本 3.6.20.94

4.63.1 添加 C 版激光相关接口(适用于所有相机)

```
/**
 * @brief 是否支持C版功能
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return 返回VzTrue表示支持，否则为不支持。
 */
VZNLAPI VzBool VzNL_IsSupportStrobe(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 开启/关闭C版功能
 * @param [in] hDevice 设备句柄
 * @param [in] bEnable VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableStrobe(VZNLHANDLE hDevice, VzBool bEnable);

/**
 * @brief 激光器是否开启C版功能
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:开启，VzFalse:未开启。
 */
VZNLAPI VzBool VzNL_IsEnableStrobe(VZNLHANDLE hDevice, int* pnErrorCode);


/**
 * @brief 点亮激光器
 * @param [in] hDevice 设备句柄
 * @param [in] bOn VzTrue:点亮，VzFalse:关闭
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_TurnOnStrobeLaser(VZNLHANDLE hDevice, VzBool bOn);

/**
 * @brief 激光器是否已点亮
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return  VzTrue:点亮，VzFalse:关闭
 */
VZNLAPI VzBool VzNL_IsTurnOnStrobeLaser(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
/**
 * @brief 激光器是否已点亮
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return  VzTrue:点亮, VzFalse:关闭
 */
VZNLAPI VzBool VzNL_IsTurnOnStrobeLaser(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置C版激光器点亮时长（微妙）
 * @param [in] hDevice 设备句柄
 * @param [in] nTime   时长
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetStrobeTime(VZNLHANDLE hDevice, unsigned int nTime);
VZNLAPI unsigned int VzNL_GetStrobeTime(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置C版激光器点亮延迟时长（微妙）
 * @param [in] hDevice 设备句柄
 * @param [in] nTimeOffset   时长偏移
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetStrobeTimeOffset(VZNLHANDLE hDevice, int nTimeOffset);
VZNLAPI int VzNL_GetStrobeTimeOffset(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置C版激光器亮度值(点亮时间与单帧采集时间的比例值：浮点百分比：[0,100])
 * @param [in] hDevice 设备句柄
 * @param [in] nTimeRatio: 点亮时间与单帧采集时间(1/nFrameRate)的比例值
 * @return 成功返回0，否则为错误码，VzNL_GetStrobeTimeRatio() 函数返回错误可认为当前设备不支持此功能；
 */
VZNLAPI int VzNL_SetStrobeTimeRatio(VZNLHANDLE hDevice, float nTimeRatio);
VZNLAPI float VzNL_GetStrobeTimeRatio(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 是否支持激光器PWM调节功能
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:支持, VzFalse:不支持
 */
VZNLAPI VzBool VzNL_IsSupportPwmControl(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
/**
 * @brief 配置PWM频率
 * @param [in] hDevice 设备句柄
 * @param [in] nFreq 频率
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetPwmFreq(VZNLHANDLE hDevice, unsigned int nFreq);
VZNLAPI unsigned int VzNL_GetPwmFreq(VZNLHANDLE hDevice, int* pnErrorCode);


/**
 * @brief 配置PWM占空比
 * @param [in] hDevice 设备句柄
 * @param [in] nRatio 占空比 例如：50 ，代表占空比 50%
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetPwmRatio(VZNLHANDLE hDevice, unsigned int nRatio);
VZNLAPI unsigned int VzNL_GetPwmRatio(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 是否支持补光灯
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return 返回VzTrue表示支持，否则为不支持。
 */
VZNLAPI VzBool VzNL_IsSupportExtLight(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 开启/关闭补光灯
 * @param [in] hDevice 设备句柄
 * @param [in] bOn VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_TurnOnExtLight(VZNLHANDLE hDevice, VzBool bOn);

/**
 * @brief 是否开启了补光灯
 * @param [in] hDevice 设备句柄
 * @param [out] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:开启，VzFalse:未开启。
 */
VZNLAPI VzBool VzNL_IsTurnOnExtLight(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
/**
 * @brief 设置/获取抗反光增益
 * @param [in] hDevice 设备句柄
 * @param [out] eAntiReflectGainType 抗反光增益类型
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetAntiReflectGainType(VZNLHANDLE hDevice, EVzAntiReflectGainType eAntiReflectGainType);
VZNLAPI EVzAntiReflectGainType VzNL_GetAntiReflectGainType(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 开启/关闭补光灯自动控制
 * @param [in] hDevice 设备句柄
 * @param [in] bEnable VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableAutoControlExtLight(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnableAutoControlExtLight(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 配置自动控制补光灯开启后的等待时间
 * @param [in] hDevice 设备句柄
 * @param [in] nWaitTime  微秒
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetAutoControlWaitTime(VZNLHANDLE hDevice, unsigned int nWaitTime);
VZNLAPI unsigned int VzNL_GetAutoControlWaitTime(VZNLHANDLE hDevice, int* pnErrorCode);


/**
 * @brief 是否支持ch3功能
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return VzTrue:支持，VzFalse:不支持
 */
VZNLAPI VzBool VzNL_IsSupportCh3(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 开启/关闭ch3功能
 * @param [in] hDevice 设备句柄
 * @param [in] bEnable VzTrue:开启，VzFalse:不开启
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableCh3(VZNLHANDLE hDevice, VzBool bEnable);
```

```
/**
* @brief 激光器是否ch3功能
* @param [in] hDevice 设备句柄
* @param [out] pnErrorCode 错误码,为nullptr时不返回
* @return VzTrue:开启，VzFalse:未开启。
*/
VZNLAPI VzBool VzNL_IsEnableCh3(VZNLHANDLE hDevice, int* pnErrorCode);

/**
* @brief 配置ch3频率
* @param [in] hDevice 设备句柄
* @param [in] nFreq 频率 10-50k
* @return 成功返回0，否则为错误码。
*/
VZNLAPI int VzNL_SetCh3Freq(VZNLHANDLE hDevice, unsigned int nFreq);
VZNLAPI unsigned int VzNL_GetCh3Freq(VZNLHANDLE hDevice, int* pnErrorCode);


/**
* @brief 配置ch3占空比
* @param [in] hDevice 设备句柄
* @param [in] nRatio 占空比 例如：50 ，代表占空比 50%
* @return 成功返回0，否则为错误码。
*/
VZNLAPI int VzNL_SetCh3Ratio(VZNLHANDLE hDevice, unsigned int nRatio);
VZNLAPI unsigned int VzNL_GetCh3Ratio(VZNLHANDLE hDevice, int* pnErrorCode);


/**
* @brief 配置C版激光自动关闭的时间
* @param [in] hDevice 设备句柄
* @param [in] nTime 时间，单位分钟，默认1分钟
* @return 成功返回0，否则为错误码。
*/
VZNLAPI int VzNL_SetAutoTurnOffStrobeLaserTime(VZNLHANDLE hDevice, unsigned int nTime);
VZNLAPI unsigned int VzNL_GetAutoTurnOffStrobeLaserTime(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.64 版本 3.6.20.95

4.64.1 添加获取映射后的 2D 图像数据和同分辨率的点云序列相关接口(适用于所有相机)

```
/**
* @brief 输入2D RGB图像数据 + 扫描点云数据(静态RGBD扫描)，获取映射后的2D RGB图像数据和同分辨率的点云序列
* @param hDevice        输入设备句柄
* @param pRGBImage      输入与点云匹配的RGB 图像
* @param eDataType      输入点云数据类型，同样也为输出的点云数据类型
* @param nLineCnt       输入激光线数据条数
* @param pLineDataList  输入激光线数据列表
* @param bEnableClip    输入是否进行有效图像裁剪，true：输出点云实际对应的有效图像区域及映射点云；false：输出原始图像及映射点云；
* @param ppOutImage     输出处理后的图像(参考 bEnableClip 标志决定是否裁剪图像)
* @param pPointCnt      输出点云数据中的点个数
* @param ppMappedPointList  输出映射后与ppOutImage 相同分辨率的点云数据，点数据类型为eDataType
*/
VZNLAPI int VzNL_GetMappedRGBImageAndPointCloud(VZNLHANDLE hDevice,
                                                SVzNLImageData* pRGBImage,
                                                EVzResultDataType eDataType,
                                                unsigned int nLineCnt,
                                                SVzLaserLineData* pLineDataList,
                                                bool bEnableClip,
                                                SVzNLImageData** ppOutImage,
                                                unsigned int* pPointCnt,
                                                void** ppMappedPointList);
```

```
/**
 * @brief 输入左目或右目图像数据 + 扫描点云数据(静态非RGBD扫描)，获取映射后的2D 左目或右目图像数据和同分辨率的点云序列
 * @param hDevice          输入设备句柄
 * @param bIsRightImage    输入当前图像是否为右目图像
 * @param pImage           输入与点云匹配的左目或右目图像数据
 * @param eDataType        输入点云数据类型，同样也为输出的点云数据类型
 * @param nLineCnt         输入激光线数据条数
 * @param pLineDataList    输入激光线数据列表
 * @param bEnableClip      输入是否进行有效图像裁剪，true：输出点云实际对应的有效图像区域及映射点云；false：输出原始图像及映射点云；
 * @param ppOutImage       输出处理后的图像(参考 bEnableClip 标志决定是否裁剪图像)
 * @param pPointCnt        输出点云数据中的点个数
 * @param ppMappedPointList  输出映射后与ppOutImage 相同分辨率的点云数据，点数据类型为eDataType
 */
VZNLAPI int VzNL_GetMappedLRImageAndPointCloud(VZNLHANDLE hDevice,
                                               bool bIsRightImage,
                                               SVzNLImageData* pImage,
                                               EVzResultDataType eDataType,
                                               unsigned int nLineCnt,
                                               SVzLaserLineData* pLineDataList,
                                               bool bEnableClip,
                                               SVzNLImageData** ppOutImage,
                                               unsigned int* pPointCnt,
                                               void** ppMappedPointList);
```

```
/**
 * @brief 释放VzNL_GetMappedRGBImageAndPointCloud()/VzNL_GetMappedLRImageAndPointCloud() 函数返回的图像和3D 点数据
 */
VZNLAPI int VzNL_ReleaseImageAndPointCloudMappedResult(SVzNLImageData** ppImage, void** ppPointList);
```

4.65 版本 3.6.20.96

4.65.1 添加获取点云平滑相关接口(适用于所有相机)

```
/**
 * @brief 配置平滑门限
 * @param hDevice 当前设备句柄
 * @param nValue 门限值
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetSmoothingTh(VZNLHANDLE hDevice, unsigned int nValue);
VZNLAPI int VzNL_GetSmoothingTh(VZNLHANDLE hDevice, unsigned int* pnValue);

/**
 * @brief 配置激光线检测模式
 * @param hDevice 当前设备句柄
 * @param nValue 检测模式值
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetMLineMode(VZNLHANDLE hDevice, unsigned int nValue);
VZNLAPI int VzNL_GetMLineMode(VZNLHANDLE hDevice, unsigned int* pnValue);
```

4.66 版本 3.6.20.97

4.66.1 添加执行静态扫描点云拍照(阻塞执行)，输出完整扫描点云，如果为 RGBD 扫描则同步返回 RGB 图像相关接口(适用于所有相机)

```
/**
 * @brief 配置平滑门限
 * @param hDevice 当前设备句柄
 * @param nValue 门限值
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetSmoothingTh(VZNLHANDLE hDevice, unsigned int nValue);
VZNLAPI int VzNL_GetSmoothingTh(VZNLHANDLE hDevice, unsigned int* pnValue);

/**
 * @brief 配置激光线检测模式
 * @param hDevice 当前设备句柄
 * @param nValue 检测模式值
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetMLineMode(VZNLHANDLE hDevice, unsigned int nValue);
VZNLAPI int VzNL_GetMLineMode(VZNLHANDLE hDevice, unsigned int* pnValue);
```

4.66.2 添加 RGB HDR 功能相关接口(适用于所有相机)

```
/**
 * @brief 是否支持RGB HDR功能
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return 返回VzTrue表示支持，否则为不支持。
 */
VZNLAPI VzBool VzNL_IsSupportRGBHDR(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 使能彩色sensor hdr功能
 * @param hDevice 当前设备句柄
 * @param bEnable VzTrue启用 / VzFalse禁用
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_EnableRGBHDR(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnableRGBHDR(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 设置彩色Sensor hdr 高曝光
 * @param hDevice 当前设备句柄
 * @param nValue 曝光值[20~100000]
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_SetRGBHDRHighExpose(VZNLHANDLE hDevice, unsigned int nValue);
VZNLAPI unsigned int VzNL_GetRGBHDRHighExpose(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 设置彩色Sensor hdr 低曝光
 * @param hDevice 当前设备句柄
 * @param nValue 曝光值[20~100000]
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_SetRGBHDRLowExpose(VZNLHANDLE hDevice, unsigned int nValue);
VZNLAPI unsigned int VzNL_GetRGBHDRLowExpose(VZNLHANDLE hDevice, int* pnErrorCode);
```

```
/*
 * @brief 开始实时采集RGB图像
 * @param [in] hDevice       设备Handle
 * @param [in] pImageCB      图像回调接口
 * @param [in] pCBParam      回调接口参数
 * @return 调用接口成功返回0
 */
VZNLAPI int VzNL_StartRGBCapture(VZNLHANDLE hDevice, VzNL_OutputRGBImageCB pImageCB, void* pCBParam);

/**
 * @brief 停止实时采集RGB图像
 * @param [in] hDevice       设备Handle
 * @return 调用接口成功返回0
 */
VZNLAPI int VzNL_StopRGBCapture(VZNLHANDLE hDevice);
```

```
/**
 * @brief 设置熔池检测区域
 * @param [in] hDevice 设备句柄
 * @param [in] pROIArea RGB ROI区域
 * @return 返回错误码
 */
VZNLAPI int VzNL_ConfigMoltenPoolROI(VZNLHANDLE hDevice, const SVzNLROIRect* pROIArea);
VZNLAPI int VzNL_GetConfigMoltenPoolROI(VZNLHANDLE hDevice, SVzNLROIRect* pROIArea);

/**
 * @brief 设置RGBHDR 背景图叠加帧数
 * @param hDevice 当前设备句柄
 * @param nFrames 背景图叠加帧数：[0~100]，设置为0和1 效果一样，都表示不用叠加
 * @return 正确返回0， 失败返回返回错误码
 */
VZNLAPI int VzNL_SetRGBHDRBGOverlayFrames(VZNLHANDLE hDevice, unsigned int nFrames);
VZNLAPI unsigned int VzNL_GetRGBHDRBGOverlayFrames(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 设置RGBHDR 背景图叠加方法
 * @param hDevice 当前设备句柄
 * @param eMethod 背景图叠加方法
 * @return 正确返回0， 失败返回返回错误码
 */
VZNLAPI int VzNL_SetRGBHDRBGOverlayMethod(VZNLHANDLE hDevice, EVzRGBHDRBGOverlayMethod eMethod);
VZNLAPI EVzRGBHDRBGOverlayMethod VzNL_GetRGBHDRBGOverlayMethod(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 设置RGBHDR 出图模式
 * @param hDevice 当前设备句柄
 * @param nMode 输出图像模式：原图，HDR图，背景图，熔池图
 * @return 正确返回0， 失败返回返回错误码
 */
VZNLAPI int VzNL_SetRGBHDRImageMode(VZNLHANDLE hDevice, EVzRGBHDROutImageMode eMode);
VZNLAPI EVzRGBHDROutImageMode VzNL_GetRGBHDRImageMode(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.67 版本 3.6.20.98

4.67.1 调整点云到伪彩图生成接口，增加转换信息输出功能接口(适用于所有相机)

```
/*
* 功能：将输入的点云数据转换为指定分辨率的 2D 伪彩图像输出
* 参数说明：
*   EVzResultDataType eDataType：输入点数据类型，内部根据数据类型解析数据；
*   unsigned int nPointCnt：输入 pPointList 中点数据个数；
*   void* pPointList：输入点数据列表。
*   unsigned int nImageWidth：输入希望输出的图像宽度；
*   unsigned int nImageHeight：输入希望输出的图像高度；
*   unsigned int nColorGradient：输出点数据颜色梯度
*   unsigned int nEdgeMargin：输入图像边缘的留白宽度(像素值，四边留白)；
*   SVzNLImageData* pImage：输出的图像数据；
*   SVzPColorImageConvertInfo* pConvertInfo：转换信息，用于从图像回算到点云；
*/
VZNLAPI int VzNL_PointCloudTo2DFixedSizeImage(EVzResultDataType eDataType,
    unsigned int nPointCnt,
    void* pPointList,
    unsigned int nImageWidth,
    unsigned int nImageHeight,
    unsigned int nColorGradient,
    unsigned int nEdgeMargin,
    SVzNLImageData* pImage,
    SVzPColorImageConvertInfo* pConvertInfo);

/*
* 功能：将输入的点云数据转换为指定像素精度的 2D 伪彩图像输出
* 参数说明：
*   EVzResultDataType eDataType：输入点数据类型，内部根据数据类型解析数据；
*   unsigned int nPointCnt：输入 pPointList 中点数据个数；
*   void* pPointList：输入点数据列表。
*   float fWHPixelResolution：输入图像像素精度，代表每个像素对应的实际物理距离，例如：0.1 表示像素间隔为0.1毫米；
*   unsigned int nColorGradient：输出点数据颜色梯度
*   unsigned int nEdgeMargin：输入图像边缘的留白宽度(像素值，四边留白)；
*   SVzNLImageData* pImage：输出的图像数据；
*   SVzPColorImageConvertInfo* pConvertInfo：转换信息，用于从图像回算到点云；
*/
VZNLAPI int VzNL_PointCloudTo2DFixedResolutionImage(EVzResultDataType eDataType,
    unsigned int nPointCnt,
    void* pPointList,
    float fWHPixelResolution,
    unsigned int nColorGradient,
    unsigned int nEdgeMargin,
    SVzNLImageData* pImage,
    SVzPColorImageConvertInfo* pConvertInfo);
```

4.67.2 增加伪彩图上的点到 3D 点云的回算接口(适用于所有相机)

```
/*
* 功能：基于伪彩图像上的点坐标回算3D点云点坐标(输出点云中距离最近的点坐标)
* 参数说明:
*   SVzNLImageData* pImage: 输入由点云生成的伪彩图像；
*   EVzResultDataType eDataType: 输入点数据类型，内部根据数据类型解析数据；
*   unsigned int nPointCntIn: 输入 pPointList 中点数据个数；
*   void* pPointListIn: 输入点数据列表。
*   SVzPColorImageConvertInfo* pConvertInfo: 转换信息，生成伪彩图时的信息
*   SVzPColorImageTo3DPointMethod eConvertMethod: 回算点输出方法
*   unsigned int n2DPointCntIn: 输入伪彩图像上的2D点坐标个数
*   SVzNL2DPoint* p2DPointListIn: 输入伪彩图上的2D 点坐标列表
*   double dSearchDis: 从点云中搜索目标点的搜索范围
*   SVzNL3DPoint* p3DPointList: 输出2D坐标列表回算到3D 点的坐标列表，外部分配好空间
*/
VZNLAPI int VzNL_Get3DPointsFrom2DPseudoColorImagePoints(SVzNLImageData* pImage,
                                        EVzResultDataType eDataType,
                                        unsigned int nPointCntIn,
                                        void* pPointListIn,
                                        SVzPColorImageConvertInfo* pConvertInfo,
                                        SVzPColorImageTo3DPointMethod eConvertMethod,
                                        unsigned int n2DPointCntIn,
                                        SVzNL2DPoint* p2DPointListIn,
                                        double dSearchDis,
                                        SVzNL3DPoint* p3DPointList);

/*
* 功能：基于伪彩图像上的ROI区域获取对应区域的点云数据（点数据类型与输入一致）
* 参数说明:
*   SVzNLImageData* pImage: 输入由点云生成的伪彩图像；
*   EVzResultDataType eDataType: 输入点数据类型，内部根据数据类型解析数据；
*   unsigned int nPointCntIn: 输入 pPointList 中点数据个数；
*   void* pPointListIn: 输入点数据列表。
*   SVzPColorImageConvertInfo* pConvertInfo: 转换信息，生成伪彩图时的信息
*   SVzNL2DRect s2DRect: 输入伪彩图上的目标区域；
*   unsigned int* pPointCntOut: 输出目标区域内的3D 点云数据点个数；
*   void** ppPointListOut: 输出目标区域内的3D 点云数据列表，点数据类型与输入一致
*/
VZNLAPI int VzNL_Get3DPointListFrom2DPseudoColorImageRect(SVzNLImageData* pImage,
                                        EVzResultDataType eDataType,
                                        unsigned int nPointCntIn,
                                        void* pPointListIn,
                                        SVzPColorImageConvertInfo* pConvertInfo,
                                        SVzNL2DRect s2DRect,
                                        unsigned int* pPointCntOut,
                                        void** ppPointListOut);
```

4.68 版本 3.6.20.99

4.68.1 添加配置实时采图模式相关接口(适用于所有相机)

```
/**
 * @brief 配置实时采图的模式
 * @param hDevice 当前设备句柄
 * @param eCaptureType 采图模式
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_SetCaptureMode(VZNLHANDLE hDevice, EVzCaptureMode eCaptureMode);
VZNLAPI EVzCaptureMode VzNL_GetCaptureMode(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 是否支持设置采图模式
 * @param hDevice 当前设备句柄
 * @param [out] pnErrorCode 当pnErrorCode不为nullptr时,输出错误码
 * @return 返回VzTrue,表示支持,返回VzFalse表示不支持
 */
VZNLAPI VzBool VzNL_IsSupportCaptureMode(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.69 版本 3.6.20.100(没有接口变化)

4.70 版本 3.6.20.101

4.70.1 添加对焊缝跟踪点云扫描相关接口(适用于静态相机)

```
/**
 * 焊缝跟踪参数操作建议
 * 1、配置信息时：先将相机当前参数调试到满足要求，然后调用：VzNL_SetTrackingScanPara(hDevice, nullptr)；记录当前配置参数为焊缝跟踪参数；
 * 2、获取信息时：VzNL_GetTrackingScanPara(hdevice, pPara) 只用于查看焊缝跟踪参数值信息(可配参数+固定参数)，并不会更改当前相机参数。
 * 3、想要查看焊缝跟踪参数效果：直接调用VzNL_SetToTrackingScanMode(hDevice, pPara)配置相机参数后执行扫描即可；函数内主动获取焊缝跟踪并进行配置，同时项pPara 返回实际的配置信息；
 * 4、VzNL_SetToTrackingScanMode() 后使用返回的pPara 进行软件端的参数更新显示。
 */
/**
 * @brief 设置焊缝跟踪功能需要的相机配置参数信息（只保存到相机中，不做配置）
 * @param [in] hDevice       设备句柄
 * @param [in] pPara         输入焊缝跟踪功能配置参数：(pPara== nullptr)：函数内部自动获取当前相机配置参数作为焊缝跟踪配置参数进行保存；
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetTrackingScanPara(VZNLHANDLE hDevice, SVzTrackingScanPara* pPara);

/**
 * @brief 获取焊缝跟踪功能的相机配置参数信息，只获取信息，不会进行配置
 * @param [in] hDevice       设备句柄
 * @param [out] pPara        返回焊缝跟踪功能需要的配置参数；
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_GetTrackingScanPara(VZNLHANDLE hDevice, SVzTrackingScanPara* pPara);

/**
 * @brief 配置当前相机为焊缝跟踪状态：配置焊缝跟踪参数 + 配置固定的工作模式(固定参数部分)
 * @detail 主动读取SDK内存储的配置参数，并使用焊缝跟踪参数配置相机，同时设置相机固定的工作模式，返回当前实际的配置参数
 * @param [in] hDevice    设备句柄
 * @param [out] pPara     (pPara!= nullptr)：返回实际的焊缝跟踪配置参数，外部可用于更新本地参数及软件显示；(pPara== nullptr)：不返回参数；
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetToTrackingScanMode(VZNLHANDLE hDevice, SVzTrackingScanPara* pPara);

/**
 * @brief 恢复当前相机为焊缝跟踪前的初始状态：按备份的参数恢复相机状态
 * @detail 主动读取SDK内存储的备份参数，并使用备份参数配置相机
 * @param [in] hDevice    设备句柄
 * @param [out] pPara     (pPara!= nullptr)：返回操作后实际的配置参数，外部可用于更新本地参数及软件显示；(pPara== nullptr)：不返回参数；
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_RestoreToNonTrackingScanMode(VZNLHANDLE hDevice, SVzTrackingScanPara* pPara);
```

```
/**
 * @brief 执行焊缝跟踪点云扫描
 * @detail 保存当前工作模式及配置参数->设置相机为焊缝跟踪模式(预先设置的焊缝跟踪参数)->开始点云扫描
 * @param [in] hDevice       设备句柄
 * @param [in] sPara         焊缝跟踪功能需要的可配参数，保存到相机中。
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_StartAutoDetectForTracking(VZNLHANDLE hDevice, EVzResultDataType eResultType, EVzFlipType eFlipType, VzNL_AutoOutputLaserLineExCB pCB, void* pCBParam);

/**
 * @brief 停止焊缝跟踪点云扫描
 * @detail 停止点云扫描->恢复相机参数及工作模式
 * @param [in] hDevice       设备句柄
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_StopAutoDetectForTracking(VZNLHANDLE hDevice);
```

4.71 版本 3.6.20.102

4.71.1 移除设置数据扫描长度,当扫描达到指定长度后，自动停止扫描相关接口(适用于所以相机)

```
/**
 * @brief 设置数据采集间隔
 * @param [in] hDevice          设备句柄
 * @param [in] nCaptureInterval 采集间隔
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetLaserCaptureInterval(VZNLHANDLE hDevice, unsigned int nCaptureInterval);
VZNLAPI int VzNL_GetLaserCaptureInterval(VZNLHANDLE hDevice, unsigned int* pnCaptureInterval);
```

4.71.2 移除设置数据采集间隔相关接口(适用于所有相机)

```
/**
 * @brief 设置数据扫描长度,当扫描达到指定长度后，自动停止扫描。
 * @param [in] hDevice          设备句柄
 * @param [in] nDistance        扫描长度,单位毫米(mm)
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetLaserCaptureDistance(VZNLHANDLE hDevice, double dDistance);
VZNLAPI int VzNL_GetLaserCaptureDistance(VZNLHANDLE hDevice, double* dnDistance);
```

4.72 版本 3.6.20.103 (没有接口变化)

4.73 版本 3.6.20.104

4.73.1 修改设置焊缝跟踪功能需要的相机配置参数信息接口(适用于静态相机)

```
/**
 * 焊缝跟踪参数操作建议
 * 1、配置信息时：先将相机当前参数调试到满足要求，然后调用：VzNL_SetTrackingScanPara(hDevice, nullptr)：记录当前配置参数为焊缝跟踪参数；
 * 2、获取信息时：VzNL_GetTrackingScanPara(hdevice, pPara) 只用于查看焊缝跟踪参数值信息(可配参数+固定参数)，并不会更改当前相机参数。
 * 3、想要查看焊缝跟踪参数效果：直接调用VzNL_SetToTrackingScanMode(hDevice, pPara)配置相机参数后执行扫描即可：函数内主动获取焊缝跟踪并进行配置，同时项pPara 返回实际的配置信息；
 * 4、VzNL_SetToTrackingScanMode() 后使用返回的pPara 进行软件端的参数更新显示。
 */
/**
 * @brief 设置焊缝跟踪功能需要的相机配置参数信息（只保存到相机中，不做配置）
 * @param [in] hDevice       设备句柄
 * @param [in] pPara         输入焊缝跟踪功能配置参数：(pPara== nullptr)：函数内部自动获取当前相机配置参数作为焊缝跟踪配置参数进行保存；
 * @param [in] nOutputFrameRate  输入输出帧率参数：(pPara== nullptr)：使用此参数作为输出帧率；(pPara !=nullptr)：使用pPara 中的参数作为输出帧率；
 * @description: 参数限制：帧率大于等于1000，禁用图像校正，ROI尺寸固定为：64*256；输出帧率小于采集帧率；
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetTrackingScanPara(VZNLHANDLE hDevice, SVzTrackingScanPara* pPara, unsigned int nOutputFrameRate);
```

4.74 版本 3.6.20.105 (没有接口变化)

4.75 版本 3.6.20.106

4.75.1 添加查找摆动机构的接口(适用于静态相机)

```
/**
 * @brief 查找摆动机构模块
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode  错误码
 * @return 找到摆动机构返回VzTrue，否则返回VzFalse
 */
VZNLAPI VzBool VzNL_FindSwing(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.76 版本 3.6.20.107 (没有接口变化)

4.77 版本 3.6.20.108 (没有接口变化)

4.78 版本 3.6.20.109 (没有接口变化)

4.79 版本 3.6.20.110 (没有接口变化)

4.80 版本 3.6.20.111

4.80.1 添加获取相机当前时间戳的接口(适用于所有相机)

```
/**
 * @brief 获取相机当前时间戳(单位 微妙)
 * @param [in] hDevice       设备句柄
 * @param [out] pnErrorCode 当pnErrorCode不为nullptr时,输出错误码
 * @return 成功返回时间戳
 */
VZNLAPI unsigned long long VzNL_GetTimeStamp(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.80.2 添加获取相机 RGB 内参以及转换矩阵的接口(适用于静态相机)

```
/**
 * @brief 获取RGB 相机内参及转换矩阵
 * @detail 获取RGB相机内参数据及相机左目坐标系(3D坐标系)到RGB 图像坐标系的转换矩阵
 * @param [Out] fIntrinsic: 输出RGB相机内参: 3*3 矩阵;
 * @param [Out] fCalibMatrix: 输出转换矩阵: 4*4 矩阵;
 * @param [Out] pResolution: 输出RGB图像分辨率, MapData 尺寸与此相同;
 * @param [Out] ppMapData: 输出RGB图像Map数据, 用于进行校正图与原图的转换;
 * @return 成功返回0, 否则为其他错误码
 */
VZNLAPI int VZNL_GetRGBCalibParam(VZNLHANDLE hDevice, float fIntrinsic[9], float fCalibMatrix[16],
    SVzVideoResolution* pResolution, SVzNL2DPointShortU** ppMapData);
/**
 * @brief 释放获取的RGB Map数据
 * @detail 释放由GetRGBCalibParam() 接口获取的Map数据;
 * @param [In] ppMapData: 由GetRGBCalibParam() 接口输出的Map 数据;
 * @return 成功返回0, 否则为其他错误码
 */
VZNLAPI int VZNL_ReleaseRGBMapData(VZNLHANDLE hDevice, SVzNL2DPointShortU** ppMapData);
```

4.81 版本 3.6.20.112 (没有接口变化)

4.82 版本 3.6.20.113

4.82.1 添加 USB 串口继电器搜索，打开，通道控制等接口接口(适用于控制气动翻盖的相机)

```
/**
* @brief USB 串口继电器设备搜索
* @param pDeviceNum             返回搜索到的设备个数
* @param ppDeviceInfoList       返回搜索到的设备名称列表
*/
VZNLAPI int VzNL_SearchUSBSerialRelay(unsigned int* pDeviceNum, SVzSerialPortInfo** ppDeviceInfoList);

/**
* @brief 获取指定USB 串口继电器设备的配置信息
* @param pDeviceName        输入设备名称(搜索函数返回)
* @param pCfgPara           返回设备配置信息
*/
VZNLAPI int VzNL_GetUSBSerialRelayCfgPara(char* pDeviceName, SVzUSBSerialRelayCfgPara* pCfgPara);

/**
* @brief 查看指定USB 串口继电器设备是否打开
* @param pDeviceName        输入设备名称(搜索函数返回)
*/
VZNLAPI bool VzNL_IsUSBSerialRelayOpen(char* pDeviceName);

/**
* @brief 打开指定端口的USB 串口继电器设备，按配置参数打开指定设备
* @param pDeviceName        输入设备名称(搜索函数返回)
* @param sCfgPara           输入设备配置信息
*/
VZNLAPI int VzNL_OpenUSBSerialRelay(char* pDeviceName, SVzUSBSerialRelayCfgPara sCfgPara);

/**
* @brief 关闭指定端口的USB 串口继电器设备
* @param pDeviceName        输入设备名称(搜索函数返回)
*/
VZNLAPI int VzNL_CloseUSBSerialRelay(char* pDeviceName);

/**
* @brief 获取所有打开的USB 串口继电器设备端口列表
* @param pDeviceName        输入设备名称(搜索函数返回)
* @param ppDeviceInfoList   返回已打开设备的名称列表
*/
VZNLAPI int VzNL_GetOpenedUSBSerialRelayList(unsigned int* pDeviceNum, SVzSerialPortInfo** ppDeviceInfoList);
```

```
/**
* @brief 获取指定USB继电器通道当前开关状态
* @param pDeviceName        输入设备名称(搜索函数返回)
* @param nChannelIdx        输入继电器通道索引
* @param pState             返回该通道当前开关状态
*/
VZNLAPI int VzNL_GetUSBSerialRelayChannelOnOffState(char* pDeviceName, unsigned int nChannelIdx, bool* pState);

/**
* @brief 打开/关闭指定继电器通道，pState 返回操作后该通道状态
* @param pDeviceName        输入设备名称(搜索函数返回)
* @param bOn                输入需要设置的开关状态
* @param nChannelIdx        输入继电器通道索引
* @param pState             不为null时返回该通道当前开关状态
*/
VZNLAPI int VzNL_SetUSBSerialRelayChannelOnOff(char* pDeviceName, bool bOn, unsigned int nChannelIdx, bool* pState);

/**
* @brief 获取所有继电器通道开关状态: unsigned int* pState: 低位开始，每位代表一个通道；
* @param pDeviceName        输入设备名称(搜索函数返回)
* @param pState             返回该通道当前开关状态
*/
VZNLAPI int VzNL_GetUSBSerialRelayAllChannelOnOffState(char* pDeviceName, unsigned int* pState);

/**
* @brief 打开/关闭所有继电器通道，pState 返回操作后所有继电器通道状态(低位开始，每位代表一个通道)
* @param pDeviceName        输入设备名称(搜索函数返回)
* @param bOn                输入需要设置的开关状态
* @param pState             不为null时返回所有通道当前开关状态
*/
VZNLAPI int VzNL_SetUSBSerialRelayAllChannelOnOff(char* pDeviceName, bool bOn, unsigned int* pState);
```

4.83 版本 3.6.20.114 (没有接口变化)

4.84 版本 3.6.20.115 (没有接口变化)

4.85 版本 3.6.20.116

4.85.1 添加启用传图模式的接口(适用于所有相机)

```
/**
 * @brief 启用传图模式
 * @param hDevice 当前设备句柄
 * @param [in] bEnable 是否启用
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableTransPicMode(VZNLHANDLE hDevice, VzBool bEnable);
```

4.85.2 移除设置 RGBHDR 背景图叠加帧数的接口(适用于熔池相机)

```
/**
 * @brief 设置RGBHDR 背景图叠加帧数
 * @param hDevice 当前设备句柄
 * @param nFrames 背景图叠加帧数：[0~100]，设置为0和1 效果一样，都表示不用叠加
 * @return 正确返回0， 失败返回返回错误码
 */
VZNLAPI int VzNL_SetRGBHDRBGOverlayFrames(VZNLHANDLE hDevice, unsigned int nFrames);
VZNLAPI unsigned int VzNL_GetRGBHDRBGOverlayFrames(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.85.3 移除设置 RGBHDR 背景图叠加方法的接口(适用于熔池相机)

```
/**
 * @brief 设置RGBHDR 背景图叠加方法
 * @param hDevice 当前设备句柄
 * @param eMethod 背景图叠加方法
 * @return 正确返回0， 失败返回返回错误码
 */
VZNLAPI int VzNL_SetRGBHDRBGOverlayMethod(VZNLHANDLE hDevice, EVzRGBHDRBGOverlayMethod eMethod);
VZNLAPI EVzRGBHDRBGOverlayMethod VzNL_GetRGBHDRBGOverlayMethod(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.86 版本 3.6.20.117

4.86.1 添加通过双目校正图 2D 浮点坐标计算 3D 点坐标，根据当前设备内部参数计算的接口(适用于所有相机)

```
/**
 * @brief 通过双目校正图2D 坐标计算3D 点坐标，根据当前设备内部参数计算
 * @param hDevice        输入设备句柄
 * @param nPointCnt      输入点个数
 * @param p2DPointList   输入双目校正图2D 点坐标列表
 * @param p3DPointList   输出计算得到的3D 点列表
 */
VZNLAPI int VzNL_Cal3DPointsFromStereo2DPoints(VZNLHANDLE hDevice, unsigned int nPointCnt, SVzNL2DLRPoint* p2DPointList, SVzNL3DPoint* p3DPointList);
VZNLAPI int VzNL_Cal3DPointsFromStereo2DFPoints(VZNLHANDLE hDevice, unsigned int nPointCnt, SVzNL2DLRPointF* p2DPointList, SVzNL3DPoint* p3DPointList);
```

4.86.2 添加焊缝跟踪两套 ROI 尺寸设置的支持的接口(适用于所有相机)

```
/**
 * @brief 获取焊缝跟踪功能是否使用默认ROI尺寸
 * @detail 读取是否使用默认ROI 尺寸标志，用于判断当前相机ROI 是否满足焊缝跟踪要求
 * @param [in] hDevice   设备句柄
 * @return 返回标志
 */
VZNLAPI VzBool VzNL_IsTrackingScanUseDefaultROISize(VZNLHANDLE hDevice);
/**
 * @brief 设置焊缝跟踪功能是否使用默认ROI尺寸
 * @detail 设置焊缝跟踪功能是否使用默认ROI 尺寸，以便正确的对ROI 尺寸进行检查
 * @param [in] hDevice   设备句柄
 * @param [out] bUse     是否使用默认ROI尺寸：true：使用默认ROI尺寸；false：使用用户可配的ROI尺寸(支持配置)
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetTrackingScanUseDefaultROISize(VZNLHANDLE hDevice, bool bUse);

/**
 * @brief 获取焊缝跟踪默认ROI 尺寸
 * @param [in] hDevice   设备句柄
 * @param [in] pROIWidth  用于输出默认ROI 宽度
 * @param [in] pROIHeight 用于输出默认ROI 高度
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_GetTrackingScanDefaultROISize(VZNLHANDLE hDevice, unsigned int* pROIWidth, unsigned int* pROIHeight);

/**
 * @brief 获取焊缝跟踪用户ROI 尺寸
 * @param [in] hDevice   设备句柄
 * @param [in] pROIWidth  用于输出ROI 宽度
 * @param [in] pROIHeight 用于输出ROI 高度
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_GetTrackingScanUserROISize(VZNLHANDLE hDevice, unsigned int* pROIWidth, unsigned int* pROIHeight);
/**
 * @brief 设置焊缝跟踪功能用户ROI 尺寸
 * @param [In] nROIWidth  用户ROI 宽度，需要32像素对齐
 * @param [In] nROIHeight 用户ROI 高度，需要128像素对齐
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetTrackingScanUserROISize(VZNLHANDLE hDevice, unsigned int nROIWidth, unsigned int nROIHeight);
```

4.86.3 移除左目坐标原点功能的相关接口(适用于所有相机)

```
/**
 * @brief 开启/关闭以左目坐标为原点来计算点云方式
 * @param hDevice 当前设备句柄
 * @param [in] bEnable 是否启用
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_EnableLeftEyeOrigin(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnableLeftEyeOrigin(VZNLHANDLE hDevice, int* pnErrorCode);


/**
 * @brief 获取左目内参
 * @param hDevice 当前设备句柄
 * @param [in] dMatrix 左目内参矩阵 3*3 + 5*1
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_QueryLeftEyeCalibData(VZNLHANDLE hDevice, double dMatrix[14]);

/**
 * @brief 获取左目到3D点云的转换矩阵
 * @param hDevice 当前设备句柄
 * @param [in] dMatrix 左目到3D的转换矩阵 4*4
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_QueryLeftEye2StereoMatrix(VZNLHANDLE hDevice, double dMatrix[16]);

/**
 * @brief 获取3D点云到左目的转换矩阵
 * @param hDevice 当前设备句柄
 * @param [in] dMatrix 3D到左目的转换矩阵 4*4
 * @return 成功返回0，否则为错误码。
 */
VZNLAPI int VzNL_QueryStereo2LeftEyeMatrix(VZNLHANDLE hDevice, double dMatrix[16]);
```

4.87 版本 3.6.20.118

4.87.1 添加扫描传图信号功能接口(适用于所有相机)

```
/**
 * @brief 是否支持扫描传图信号功能
 * @param [in] hDevice 设备句柄
 * @param [in] pnErrorCode 错误码,为nullptr时不返回
 * @return 返回VzTrue表示支持，否则不支持。
 */
VZNLAPI VzBool VzNL_IsSupportScanModeExtSignal(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 使能扫描传图信号功能
 * @param hDevice 当前设备句柄
 * @param bEnable VzTrue启用 / VzFalse禁用
 * @return 正确返回0， 失败返回其他值
 */
VZNLAPI int VzNL_EnableScanModeExtSignal(VZNLHANDLE hDevice, VzBool bEnable);
VZNLAPI VzBool VzNL_IsEnableScanModeExtSignal(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.87.2 添加激光抽帧功能接口(适用于所有相机)

```
/**
 * @brief 是否支持抽帧功能
 * @param [in] hDevice      设备句柄
 * @param [in]  pnErrorCode 当pnErrorCode不为nullptr时,输出错误码
 * @return 返回是否支持，VzTrue支持 / VzFalse不支持
 */
VZNLAPI VzBool VzNL_IsSupportDivFrame(VZNLHANDLE hDevice, int* pnErrorCode);

/**
 * @brief 设置抽帧数[1,10]
 * @param [in] hDevice      设备句柄
 * @param [in] nDivFrame    抽帧数
 * @return 返回错误码
 */
VZNLAPI int VzNL_SetDetectDivFrame(VZNLHANDLE hDevice, unsigned int nDivFrame);
VZNLAPI unsigned int VzNL_GetDetectDivFrame(VZNLHANDLE hDevice, int* pnErrorCode);
```

4.87.3 添加设置熔池检测区域尺寸的接口(适用于熔池相机)

```
/**
 * @brief 设置熔池检测区域尺寸
 * @param [in] hDevice 设备句柄
 * @param [in] sROISize ROI区域尺寸
 * @return 返回错误码
 */
VZNLAPI int VzNL_SetMoltenPoolROISize(VZNLHANDLE hDevice, const SVzNLSize sROISize[4]);
VZNLAPI int VzNL_GetMoltenPoolROISize(VZNLHANDLE hDevice, SVzNLSize sROISize[4]);
```

4.88 版本 3.6.20.119

4.88.1 移除用户 ROI 尺寸配置功能接口(适用于所有相机)

```
/**
 * @brief 获取焊缝跟踪功能是否使用默认ROI尺寸
 * @detail 读取是否使用默认ROI 尺寸标志，用于判断当前相机ROI 是否满足焊缝跟踪要求
 * @param [in] hDevice   设备句柄
 * @return 返回标志
 */
VZNLAPI VzBool VzNL_IsTrackingScanUseDefaultROISize(VZNLHANDLE hDevice);
/**
 * @brief 设置焊缝跟踪功能是否使用默认ROI尺寸
 * @detail 设置焊缝跟踪功能是否使用默认ROI 尺寸，以便正确的对ROI 尺寸进行检查
 * @param [in] hDevice   设备句柄
 * @param [out] bUse     是否使用默认ROI尺寸：true：使用默认ROI尺寸；false：使用用户可配的ROI尺寸(支持配置)
 * @return 成功返回0，否则为其他错误码
 */
VZNLAPI int VzNL_SetTrackingScanUseDefaultROISize(VZNLHANDLE hDevice, bool bUse);
```

```
 /**
-* @brief 获取焊缝跟踪用户ROI 尺寸
-* @param [in] hDevice   设备句柄
-* @param [in] pROIWidth  用于输出ROI 宽度
-* @param [in] pROIHeight 用于输出ROI 高度
-* @return 成功返回0，否则为其他错误码
-*/
-VZNLAPI int VzNL_GetTrackingScanUserROISize(VZNLHANDLE hDevice, unsigned int* pROIWidth, unsigned int* pROIHeight);
-/**
-* @brief 设置焊缝跟踪功能用户ROI 尺寸
-* @param [In] nROIWidth  用户ROI 宽度，需要32像素对齐
-* @param [In] nROIHeight 用户ROI 高度，需要128像素对齐
-* @return 成功返回0，否则为其他错误码
-*/
-VZNLAPI int VzNL_SetTrackingScanUserROISize(VZNLHANDLE hDevice, unsigned int nROIWidth, unsigned int nROIHeight);
```

5 示例代码

由于 SDK 示例较多，在这里将不再占用篇幅，详见 SDK 的 Demo 文件